Frontispiece

TRAINING MANUAL

Coding With ScreenReaders

CONFIDENTIAL — For Internal Use Only

Copyright © 2026 Michael Ryan Hunsaker, M.Ed., Ph.D. Coding With Screen-Readers is licensed under a Creative Commons Attribution-NoNCommerical-NoDerivatives 4.0 International: https://creativecommons.org/licenses/by-nc-nd/4.0/
All rights reserved.
NO AI TRAINING: Without in any way limiting the author's rights under copyright, any use of this publication to "train" generative artificial intelligence (AI) technologies to generate text is expressly prohibited.

Contents

Introduction 9

Curriculum Package — Python, Java, and JavaScript 9

Package Contents 9

Curriculum Comparison 9

Accessibility Notes (All Languages) 10

Recommended Learning Paths 10

Credits 11

I Python Curriculum 1

Teacher's Guide: Introduction to Computer Science with Python 3

Unit 0: Setting Up Your Accessible Python Development Environment 43

Unit 1: Foundations of Computing and Getting Started with Python 50

Unit 2: Writing Simple Programs and Numeric Computations 56

Unit 3: Control Flow — Decisions and Loops 63

Unit 4: Data Structures — Lists and Dictionaries 69

Unit 5: User Input and `while` Loops 72

Unit 6: Functions 75

Unit 7: Classes and Object-Oriented Programming 79

Unit 8: Files and Exceptions 83

Unit 9: Testing and Data Analysis 87

Unit 10: Project-Based Learning — Building Text-Based Applications 91

Unit 11: Capstone Project — Accessible Personal Data Manager 97

II JavaScript Curriculum 101

Teacher's Guide: Introduction to Computer Science with JavaScript 105

Unit 0: Setting Up Your JavaScript Development Environment 137

Unit 1: Foundations — Getting Started with JavaScript 139

Unit 2: Numbers, Input, and the Math Object 143

Unit 3: Control Flow — Decisions and Loops 146

Unit 4: Data Structures — Arrays and Objects 150

Unit 5: Strings and String Methods 155

Unit 6: Functions 156

Unit 7: Classes and Object-Oriented Programming 159

Unit 8: Files, Modules, and Error Handling 162

Unit 9: Modules, Collections, and Testing 166

Unit 10: Project-Based Learning 171

Unit 11: Capstone Project — Personal Data Manager in JavaScript 176

III JAVA Curriculum 179

Teacher's Guide: Introduction to Computer Science with Java 183

Unit 0: Setting Up Your Java Development Environment 210

Unit 1: Foundations of Computing and Getting Started with Java 213

Unit 2: Writing Simple Programs and Input/Output 218

Unit 3: Control Flow — Decisions and Loops 220

Unit 4: Data Structures — Arrays and ArrayLists 224

Unit 5: Strings and String Processing 228

Unit 6: Methods (Functions) 230

Unit 7: Classes and Object-Oriented Programming 232

Unit 8: Files and Exception Handling 236

Unit 9: Collections, Data Analysis, and Testing 241

Unit 10: Project-Based Learning 246

Unit 11: Capstone Project — Personal Data Manager in Java 250

Dedication

To the countless individuals who have been told they lack the capability to code, yet remain determined to learn and create through accessible coding practices, this book is dedicated.

Epigraph

The power of the Web is in its universality. Access by everyone regardless of disability is an essential aspect

— Tim Berners-Lee, Inventor of the World Wide Web

Introduction

Curriculum Package — Python, Java, and JavaScript

This package contains three complete introductory programming curricula, each adapted for visually impaired students who use screen readers.

Package Contents

```
curriculum/
|--- README.md                          (this file)
|--- python/
|   |--- unit00-setup.md
|   |--- unit01-foundations.md
|   |--- unit02-numeric-computations.md
|   |--- unit03-control-flow.md
|   |--- unit04-06-data-functions.md
|   |--- unit07-09-oop-files.md
|   +--- unit10-11-projects-capstone.md
|--- java/
|   |--- README.md
|   |--- unit00-01-setup-foundations.md
|   |--- unit02-04-io-control-arrays.md
|   |--- unit05-08-strings-methods-oop-files.md
|   +--- unit09-11-collections-projects.md
+--- javascript/
    |--- README.md
    |--- unit00-01-setup-foundations.md
    |--- unit02-04-io-control-data.md
    |--- unit05-08-strings-functions-oop-files.md
    +--- unit09-11-modules-collections-projects.md
```

Curriculum Comparison

Feature	Python	Java	JavaScript
Difficulty	Beginner	Intermediate	Beginner-Intermediate
Typing	Dynamic	Static	Dynamic
Syntax	Minimal	Verbose	Moderate
Best For	First language, data science, AI	Enterprise, Android, CS theory	Web development, full-stack
Terminal Tool	`python3 file.py`	`javac` + `java`	`node file.js`
Testing	`unittest`	JUnit 5	Jest
JSON	`json` module	Gson library	Built-in `JSON` object
Input	`input()`	`Scanner`	`readline` module

Accessibility Notes (All Languages)

All three curricula share these accessibility principles:

1. **Screen Reader Optimized Mode in VS Code:** Press `Ctrl + Shift + P` -> "Toggle Screen Reader Optimized".
2. **Indentation Announcement:** Configure your screen reader to announce indentation changes — critical for Python's block structure; helpful for Java and JavaScript too.
3. **Error Navigation:** Use `F8` / `Shift+F8` to jump between errors without scanning visually.
4. **All Output is Text-Based:** No graphical output required for any core exercises.
5. **Terminal First:** All programs run from the integrated terminal, which is accessible with screen readers.

Recommended Learning Paths

Complete Beginner Start with **Python** -> then **JavaScript** (many concepts transfer directly).

Web Developer Track Start with **JavaScript** -> deepen with Node.js and frameworks.

Computer Science Theory Track Start with **Python** -> move to **Java** for stronger OOP and typing foundations.

All-Rounder **Python** (learn concepts) -> **Java** (learn strict typing and OOP) -> **JavaScript** (apply to the web).

Credits

Curriculum designed for accessibility-first computer science education. Adapted from:

- Matthes, E. (2023). *Python Crash Course* (3rd ed.). No Starch Press.
- Zelle, J.M. (2017). *Python Programming: An Introduction to Computer Science* (3rd ed.). Franklin, Beedle & Associates.
- Eck, D.J. (2022). *Introduction to Programming Using Java* (9th ed.). [Free at math.hws.edu/javanotes]
- MDN Web Docs. [developer.mozilla.org]

Original Python curriculum by Michael Ryan Hunsaker, M.Ed., Ph.D. — Davis School District.

Part I

Python Curriculum

Teacher's Guide: Introduction to Computer Science with Python

For Students with Visual Impairments — Complete Instructor Edition

---

Table of Contents

1. Philosophy and Foundational Principles
2. Understanding Your Students
3. Classroom Setup and Technology Configuration
4. Universal Spoken Code Conventions
5. Unit 0 — Setup and Environment
6. Unit 1 — Foundations and Getting Started
7. Unit 2 — Numeric Computations and I/O
8. Unit 3 — Control Flow, Decisions, and Loops
9. Unit 4 — Data Structures: Lists and Dictionaries
10. Unit 5 — User Input and While Loops
11. Unit 6 — Functions
12. Unit 7 — Classes and Object-Oriented Programming
13. Unit 8 — Files and Exceptions
14. Unit 9 — Testing and Data Analysis
15. Unit 10 — Project-Based Learning
16. Unit 11 — Capstone Project
17. Assessment Strategies
18. Supporting Diverse Needs Within the Visually Impaired Population
19. Teaching AI Tools Responsibly
20. Alternative and Supplementary Resources
21. Quick Reference Sheets

---

1. Philosophy and Foundational Principles

Why Python for Blind Students? Python is widely recognized as one of the most accessible first programming languages for visually impaired learners for several concrete reasons:

- **Text-based I/O is the default.** Unlike languages that push beginners toward GUI frameworks or graphical plotting, Python's most natural outputs are plain text to the terminal — fully accessible to any screen reader.
- **Indentation enforces structure.** While indentation introduces a learning challenge (addressed in detail below), it also means that code read aloud by a screen reader carries structural meaning in the form

of announced indent levels — something curly-brace languages do not provide.

- **Error messages are descriptive.** Python's tracebacks include the file name, line number, error type, and a message. A screen reader user can navigate directly to the problem without visual scanning.
- **The standard library is enormous.** Python ships with modules for file I/O, JSON, math, random numbers, CSV parsing, unit testing, and more — reducing the complexity of dependency management in a learning context.
- **The REPL is an accessible sandbox.** Python's interactive interpreter provides immediate spoken feedback for any expression, enabling rapid experimentation without the overhead of creating and running files. [1]

Indentation announcement must be configured in the screen reader before the first Python lesson—not as an afterthought. See Section 3 (VS Code Configuration) for step-by-step setup for NVDA, JAWS, VoiceOver, and Orca.

Core Pedagogical Commitments **Commit to ears-first instruction.** Every demonstration, every example, every debugging session should be conducted as if the teacher is also using a screen reader. Ask yourself before every lesson: "right now?"

[1] The REPL is the single most accessible Python environment for early learners: no file navigation, no run command, immediate audio response. Start every new concept here before moving to files.

Commit to explicit navigation. Sighted students can glance at code and immediately locate structures. Blind students must navigate deliberately using keyboard shortcuts and screen reader commands. Teach navigation skills as primary content, not auxiliary support.

This self-check is the single most useful habit an instructor can build. Apply it to every demo, every error message, every output format. If the answer is unclear, redesign the demonstration.

Commit to verbal precision. Vague instructions like "look at the next line" or "you can see here that…" create invisible barriers. Replace all visual language with navigation language: "Move down two lines," "Jump to line 14 with Ctrl+G," "Navigate to the end of the file with Ctrl+End." [2]

[2] Audit your own speech during the first three lessons. Most sighted instructors use spatial language constantly without noticing: "you can see," "look here," "over on the right." Recording a lesson and reviewing it is the fastest way to identify and correct these habits.

Commit to error-positive culture. For a sighted student, an error is a red underline they can ignore or address at leisure. For a screen reader user, an unaddressed error is noise that clutters the output of every subsequent navigation. Treat errors as immediate priorities and build the habit of reading and resolving them before moving on.

Commit to redundancy. Provide concepts in multiple formats — spoken explanation, written pseudocode, physical analogy, and code. Blind students cannot fall back on visual re-reading; redundant representation builds retention.

---

2. Understanding Your Students

The Heterogeneity of Visual Impairment "Visually impaired" encompasses a wide spectrum. Your students may include:

Profile	Implications for Teaching
Congenitally blind (blind from birth)	May have no visual spatial metaphors. Analogies must be tactile, auditory, or conceptual. May be faster with screen readers from years of experience.
Late-onset blindness (lost vision after sighted years)	May retain strong visual mental models. Can use these as bridges but should not be required to. May still be learning screen reader skills.
Low vision (functional but impaired vision)	May use a combination of magnification and screen reading. May be able to see some screen content but cannot rely on fine detail. Do not assume screen-only or vision-only.
Color blindness	Primarily relevant for syntax highlighting. Ensure themes are not relied upon for meaning.
Progressive conditions	Vision may change during the course. Build habits that remain useful at any level of vision.

Never assume a student's profile without asking. Some students are private about their diagnoses; frame questions around what tools they use and what works best, not what their condition is.

Screen Reader Literacy Varies Some students arrive with years of fluent screen reader experience. Others are relatively new to their screen reader and may still be developing efficiency. Assess screen reader fluency separately from programming aptitude — a student who struggles to navigate a file is not struggling with Python; they need more time with their tool.

Suggest that students spend time with their screen reader in non-programming contexts (browsing documentation, reading formatted text) before or alongside the first programming units.

Braille Display Users Some students use a refreshable braille display in addition to or instead of audio output. Key considerations:

- Braille displays have limited width (typically 40 or 80 cells). Long lines of code may need to be panned across. Encourage keeping line length under 79 characters (PEP 8 standard) for this reason. [3] [4]
- Code rendered in braille uses Grade 1 (uncontracted) braille because contractions would misrepresent code characters. Confirm this setting is active.
- Navigation with a braille display typically involves routing cursor buttons — physical buttons above each braille cell that move the cursor to that position. Students using these may navigate differently than audio-only users.
- Braille display users often prefer lower verbosity audio output since they read braille silently. Do not require audio announcements as the only feedback mechanism.

[3] PEP 8'slength under 79 characters (PEP 8 standard) for this reason

Accessibility Tip: Set `editor.rulers: [79]` in VS Code settings to display a vertical guide at column 79. With Screen Reader Optimized Mode on, VS Code will announce when the cursor crosses this boundary, giving auditory feedback about line length without visual scanning.

[4] PEP 8's 79-character line limit was originally a terminal width convention, but it maps almost perfectly to an 80-cell braille display. Frame this limit to students as a practical accessibility constraint, not an arbitrary style rule.

Confirm Grade 1 (uncontracted) braille is active *before* the first coding session. A student unknowingly reading contracted braille will misread operators, brackets, and keywords.

Students Who Are Also Deaf-Blind Students who are deaf-blind rely exclusively on braille for all information. Audio feedback from the screen reader is not available. Considerations:

- All instructional communication must be via tactile means, text-based communication, or tactile sign language interpreters.
- Braille output of code is the primary interface. Pay special attention to line length and symbol representation.
- Terminal output must be navigable via braille display — avoid programs that produce output faster than a braille display can be read.
- this guide.

The HKNC (Helen Keller National Center) and Perkins School for the Blind both publish guidance on programming education for deafblind learners. `hknc.org` and `perkins.org`.

---

3. Classroom Setup and Technology Configuration

Physical Environment

- Ensure the student's workstation does not have excessive background noise that interferes with screen reader audio. Headphones are often preferable in a shared classroom.
- Position the workstation so the teacher can sit beside (not in front of) the student, allowing them to listen to the screen reader output together.
- If a classroom assistant or orientation and mobility specialist is present, coordinate roles in advance. The programming teacher leads on programming content; the VI specialist leads on accessibility tool configuration.

VS Code Configuration Checklist Work through this checklist with the student on day one:

Screen Reader

- ☐ Screen Reader Optimized Mode ON (Ctrl+Shift+P -> Toggle Screen Reader Optimized)
- ☐ Accessibility help panel reviewed (Shift+Alt+F1)

Without Screen Reader Optimized Mode, VS Code suppresses many accessibility announcements to reduce "noise" for sighted users. This mode must be ON for the editor to be functional with a screen reader.

Editor Settings (Ctrl+,)

- ☐ editor.lineNumbers = "on"
- ☐ editor.renderWhitespace = "all" or "boundary"
- ☐ editor.wordWrap = "on" (for long lines)
- ☐ workbench.reduceMotion = "on" (reduces distracting animations)
- ☐ editor.tabSize = 4
- ☐ editor.insertSpaces = true (spaces not tabs for Python) [5]

[5] Mixing spaces and tabs is the most common source of `IndentationError` for new Python students. Setting `insertSpaces = true` at the editor level prevents the problem entirely and should be verified on day one.

Screen Reader Indentation (in screen reader settings, not VS Code)

- ☐ NVDA: Document Formatting -> Indentation -> "Say all" or "Beeps"
- ☐ JAWS: Configure speech for VS Code environment
- ☐ VoiceOver: Verbosity -> Text -> announce indentation
- ☐ Narrator: No built-in indent announcement; use Beep plugin or extension

Extensions Installed

- ☐ Python (Microsoft)
- ☐ Pylance
- ☐ Python Indent (for consistent auto-indentation)
- ☐ GitLens or Git Graph (for version control visibility)
- ☐ Error Lens (inline error annotations — screen-reader-readable)

Keyboard Shortcuts Practiced

- ☐ Ctrl+G — Go to line
- ☐ F8 — Next error
- ☐ Shift+F8 — Previous error
- ☐ Ctrl+Shift+M — Problems panel Ctrl+Shift+M — Problems panel
- ☐ Shift+Alt+F2 — Accessible View
- ☐ Ctrl+Shift+P — Command Palette
- ☐ Ctrl+‘ — Toggle terminal
- ☐ Ctrl+Shift+E — Explorer
- ☐ Ctrl+Shift+O — Go to symbol (function/class navigation)
- ☐ Alt+F12 — Peek Definition
- ☐ F12 — Go to Definition
- ☐ Shift+F12 — Find all References

Accessibility Tip: The Problems panel (`Ctrl+Shift+M`) shows every error and warning as a list you can navigate with arrow keys. Use `F8` to jump to the next error location in the editor. This is the most reliable way to find errors without visually scanning the code.

Terminal Configuration

- **Windows Terminal** (preferred over legacy cmd.exe on Windows) has better accessibility with NVDA and JAWS.
- **NVDA + Windows Terminal:** Enable "Use UI Automation" in NVDA settings for better terminal reading.
- **VoiceOver + macOS Terminal:** Works well by default. Use `VO+Shift+Down` to interact with terminal content.
- **Orca + Linux terminal:** Orca reads terminal output well. Use `Orca+T` (flat review) to re-read recent output.
- Configure the terminal font to a monospace font at a size comfortable for any residual vision.

Python Virtual Environment Setup for Accessibility

```
# Create environment with a consistent name the student will always type
python3 -m venv .venv

# Activation --- save this as a text file the student can open and copy from
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate

# Verify with a spoken command
python3 --version
pip list
```

Consider creating a startup script (`start.bat` on Windows, `start.sh` on macOS/Linux) that activates the environment and opens VS Code. Students can run one command to get their environment ready, reducing setup cognitive load.

Store the activation command in a plain `.txt` file the student can open and copy from. Never require them to recall multi-step activation paths from memory during the first weeks of the course.

---

4. Universal Spoken Code Conventions Establish this vocabulary on the first day and reinforce it consistently. When a student is reading code aloud (for debugging, peer review, or just narrating their work), they should use these terms without hesitation.

Symbol Pronunciation Table

Written Symbol	Spoken Form	Notes
`=`	"gets"	Assignment only
`==`	"equals equals" or "is equal to"	Comparison
`!=`	"not equal to" or "bang equals"	
`>=`	"greater than or equal to"	
`<=`	"less than or equal to"	
`>`	"greater than"	
`<`	"less than"	
`+=`	"plus equals" or "add and assign"	
`-=`	"minus equals"	
`*=`	"times equals"	
`**`	"to the power of" or "double star"	
`//`	"floor divide" or "integer divide"	
`%`	"modulo" or "remainder"	
`()`	"open paren … close paren"	Always say both
`[]`	"open bracket … close bracket"	
`{}`	"open brace … close brace"	
`:`	"colon"	End of if/for/def/class line
`"text"`	"quote text quote"	
`'text'`	"single-quote text single-quote"	
`f"..."`	"f-string"	Mention the f prefix

Written Symbol	Spoken Form	Notes
`{var}` inside f-string	"open brace var close brace"	
`#`	"hash"	Followed by comment text
`\n`	"backslash n" or "newline"	
`\t`	"backslash t" or "tab"	
`_`	"underscore"	In names: "my underscore name"
`__init__`	"dunder init"	Double-underscore methods
`__str__`	"dunder str"	
`.`	"dot"	In `math.sqrt`, say "math dot sqrt"
`->`	"arrow"	Type hints
`*args`	"star args"	
`**kwargs`	"double-star kwargs"	
`None`	"None" (capital N)	Distinct from the word "none"
`True` / `False`	"True" / "False" (capital)	
`import`	"import"	
`from x import y`	"from x import y"	Read as English sentence

Reading a Line of Code Aloud

Reading a Line of Code Aloud Teach students to read full lines this way, left to right, then explain:

```
total_price = unit_price * quantity + tax
```

"total underscore price gets unit underscore price times quantity plus tax"

Then explain: *"This line calculates the total price by multiplying the unit price by the quantity, then adding tax, and stores the result in total_price."*

The *spoken* distinction between `=` ("gets") and `==` ("equals equals") is one of the most important conventions to establish early. Many bugs in beginner Python trace directly to mixing these up. Consistent spoken vocabulary prevents this class of error.

Accessibility Tip: Use f-strings (`f"Value: {x}"`) rather than string concatenation (`"Value: " + str(x)`) for output. F-strings are read more naturally by screen readers and make the intent of each print statement immediately clear when navigating code.

The two-pass protocol mirrors how experienced blind developers actually work: one pass for syntax verification, a second for semantic reasoning. Building this habit early prevents the tendency to skip structural details.

---

5. Unit 0 — Setup and Environment

Lesson 0.1: Installing Python and VS Code

Before This Lesson Survey the student's current technology setup:

- What screen reader do they use, and what version?
- Do they use a braille display?
- What operating system?
- Have they used a terminal before?
- Do they have administrator access to their computer?

Confirm administrator access before scheduling this lesson. Installation without admin rights will fail.

Accessibility Tip: Always verify Python is on PATH immediately after installation by opening a fresh terminal and running `python3 --version`. A PATH failure produces a "command not found" error that is easy to miss if the student navigates away from the terminal before reading the output.

Instructional Cues Walk through every installer step verbally as the student performs it. Do not say "click the next button" — say "Tab until you hear 'Next button,' then press Enter." Treat this as teaching screen reader navigation, not just software installation.

before anything else: `python3 --version`. If this fails, stop and address the PATH issue before proceeding. A student who continues without Python on PATH will encounter mysterious failures throughout the course.

PATH failures are the most common first-day blocker. Symptoms: `python3` gives "command not found" or opens the Microsoft Store on Windows. Fix: rerun installer and explicitly check "Add Python to PATH," or manually add the Python directory to the system PATH via Settings.

Non-Obvious Accessibility Strategies

- **Windows Defender SmartScreen:** On Windows, running a downloaded `.exe` from Python.org may trigger a SmartScreen dialog. This dialog can be navigated with Tab and Enter but is not always announced immediately by NVDA. Tell the student what to expect before they run the installer.
- **macOS Gatekeeper:** Downloaded packages may require right-click -> Open to bypass security. On a screen reader, right-click is VO+Shift+M (VoiceOver) or App Key (NVDA with keyboard access).
- **Linux package managers as an alternative:** For Linux students, `sudo apt install python3 python3-pip python3-venv` is often more reliable than the Python.org download and is fully scriptable.
- **Portable Python for school computers:** Some schools restrict software installation. reasonably accessible. [6]

[6] GitHub Codespaces provides a full VS Code environment in the browser with no local installation. It inherits all of VS Code's screen reader support and is often more reliable in locked-down school IT environments than local installs.

Alternative Installation Methods

Method	Accessibility	Best For
Python.org installer	High — standard dialog flow	Most installations
Chocolatey (Windows)	High — command-line, screen-reader friendly	Windows power users
Homebrew (macOS)	High — command-line	macOS users comfortable with terminal
apt/dnf/pacman (Linux)	High — command-line	Linux users
Replit.com	Medium — web-based, some accessibility gaps	No local install possible
Thonny IDE	Medium — simpler UI, bundled Python	Beginners who struggle with VS Code

On Thonny: Thonny is a Python IDE specifically designed for beginners. It has a simpler interface than VS Code and is sometimes more accessible for students who are new to both screen readers and programming. The trade-off is fewer advanced accessibility features long-term.

Students who start on Thonny sometimes resist the VS Code migration. Frame it as a deliberate progression: Thonny is a learning environment, VS Code is a professional one. The transition is part of the curriculum.

Thought Experiments

1. *"Why do you think the Python installer has a checkbox called 'Add Python to PATH'? What is a PATH, and what would happen if Python weren't on it? Think about it like a phonebook — your operating system looks up programs by name. If Python isn't in the phonebook, what happens when you type* `python3`*?"*

2. *"We're installing two things: Python and VS Code. What role does each one play? If you had only Python and no VS Code, could you still write programs? How? What would you lose?"*

3. *"Virtual environments keep each project's dependencies separate. Imagine two school projects — one requiring a specific edition of a textbook that contradicts the edition another class uses. What would happen if you tried to use both editions at the same time? How does a virtual environment solve the programming equivalent of this problem?"*

4. *"Why do we verify every installation immediately after completing it? What could go wrong between downloading something and confirming it works? Name three things that could silently fail."*

Lesson 0.2: Configuring Your Screen Reader for VS Code

Instructional Cues Before showing any configuration screens, do an unguided exploration: "Open VS Code, navigate around with only your keyboard and screen reader, and come back in ten minutes. Tell me what you found, what worked, what was confusing." This builds genuine familiarity and identifies that student's specific challenges.

Map each configuration step to a problem the student already identified. This makes the configuration feel purposeful rather than arbitrary.

Screen Reader Specific Guidance **NVDA users:**

- Enable "Use UI Automation to access VS Code" in NVDA preferences (Tools -> Manage Plugins, or via NVDA+N -> Preferences -> Settings -> Advanced -> Use UI Automation where available). This dramatically improves reading accuracy in VS Code.
- Set "Report dynamic content changes" to "All" to hear error highlights and IntelliSense popups.
- Configure Browse Mode to be OFF by default in VS Code — programming requires forms mode (interactive mode) at all times.
- Speech rate: many experienced NVDA users run at 70–90% of maximum speed. For new students, 40–60% allows better comprehension of code read aloud.

JAWS users:

- JAWS has VS Code-specific scripts maintained by the JAWS community. Check `FreedomScientific.com` and GitHub for the latest VS Code JAWS scripts.
- PlaceMarkers in JAWS can mark frequently visited locations in a file — teach `Ctrl+Win+K` to set and `Ctrl+Win+J` to jump.
- JAWS Verbosity settings: set "Announce line numbers" to ON for code files.

VoiceOver (macOS) users:

- VoiceOver interacts with VS Code via the macOS Accessibility API. Ensure VS Code is not run with elevated permissions (which can break accessibility API access).
- Use `VO+Shift+Down` to enter interactive elements (the editor, terminal), then navigate normally.
- Enable "Speak text under mouse" in VoiceOver Utility -> Verbosity if the student uses residual vision with a mouse.
- VoiceOver's "item chooser" (VO+I) can provide a quick list of interactive elements in the current window.

Windows Narrator users:

- Narrator is less feature-rich than NVDA or JAWS for programming. It works but may require more keystrokes.
- Narrator scan mode must be OFF in VS Code for normal keyboard input to work. Toggle with Caps Lock+Space.
- students using Narrator Consider recommending the free NVDA screen reader as an upgrade for students using Narrator, as NVDA's VS Code support is significantly better.

NVDA is free, open-source, and maintained by NV Access. Download at `nvaccess.org`. Its VS Code support is substantially better than Narrator's for programming tasks.

Orca (Linux) users:

- Orca works well with VS Code on Linux via the AT-SPI2 accessibility interface.
- Use `Orca+H` for "say all" from cursor position.
- Ensure the `at-spi2-core` package is installed: `sudo apt install at-spi2-core`.
- Orca's "flat review" mode (`Orca+KP_Enter` or `Orca+Insert+KP_Enter`) is useful for reading lines that are not focus-accessible.

Indentation Configuration — Step by Step **Test procedure:**

If a student's screen reader is not announcing indentation, do not proceed. Every Python control structure depends on indentation. A student writing Python without auditory indentation feedback is working blind in both senses.

1. Create a temporary file `test_indent.py`.
2. Write:

```
if True:
    print("indented")
print("not indented")
```

3. Navigate line by line with Down arrow.
4. The screen reader should announce the indent level on line 2 before reading its content.
5. On line 3, the indent should return to zero.

If announcement is not happening, troubleshoot screen reader settings before proceeding. If announcement is not happening, troubleshoot screen reader settings before proceeding.

Accessibility Tip: Test indentation announcement with a two-line file before writing any real Python. If the screen reader does not announce indent level changes, every control structure will be structurally silent. This is a hard blocker—do not proceed until it is resolved.

Thought Experiments

1. *"A sighted programmer can look at a Python file and immediately understand its structure from indentation — like an outline. How does your screen reader translate that same 2D visual structure into a 1D audio stream? What information might be lost in that translation?"*
2. *"VS Code has hundreds of keyboard shortcuts. No sighted programmer uses all of them — they use the 15 or 20 that matter for their workflow. If you could only keep 10 shortcuts, which 10 would make the biggest difference for writing Python? Why?"*
3. *"The Accessible View (Shift+Alt+F2) converts complex visual elements into flat text. What kind of information in VS Code might require this conversion? Can you think of a scenario where the 'visual' presentation of something would be unusable without translation?"*

Lesson 0.3: Setting Up Your VS Code Workspace

Alternative Workspace Tools If VS Code proves too complex initially, consider these alternatives as temporary or permanent solutions:

Tool	Platform	Accessibility Notes
IDLE (bundled with Python)	All	Very simple UI. Limited features but highly keyboard accessible. Good for first programs.
Thonny	All	Beginner-focused. Has a "simple mode." Some screen reader users find it more approachable.
Mu Editor	All	Designed for education. Simpler than VS Code. Python 3 only.

Tool	Platform	Accessibility Notes
Jupyter Notebook (terminal)	All	Text-based notebook. Accessible for structured exploration.
Braille Note Apex/Touch	Device	For students using these notetakers, Python can be run via their built-in terminals.
Replit	Web	No install required. Accessibility varies; test before recommending.
NVDA Remote + VS Code	Windows	Student uses their own screen reader on their own machine while teacher controls a shared session.

Thought Experiments

1. *"Programmers organize their code into folders by project. What happens if every programmer at a company kept all their files in one shared folder with no organization? How does this analogy apply to organizing your own Python projects?"*

2. *"VS Code Extensions add features to the editor the same way apps add features to a phone. If you could design one VS Code extension specifically for blind Python programmers that doesn't exist yet, what would it do?"*

Lesson 0.4: Python Environments and Terminal Usage

Terminal Proficiency as a Prerequisite Before writing any Python, ensure the student can fluently:

- Navigate to a directory with `cd`
- List files with `ls` (macOS/Linux) or `dir` (Windows)
- Create a directory with `mkdir`
- Print the current directory with `pwd` (macOS/Linux) or `cd` with no arguments (Windows)
- Use Tab completion for filenames
- Use Up/Down arrow for command history
- Use Up/Down arrow for command history
- Use `Ctrl+C` to stop a running program

If these are not automatic, spend additional time on terminal basics. Terminal fluency is foundational to everything else.

Accessibility Tip: Tab completion is a screen reader user's best friend in the terminal. It reduces typing, eliminates spelling errors in paths, and produces a spoken confirmation of the completed name. Teach it as the default way to enter any filename or directory name.

Alternative Terminals for Accessibility

Terminal	Platform	Screen Reader Compatibility
Windows Terminal	Windows	Excellent with NVDA/JAWS (use UI Automation mode)
cmd.exe	Windows	Good; legacy but very reliable
PowerShell	Windows	Good with NVDA; some JAWS quirks
macOS Terminal	macOS	Excellent with VoiceOver
iTerm2	macOS	Good; more features, slightly more complex
GNOME Terminal	Linux	Good with Orca
Konsole	Linux	Good with Orca
Terminator	Linux	Good; split-pane support

Thought Experiments

1. *"Virtual environments isolate each project's dependencies. Why would this matter for a professional programmer with 20 active projects? What could go wrong without them?"*
2. *"When you type* `python3 hello.py` *and press Enter, what is the operating system doing? What is Python doing? Walk through it step by step as if explaining to someone who has never used a computer."*
3. *"*`pip install requests` *downloads and installs code written by other people. What are the risks of running code written by strangers? What does PyPI (the Python Package Index) do to reduce these risks?"*

---

6. Unit 1 — Foundations and Getting Started

Lesson 1.1: Welcome to Computer Science

Instructional Cues Open with **the "Human Computer" activity** — for blind students. The student [7] gives verbal, step-by-step instructions for a simple physical task. The teacher follows them literally, making no inferences. This demonstrates the core challenge of programming: computers are perfectly literal and do not fill in gaps.

Suggest tasks that connect to the student's daily experience:

- Instructions for making tea or coffee
- Instructions for navigating from the classroom door to their desk
- Instructions for finding a contact in their phone using a screen reader

[7] The Human Computer activity works especially well for blind students because they already have deep experience giving precise verbal instructions to navigate the physical world. Orient-and-mobility training is, in a real sense, an exercise in algorithmic thinking.

After the activity, debrief: *"Where did your instructions assume I could see? Where did they assume I already knew something? A computer makes neither assumption."*

Connecting Computing to Accessibility Technology Begin with what the student already knows:

- Their screen reader is software. It runs on hardware. It was written by programmers. It converts visual information (text rendering) into audio output.
- Ask: "What are the inputs to your screen reader? What are the outputs? What would happen if a website used images of text instead of real text — how would that change what your screen reader can do?"
-

ARIA (Accessible Rich Internet Applications) is a W3C specification that adds semantic meaning to web elements. Poor ARIA implementation is a leading cause of inaccessible websites. A blind student who learns to code is uniquely positioned to understand and advocate for correct ARIA usage.

Thought Experiments

1. *"Computer science is described as the study of problem-solving, not just programming. What non-programming problems have you solved using computational thinking — breaking a problem down into steps, looking for patterns, and abstracting the details? Describe one example from daily life."*

2. *"Your screen reader converts text to audio in real time. What 'algorithm' do you think it uses? What decisions does it make — when to pause, how to handle punctuation, what to do with symbols? What would a flowchart of those decisions look like?"*

3. *"Early computers could only run one program at a time. Modern computers run hundreds simultaneously. Without changing the physical hardware, what concept makes this possible? (Hint: the answer is software — specifically, an operating system.)"*

4. *"Software has often been described as 'making the hardware do something useful.' But hardware without software is just metal and silicon. In what sense is software more important than hardware? In what sense is hardware more important?"*

5. *"Accessibility technology like screen readers, braille displays, and audio description didn't exist when computers were first invented. Who decided these should be built? What does this history tell you about who gets to shape what technology does?"*

Additional Resources for This Lesson

- **"The Computer Boys Take Over" by Nathan Ensmenger** — accessible history of how programming became a profession.

- **"Technically Wrong" by Sara Wachter-Boettcher** — explores how biases in tech affect users, including disabled users. Good for older students.
- **American Foundation for the Blind (AFB) CareerConnect** — profiles of blind professionals in STEM fields, including software engineers. Good for motivation and role models.
- **"Haben: The Deafblind Woman Who Conquered Harvard Law" by Haben Girma** — autobiography of a deafblind lawyer and disability rights advocate. Haben's work directly involves technology access.
- **Code.org's "How Computers Work" video series** — audio-described versions available. Covers hardware and software basics accessibly.

Lesson 1.2: Your First Python Program

Instructional Cues **Before any code:** Ask the student to predict. "If I write `print("Hello, World!")` and run it, what do you think will appear in the terminal? Where exactly will it appear?" Let them answer, then verify together.

Navigate to the error message together. Establish the error-reading protocol:

1. Terminal output appears after the code.
2. Read from the bottom of the error upward — the bottom is the most specific message.
3. Line number tells you where to go. Use `Ctrl+G`.
4. Error type (`SyntaxError`, `NameError`) narrows the category of problem.

Reading errors from the *bottom up* is counter-intuitive but essential. Python's traceback lists the call chain from outermost to innermost, so the most specific and actionable message is always at the bottom. Establish this habit in the first session.

Before creating any files, start with `python3` in the terminal and type `print("Hello, World!")` directly. [8] The immediate audio feedback of the REPL establishes cause and effect more directly than the file -> run -> find output workflow.

Accessibility Tip: Learn the six most common Python error types and what each means: `SyntaxError` (malformed code), `NameError` (undefined variable), `TypeError` (wrong type), `ValueError` (right type, bad value), `IndexError` (list index out of range), `KeyError` (missing dictionary key). Recognising the type before reading the message cuts debugging time in half.

Accessibility Tip: Open the Python REPL by typing `python3` and pressing Enter. Every expression you type is immediately evaluated and spoken. Type `exit()` or press `Ctrl+D` to quit. The REPL is the fastest way to experiment with Python syntax without creating and running a file.

[8] `python3` launches the REPL. On Windows, the command may be `python` (without the 3) depending on installation. Verify which command works on the student's system and stick to it consistently.

Common First Errors and How to Explain Them

Error	Likely Cause	Explanation for Student
`SyntaxError: unexpected EOF while parsing`	Missing closing `"`, `)`, or `]`	"Python reached the end of the file before it expected to. It was waiting for something — probably a closing character."
`NameError: name 'print' is not defined`	Capital P in `Print`	"Python is case-sensitive. `Print` and `print` are completely different names. Only `print` (lowercase) is the built-in function."

Error	Likely Cause	Explanation for Student
`IndentationError`	Extra space before first line	"Python's first rule: top-level code cannot be indented. Every line that is not inside a function or class must start at column 0."
`SyntaxError: invalid syntax` at `:`	Using = instead of == in a comparison	"The assignment operator = doesn't belong inside an if condition. For comparison, you need the double equals ==."

Thought Experiments

1. *"`print()` produces output in the terminal. What would a program be like with NO output at all? Could such a program still do something useful? (Hint: think about programs that read files, send network requests, or modify a database.)"*
2. *"Python is an interpreted language — each line is read and executed one at a time. Other languages are compiled — the entire program is translated before any of it runs. How does this affect the kind of errors you encounter, and when you encounter them?"*
3. *"The `#` symbol starts a comment — text that Python ignores entirely. Why do comments exist? If the computer ignores them, who are they for? Could you write good programs with no comments? Should you?"*
4. *"'Hello, World!' is the traditional first program in almost every programming language. Why do you think this tradition exists? What does this simple program prove about a working environment?"*

Alternative Resources for This Lesson

- **"Invent Your Own Computer Games with Python" by Al Sweigart (free online):** Text-based games that produce only terminal output — perfectly accessible. Chapter 1 is a good companion to this lesson.
- **CS Unplugged — "Parity Magic" activity:** Teaches binary representation and algorithms without a computer. Can precede or follow this lesson.
- **Python REPL via iOS/Android with screen reader:** Several apps provide a Python REPL on mobile. For students whose phone screen reader is more familiar than their computer setup, this may be a comfortable way to experiment.

Lesson 1.3: Variables and Simple Data Types

Instructional Cues introduce it verbally and reinforce with tactile analogy if helpful. Provide a physical box (a small container) with a label stuck to it and a slip of paper inside representing the value. The box remains; the paper changes. The label stays the same; the contents change with reassignment.

Begin every data type lesson with `type()`. `print(type(42))` -> `int`. `print(type("hello"))` -> `str`. Make this a reflex the student uses whenever they're unsure what kind of data they have. For a student who cannot visually scan code for context clues, `type()` is often the only reliable way to confirm what a variable contains.

The string/number audio confusion: When a screen reader reads `"42"` and `42`, they sound identical. This is a critical accessibility issue. Address it directly: [9]

- *"Your screen reader will read these the same way. The only reliable way to tell them apart is `type()`. Use it. Constantly."*
- Demonstrate the consequence: `"42" + 1` raises `TypeError`. `42 + 1` gives `43`.

Multiple Explanations for Variable Naming Rules Provide all of these; let the student choose what sticks:

- *Rule-based:* Must start with letter or underscore. May contain letters, digits, underscores. May not be a Python keyword.
- *Analogy:* "Variable names are like legal names. They can't start with a number (no one's legal name is '2Fast'), can't contain hyphens (the name Mc-Donald would confuse paperwork), and can't be reserved words like 'True' or 'if' that already mean something official."
- *Mnemonic for invalid starts:* "Numbers first? Never. Hyphens? Horrible. Spaces? Surely not."

Thought Experiments

1. *"If variables are labeled boxes, what happens when you do `x = 5` and then `y = x` and then `x = 10`? What is `y` now? Does reassigning `x` affect `y`? Why or why not?"*

2. *"Python is 'dynamically typed,' meaning a variable can change its type at any time: `x = 5` then `x = 'hello'`. Many languages forbid this. What bugs could dynamic typing accidentally create? What flexibility does it offer?"*

3. *"The integer `42` and the string `'42'` look identical when spoken aloud. How many other pairs of values can you think of that might be ambiguous when a screen reader reads them? What coding habits help prevent confusion?"*

For congenitally blind students without strong spatial metaphors, consider an auditory alternative: a variable is like a voice message saved under a contact name. The name stays the same; the message (value) can be re-recorded (reassigned).

`type()` is more important for blind students than for sighted ones. A sighted programmer can infer type from context and syntax highlighting. A screen reader user hears "42" whether the value is an integer or a string. `type()` is the equalizer.

Accessibility Tip: After any `input()` call, immediately call `type()` on the result before doing anything with it. `"42"` and `42` are read identically by a screen reader but behave completely differently: `"42" + 1` raises `TypeError`, `42 + 1` gives `43`.

[9] This is the accessibility gotcha that surprises even experienced VI educators. Build in a `type()` check as a classroom ritual: after every `input()` call, immediately print `type()` of the result before using it.

4. *"Constants in programming represent values that never change — like* `SPEED_OF_LIGHT` *or* `MAX_USERS`*. Python has no true constants (you can always reassign them). Why do programmers use* `CAPS_SNAKE_CASE` *as a convention for constants even though Python can't enforce them?"*

Alternative Resources

- **CS50 Python (Harvard, edX):** David Malan's accessible video lectures. Closed-captions and transcripts available for all lessons. Covers variables and types in Week 0.
- **Python.org official tutorial:** Plain HTML, navigable by screen reader. Section 3 covers numbers, strings, and basic types.
- **"Python for Everybody" by Dr. Chuck Severance (free online):** Audio-described video lectures available. Chapter 2 covers variables and expressions.
- **Codecademy "Learn Python 3":** Web-based coding environment. Accessibility is partial — code editor is accessible but some UI elements are not. Worth testing with the student's specific setup.

---

7. Unit 2 — Numeric Computations and I/O

Lesson 2.1: The Software Development Process

Instructional Cues This is a conceptual lesson. Resist the urge to rush to code. Spend the full session on design and pseudocode.

Before writing pseudocode ask the student to state the problem in one sentence. Then ask "why?" five times to reach the root of what the program really needs to do. Then ask "why?" five times to reach the root of what the program really needs to do. This technique from Lean manufacturing translates well to programming.

The Five Whys method originated in Toyota's manufacturing quality system (kaizen). Its application to software design was popularized by the lean startup movement. The technique is equally effective for VI students because it produces verbal, non-visual problem specifications.

Example: "My program calculates a restaurant tip."

- Why? "So the user knows how much to pay."
- Why do they need to calculate it? "The tip amount isn't printed on the bill."
- Why? "The restaurant leaves the amount to the customer's discretion."
- So the real problem is: "Given a bill amount, calculate 15%, 18%, and 20% tip options, plus the total."

This more complete problem statement produces a better program design.

Pseudocode format for screen reader users: Plain `.txt` files work best for pseudocode. Avoid word processors or special formatting tools

that add accessibility overhead. The student should be able to read their pseudocode fluently while writing their Python.

Thought Experiments

1. *"In engineering, a bridge is designed many times before a single beam is placed. The cost of redesigning a blueprint is near zero; the cost of redesigning a half-built bridge is enormous. How does this ratio (cheap to redesign early, expensive to redesign later) apply to software?"*
2. *"The design phase involves thinking about edge cases — unusual inputs. For a program that calculates how many buses are needed to transport N students, with each bus holding 40 students, what edge cases exist? (Consider: What if N=0? What if N=40? What if N=41? What if N is negative?)"*
3. *"Some agile software development methodologies say 'working software over comprehensive documentation.' Does this conflict with the value of design? How do you reconcile the two?"*
4. *"Testing is described as part of the development process. But some teams test after development is complete. What is lost by testing at the end rather than continuously throughout? What is gained?"*

Lessons 2.2–2.5: Expressions, Input, Types, Math

Instructional Cues [10]

```
user_input = input("Type the number 5: ")
print(type(user_input))    # <class 'str'>
print(user_input + 1)      # TypeError
print(int(user_input) + 1) # 6
```

Run this sequence and let the student hear both the error and the fix. The `TypeError` message should be read aloud and analyzed before the fix is shown.

Order of operations — the hearing test: Read `10 + 5 * 2` aloud and ask for the answer before Python is consulted. Most students say 30; Python says 20. This is a memorable demonstration of why precise symbol reading matters. "Your ear parsed it as 'add 10 and 5, then multiply by 2.' Python parses it as '10 plus the result of 5 times 2.'"

`math` module exploration via REPL:

```
import math
dir(math)  # Lists all functions in the module
help(math.sqrt)  # Reads the docstring aloud
```

`dir()` and `help()` are screen-reader-friendly tools for exploring any module. Teach them as first-resort documentation tools.

[10] This is one of two facts about `input()` that every beginner gets wrong at least once. The other is that it pauses program execution until the user presses Enter. Both are invisible to a reader scanning the code without running it.

Thought Experiments

1. *"Type conversion functions like `int()` and `float()` can fail: `int('hello')` raises a `ValueError`. Who is responsible — the programmer who called `int()` on untrusted input, or the user who typed 'hello'? How should a well-designed program handle this?"*

2. *"`math.sqrt(25)` returns `5.0`, not `5`. Why does a function that logically produces a whole number return a float? What would break if it returned an integer instead?"*

3. *"Floating-point numbers in computers are approximations — `0.1 + 0.2` gives `0.30000000000000004`. Describe a real-world application where this imprecision could cause serious problems. What strategies exist to avoid it?"*

4. *"The `//` operator performs floor division: `7 // 2` = `3`. The `%` operator gives the remainder: `7 % 2` = `1`. Together they completely describe integer division. What real-world problem does each solve? Describe a program that would need both."*

Alternative Resources

- **"Automate the Boring Stuff with Python" Chapter 1–3 (free online):** Covers expressions, data types, and input with practical exercises. Plain HTML, accessible.
- **Khan Academy "Introduction to Programming":** Video series with transcripts. Good for visual-thinker students who benefit from hearing concepts explained differently.
- **SoloLearn Python course:** Mobile app with accessible exercises. Works well with iOS VoiceOver and Android TalkBack for students who prefer mobile.
- **"Python Crash Course" by Eric Matthes:** The primary textbook for this curriculum. Chapter 2 aligns directly with this unit.

---

8. Unit 3 — Control Flow, Decisions, and Loops

Lessons 3.1–3.2: Conditionals

Instructional Cues **Verbal simulation protocol:** Before running any conditional code, have the student perform the simulation verbally:

1. "What is the value of `score`?"
2. "Evaluate the condition: is `85 >= 90`? That's False."
3. "Because the condition is False, the if block is skipped."
4. "Evaluate the next condition: is `85 >= 80`? That's True."

5. "The elif block runs. We print 'Grade: B'."

This 5-step protocol should become automatic before the student runs any code. It builds the mental simulation ability that is the core of debugging.

Mental simulation—tracing code execution step by step without running it—is more important for blind students than for sighted ones. A sighted programmer can run code and scan output visually in seconds. A blind programmer benefits from knowing what *should* happen before navigating the terminal output to verify it.

The assignment-vs-comparison trap:

```
# This is a bug that is SILENT in some languages
if score = 90:   # SyntaxError in Python (good!)
    print("Perfect!")
```

Python raises a `SyntaxError` here, which is actually a safety feature. Other languages (C, JavaScript) allow this and it becomes a silent logic bug. Appreciate Python's strictness here. [11]

[11] JavaScript's version of this bug is `if (score = 90)` which silently assigns 90 to `score` and evaluates as truthy—no error, wrong behavior. Python's refusal to allow assignment inside a condition is a genuine accessibility feature: it converts a silent semantic error into a loud syntactic one.

Thought Experiments

1. *"If Python didn't have `elif`, only `if` and `else`, could you still write a grading program? Show how. What is the cost of doing it without `elif`?"*
2. *"The `else` clause at the end of an if-elif chain is a catch-all — it runs if nothing else matched. What is the programmer asserting when they write `else: print('Invalid input')`? What assumptions about the universe of possible inputs does this make? When might those assumptions be wrong?"*
3. *"Boolean logic (`and`, `or`, `not`) comes from mathematics (George Boole, 1847). Boolean logic also describes how circuits in a computer work. In what sense is every program you write built on top of millions of tiny `and`, `or`, and `not` operations happening in hardware?"*
4. *"Short-circuit evaluation: in `if A and B`, Python doesn't evaluate `B` if `A` is False (because the result is already False). In `if A or B`, Python doesn't evaluate `B` if `A` is True. When is this behavior a useful feature rather than just an optimization?"*
5. *"Consider a program that decides whether to grant access based on two conditions: correct password AND recognized device. Map this to Python `and` logic. Now consider: unlock if either condition passes (maybe for emergency access). Map this to `or`. What are the security implications of each design?"*

Alternative Resources

- **CS Unplugged "Binary Numbers" activity:** Physical, non-computer activity that illustrates Boolean thinking. Can be adapted for tactile use.
- **"Think Python" by Allen B. Downey (free online):** Chapter 5 covers conditionals with clear explanation and exercises. HTML version is screen-reader-friendly.

- **Parsons Problems (online):** Code ordering exercises — the student arranges given lines into correct order. Does not require typing code from scratch. Available at parsons.problemsolving.org (accessibility varies; test first).

Lessons 3.3–3.4: Lists and For Loops

Instructional Cues **Mental models for lists — choose one per student:**

- *The train:* Each car has a number starting at 0. Car 0 is at the front. You can add cars to the end, remove from anywhere, and swap contents.
- *The playlist:* Songs in order, starting at position 0. You can shuffle, sort, add, or remove tracks.
- *The physical stack of cards:* A sequence of items that can be addressed by position.

The for loop as a bookmark reader: *"Imagine Python has a finger that moves along a list. It starts at position 0. It reads what's there, runs the loop body, moves the finger to position 1, reads, runs the body, and continues until the finger falls off the end."*

Tracing loops audibly — required exercise: For the first five loops students write, require them to verbally trace every iteration before running the code:

Verbal tracing is not just a learning exercise—it is a professional debugging technique. Experienced blind developers often trace loops aloud or on paper before running code because navigating loop output line-by-line with a screen reader is time-consuming. Building this habit early pays dividends.

- "Iteration 1: `fruit` gets 'apple'. We print 'I like apple.' "
- "Iteration 2: `fruit` gets 'banana'. We print 'I like banana.' "
- etc.

Non-Obvious Accessibility Strategies

- **Printing list contents accessibly:** A bare `print(my_list)` produces `['apple', 'banana', 'cherry']` with brackets and quotes. This is cluttered when read aloud. Teach the clean print pattern early:

```
for i, item in enumerate(my_list):
    print(f"  {i}: {item}")
```

  This gives numbered output, one item per line, that is easy to navigate and understand when heard. [12]

- **`len()` as a safety check before index access:** Teach students to always know the length before indexing into a list. This prevents `IndexError`. Make `len(my_list)` the first thing printed when debugging a list problem.

[12] Number-then-item output (`1: apple, 2: banana`) is the terminal equivalent of a screen reader's list navigation mode. The number gives position context that raw list notation `['apple', 'banana']` does not provide when spoken sequentially.

- **Index arithmetic verbalized:** When accessing `my_list[-1]`, say "the last element, which is at index `len(my_list) minus 1`, which Python shortens to `-1`." Connecting the shorthand to the underlying meaning prevents confusion.

Thought Experiments

1. *"Zero-based indexing means the first element is at position 0. This feels unnatural but is standard across almost all programming languages. Why? (Hint: think about memory addresses — the address of the first element, and the address of any element as an offset from there.)"*

2. *"A list allows duplicate values. A set (which you'll learn later) does not. When would duplicates in a list be meaningful data? When would they indicate an error in your program?"*

3. *"A `for` loop iterates over every element in a list. Is there any computation a `for` loop can do that a `while` loop cannot? Is there any computation a `while` loop can do that a `for` loop cannot?"*

4. *"What happens when you iterate over an empty list: `for item in []:`? Does Python crash, print an error, or do nothing? Why is 'do nothing' the correct behavior rather than an error?"*

Lesson 3.5: String Operations

Instructional Cues **Immutability demonstration:**

```
text = "hello"
text.upper()         # Creates a new string --- doesn't modify text
print(text)          # Still "hello"
text = text.upper() # Now reassigned
print(text)          # "HELLO"
```

This demonstration catches a persistent beginner misunderstanding. Ask the student to predict the output of `print(text)` after `text.upper()` before running it.

Method chaining verbalized: `"  hello world  ".strip().title()` — read this as: "Start with the string. Strip whitespace from both ends. Then convert the result to title case."

Thought Experiments

1. *"Strings are immutable — they cannot be changed, only replaced. Lists are mutable — they can be changed in-place. What architectural difference between strings and lists makes one mutable and the other not? (Think about how they're stored in memory.)"*

2. *"`.split()` splits a string into a list of words. `.join()` combines a list of strings into one string. They are inverses. What is the inverse of `.replace()`? Does it have a direct inverse? If not, how would you reverse a replacement?"*

3. *"String formatting with f-strings: `f'Value is {x:.2f}'` formats `x` as a float with 2 decimal places. Why does a programming language need a mini-language for formatting numbers? What would programs look like if we had to format every number manually with division and modulo?"*

Alternative Resources

- **Python String Methods documentation** (docs.python.org/3/library/stdtypes.html#string-methods): Plain HTML table of all string methods. Screen-reader navigable. Essential reference.
- **"Python for Data Analysis" by Wes McKinney:** Chapter on string manipulation provides real-world examples beyond what this curriculum covers. For motivated students.

---

9. Unit 4 — Data Structures: Lists and Dictionaries (See all lessons above for list material; this section adds dictionary-specific guidance.)

Dictionary Lessons

Instructional Cues **Multiple mental models for dictionaries:**

- *Real dictionary:* You look up a word (key) to find its definition (value). Words are unique; definitions can be anything.
- *Contact list:* Each name (key) maps to a phone number (value). No two contacts have the same name in a well-maintained list.
- *JSON configuration file:* Settings names map to their values. This foreshadows Unit 8.

The `.get()` method vs. direct access:

```
# This crashes on missing key
print(my_dict["nonexistent_key"])   # KeyError

# This returns a default --- never crashes
print(my_dict.get("nonexistent_key", "Not found"))  # "Not found"
```

For a screen reader user who may not immediately catch a crash in terminal output, teaching `.get()` as the default access pattern reduces debugging overhead.

A `KeyError` crash produces output that looks similar to normal output

Accessibility Tip: Use `dict.get(key, default)` instead of `dict[key]` whenever a key might not exist. A `KeyError` crash is easy to miss when navigating terminal output with a screen reader, since the error appears in the same buffer as normal output. `.get()` prevents the crash entirely.

A `KeyError` crash produces output that looks similar to normal output when navigating with a screen reader—it appears in the same terminal buffer. Using `.get()` with a default eliminates an entire category of crash that might otherwise go unnoticed mid-session

Thought Experiments

1. *"Dictionaries use keys to retrieve values in O(1) time — essentially instant regardless of dictionary size. Lists use indices, also O(1). But searching a list for a specific value (not by index) takes O(n) time — proportional to list length. When does this difference matter in practice?"*

2. *"Your dictionary cannot have duplicate keys — adding a second value for an existing key replaces the first. What real-world data problems does this property solve? What problems does it create?"*

3. *"JSON (JavaScript Object Notation) is essentially Python dictionaries written in a slightly different format. What does it mean that a data format invented for JavaScript has become the universal language of data exchange? What does this tell you about the value of common standards?"*

4. *"A 'nested dictionary' contains other dictionaries as values. Model a university course catalog as a nested dictionary. What are the outer keys? What are the inner keys? What would* `catalog['CS']['101']['instructor']` *represent?"*

---

10. Unit 5 — User Input and While Loops

Thought Experiments

1. *"A while loop with a flag variable and a while loop with* `break` *can often accomplish the same thing. Compare these two equivalent programs and explain when you would prefer each:*

```
# Version A: flag
active = True
while active:
    cmd = input("> ")
    if cmd == "quit": active = False
    else: print("You said:", cmd)

# Version B: break
while True:
    cmd = input("> ")
    if cmd == "quit": break
    print("You said:", cmd)
```

   Which is clearer? Which is easier to extend if you add more exit conditions?"

2. *"An infinite loop that waits for user input is the core pattern of every interactive program — email clients, video games, operating systems. Describe the 'event loop' of your screen reader in terms of while loops and if statements.while loops and if statements. What is it waiting for? What does it do when it gets an event?"*

This thought experiment has real depth for VI students: they *use* an event loop every day. A screen reader's main loop listens for keyboard events, focus changes, text insertions, and spoken output requests. Understanding that their screen reader is "just" a while loop with a lot of if statements demystifies the tool and connects programming concepts to lived experience.

3. *"The `continue` statement skips the rest of the current iteration. An alternative is to wrap the loop body in an `if` statement. When is `continue` cleaner? When does the `if` approach produce clearer code?"*

---

11. Unit 6 — Functions

Thought Experiments (in addition to existing guide)

1. *"A function is described as a 'black box' — you give it inputs and get outputs without knowing the internal details.the internal details. What are the benefits of this abstraction for the programmer who writes the function? For the programmer who uses it? What are the limits of this metaphor?"*

The black-box metaphor is particularly resonant for blind students: they already interact with most technology as a black box, trusting that inputs produce the correct outputs without being able to inspect the internals. Programming inverts this: the student is now the author of the black box, responsible for what comes out.

2. *"Default parameter values: `def greet(name='World')`. This allows the function to be called with or without an argument. How does this enable backward compatibility — adding a new feature without breaking existing code that calls the function?"*
3. *"Functions should 'do one thing.' What does 'one thing' mean? Is `calculate_and_print_tax()` one thing or two? If you separate it into `calculate_tax()` and `print_tax()`, what becomes possible that wasn't before?"*
4. _"Recursive functions call themselves. `factorial(n) = n _ factorial(n-1)`. What is the base case and why is it required? What goes wrong if a recursive function has no base case?"*
5. _"A function is a way to name a piece of code. Why does naming matter in programming? What is the cognitive difference between `x = a _ b + c - d`and`x = calculate_net_profit(revenue, cost, overhead, taxes)`?"*

Alternative Resources

- **"Structure and Interpretation of Computer Programs" (SICP) — MIT OpenCourseWare:** Conceptually deep treatment of functions as first-class objects. Lectures have transcripts. For advanced students.
- **Exercism.io Python Track:** Mentor-guided exercises with community feedback. Web platform with reasonable accessibility. Students can complete exercises at their own pace.
- **Python Tutor (pythontutor.com):** Step-by-step execution with variable state display. The text-based "state panel" below the code is screen-reader accessible and shows variable values at each step.

Extended Instructional Cues **Staged introduction to OOP — do not rush to syntax:**

Stage 1 (no code): Discuss the concept. What is an object? What does "bundling data and behavior" mean? Use tangible examples the student knows.

Stage 2 (pseudocode): Write a class in plain English before Python syntax:

```
Class: BankAccount
  Has: owner_name, balance, account_number
  Can do: deposit(amount), withdraw(amount), get_balance()
```

Stage 3 (Python skeleton): Write the class structure with empty methods and docstrings.

Stage 4 (full implementation): Fill in method bodies.

This staging prevents syntax confusion from obscuring the conceptual learning.

`__str__` is mandatory for accessibility: Every class a student creates must define `__str__`. define `__str__`. Without it, `print(my_object)` produces `<__main__.BankAccount object at 0x7f...>` — meaningless when heard. The requirement to define `__str__` also reinforces thinking about what the object's most important information is.

```
def __str__(self):
    return f"BankAccount(owner={self.owner_name}, balance=${self.balance:.2f})"
```

The four-stage OOP introduction (verbal, pseudocode, skeleton, implementation) maps directly to the Concrete-Representational-Abstract (CRA) instructional framework used in special education. VI students benefit from spending longer at the verbal and pseudocode stages than sighted students, who can re-read syntax examples visually.

Make `__str__` a graded requirement on every assignment involving classes. Frame it as an accessibility contract: the programmer is responsible for ensuring their objects produce meaningful audio output. This connects programming practice to the student's lived experience of inaccessible software.

Thought Experiments

1. *"Encapsulation means hiding internal details. A car's engine is encapsulated under the hood — you don't need to know how fuel injection works to drive. What are the analogous 'hidden internals' in a BankAccount class? What should be private? What should be public? Why?"*

2. *"Inheritance creates 'is-a' relationships. Composition creates 'has-a' relationships. A `SavingsAccount` is-a `BankAccount`. A `Bank` has-a list of `BankAccount` objects. Design a simple banking system using both relationships. What would go wrong if you tried to use only inheritance?"*

3. *"Method overriding allows a subclass to replace a parent's method. When is this the right design? When should you call `super().method()` inside the override vs. replace the method entirely?"*

4. *"Python allows multiple inheritance (inheriting from two parent classes). Most languages do not permit this. What problems does multiple inheritance create? What problems does it solve? Would you use it?"*

5. *"Object-oriented programming was supposed to make code more reusable. Has it delivered on this promise? What kinds of code reuse are well-served by OOP? What kinds aren't?"*

Alternative Resources

- **"Object-Oriented Programming in Python" by Michael Goldwasser and David Letscher:** Textbook specifically pairing Python and OOP concepts. More formal treatment for students who want depth.
- **"Fluent Python" by Luciano Ramalho:** Advanced but deeply accessible to screen readers. Covers Python's data model including `__str__`, `__repr__`, and special methods. For motivated students.
- **Real Python "Object-Oriented Programming in Python" tutorial (realpython.com/python3-object-oriented-programming/):** Long but thorough. Plain HTML.

---

13. Unit 8 — Files and Exceptions

Extended Instructional Cues **Files as persistence — make it tangible:** Run a program that stores a number in a variable, quits, and starts again — the variable is gone. Then run the same program storing the number in a file — it persists. Ask: "What changed? What is different about a file vs. a variable?" Let the student reason through it before explaining.

The `with` statement — demonstrate the alternative:

```
# Without 'with' --- dangerous
f = open("data.txt", "r")
contents = f.read()
# If an error occurs here, f.close() is never called
f.close()

# With 'with' --- safe
with open("data.txt", "r") as f:
    contents = f.read()
# f is closed automatically even if an error occurs
```

The `with` statement is a context manager.

Accessibility Tip: Always use `with open(filename) as f:` rather than `f = open(...)` / `f.close()`. The `with` block guarantees the file is closed even if the program crashes mid-read. Unclosed files can cause silent data loss that produces no error message.

The `with` statement is a context manager. Python's context manager protocol (`__enter__` and `__exit__`) guarantees cleanup even when exceptions are raised. For files, databases, and network connections, unclosed resources cause silent data loss or corruption—particularly hard to debug with a screen reader.

Exception types — teach by demonstration, not memorization: Create a "crash collection" — a series of small programs that each produce a specific exception. Let the student run each one, read the error, and discuss why it occurred before seeing the solution. before seeing the solution. [13] The goal is pattern recognition, not memorization.

[13] A crash collection serves double duty: it teaches exception recognition *and* builds screen reader navigation fluency. Each crash is an opportunity to practice the bottom-up error reading protocol: terminal, bottom line first, error type, line number, Ctrl+G to navigate.

Thought Experiments

1. *"Files are persistent storage — they survive when your program exits. Variables are temporary — they're gone when the program ends. What deter-*

mines which data should be stored in files vs. kept only in memory? What are the tradeoffs?"

2. *"The `with` statement uses Python's 'context manager' protocol. This protocol says: 'when this block starts, do X; when it ends (even if it ends with an error), do Y.' What other situations in programming require guaranteed cleanup? What happens in real systems when cleanup is skipped?"*

3. *"Exceptions propagate up the call stack — if function A calls B which calls C, and C raises an exception, it moves up to B, then A, until something catches it. If nothing catches it, the program crashes. Design the exception handling strategy for a program that reads a config file, parses it, and then connects to a database. Where should exceptions be caught?"*

4. *"JSON stores data as text. Binary formats (like `pickle` in Python) store data in a compact binary form. JSON is human-readable; binary is not. JSON is language-independent; pickle is Python-only. For each of these use cases, which format would you choose: (a) a config file the user might edit by hand, (b) a large dataset processed only by your own Python code, (c) data shared between a Python backend and a JavaScript frontend?"*

---

14. Unit 9 — Testing and Data Analysis

Extended Instructional Cues **Tests as documentation — make this concrete:** Show two identical functions — one with tests that describe the expected behavior, one without. one without. Ask: "Which function is easier to understand without reading the implementation?" The tests are a specification in executable form.

introduce TDD: write the test before the implementation. [14] This inverts the usual workflow:

1. Write a test that fails (because the function doesn't exist yet).
2. Write the minimum implementation to make the test pass.
3. Refactor.

This discipline is genuinely useful for beginners because it forces them to think about what a function should do before worrying about how.

For VI students, tests-as-documentation has a concrete accessibility payoff: a well-written test suite is a specification written in plain text that can be read aloud. When a student is handed unfamiliar code, running the tests and reading their names (`test_add_negative_numbers`, `test_empty_list`) is a faster way to understand the code's intent than reading the implementation.

[14] TDD has an unexpected accessibility benefit: it forces the programmer to specify behavior in text before implementing it, creating an audio-friendly specification document. The test file itself serves as a checklist of requirements.

Thought Experiments

1. *"Unit testing tests one function in isolation. Integration testing tests multiple components working together. System testing tests the entire program. Why have three levels rather than just one? What does each level catch that the others might miss?"*

2. *"A test can only prove that a program behaves correctly for the inputs tested. It cannot prove correctness for all possible inputs. What does this mean for the value of testing? How do you decide which inputs to test?"*

3. *"Random numbers make testing difficult — if the output changes every run, how do you write an assertion? Describe a strategy for testing a function that uses* `random.randint()`*. (Hint: think about seeding.)"*

4. *"CSV data from the real world is often 'dirty' — missing values, extra commas, inconsistent formatting, encoding issues. What does a robust CSV parser need to handle? What assumptions would a naive parser make that would cause it to fail on real-world data?"*

---

15. Unit 10 — Project-Based Learning

Extended Instructional Cues **Design document requirements:** Require a written design document before any code is written. For a screen reader user, a plain text file works best. The document should include:

Plain `.txt` design documents are more accessible than Word, Google Docs, or Markdown-rendered HTML. The student can keep the design doc open in a second terminal pane and navigate between it and their code file with `Ctrl+`` and `Ctrl+Shift+``.

```
Project Name: [title]
One-sentence purpose: [what problem does this solve?]
Target user: [who will use this?]

Features (ranked by priority):
1. [most important feature]
2. ...

Data I will store: [what information needs to persist?]
Data structures: [which Python structures will hold this?]

Functions I will write:
- [function name](inputs) -> output: [brief description]
- ...

Main loop pseudocode:
[plain English description of program flow]

Edge cases I have considered:
- [what happens if the user enters nothing?]
- [what happens if the file doesn't exist?]
- ...
```

Iterative development enforcement: Require demonstrating a working version at the end of every session, no matter how minimal. "Work-

ing" means runnable and producing correct output for at least one input. A student who cannot demonstrate working code at session end needs to reduce scope, not work harder overnight. [15]

[15] The "working at every session" requirement has an accessibility motivation: a non-working program is invisible to a screen reader. A student who has written fifty lines but none of them run has nothing to navigate, nothing to test, and no audio feedback. Iterative development—even one working function per session—provides continuous audio checkpoints.

Thought Experiments

1. *"A program designed for sighted users might use color to indicate status (red for error, green for success), icons to represent actions, and layout to group related information. How would you replicate each of these design patterns in a screen-reader-accessible terminal program?"*
2. *"Your contact manager has a search function. How many different search behaviors could be useful? Search by exact name match, partial match, phone prefix, email domain, notes keywords? What should the program do when there are 0 results? 1 result? Multiple results?"*
3. *"Error messages should be actionable — they should tell the user not just what went wrong, but what to do next. Rewrite these unhelpful messages to be actionable: 'Error: invalid input', 'File not found', 'Operation failed.'"*

---

16. Unit 11 — Capstone Project

Assessment Guidance

Process Checkpoints

Checkpoint	Timing	What to Evaluate
Written design document	Before any code	Is the scope realistic? Are features prioritized? Are data structures appropriate?
Data model working	End of week 1	Can items be created, stored in memory, and printed?
Core CRUD functional	End of week 2	Add, view, edit, delete all work?
Persistence working	End of week 3	Data survives program restart?
Error handling complete	End of week 4	Program handles all invalid inputs without crashing?
Is all output meaningful when read aloud?		
Final demo	End	Full demonstration with screen reader active

The accessibility review checkpoint is not optional or cosmetic. A program whose output is inaccessible when read aloud has failed its primary user requirement, regardless of whether it compiles and runs correctly.

The Screen Reader Demo Rubric Evaluate each:

Criterion	Excellent	Satisfactory	Needs Work
Menu navigation	Student navigates menu by listening; confirms choice before acting	Student navigates but occasionally needs visual cues	Student needs visual confirmation of menu state
Input prompts	All prompts clearly describe expected format and type	Most prompts are descriptive	Some prompts are vague or missing
Confirmation messages	Every action produces a clear spoken confirmation	Most actions confirmed	Some actions succeed silently
Error messages	All error messages are specific and actionable	Most errors give useful messages	Some errors produce generic or missing messages
Data persistence	Student demonstrates save and load by quitting and restarting	Save/load works but student doesn't demonstrate live	Save or load not fully functional
Code explanation	Student can explain any line of their code when asked	Student can explain most of their code	Student cannot explain sections of their code

Requiring the screen reader to be *active* during the final demo—not just available—ensures the student has tested their own program under real conditions. Many accessibility issues only surface when the screen reader is actually speaking the output.

---

17. Assessment Strategies

Assessment for VI students requires rethinking the standard model. Timed written tests disadvantage students who navigate code via keyboard. Oral defenses, portfolio reviews, and screen-reader-active demonstrations are more equitable and reveal deeper understanding.

Accessible Assessment Design Traditional programming assessments (written tests, timed coding) create unnecessary barriers for blind students. Use alternatives that assess the same competencies:

Think-aloud problems: Give the student a short piece of code and ask them to read it aloud and explain what it does, step by step. This assesses reading comprehension and debugging ability without requiring written answers.

Debugging exercises: Provide a working program and a broken version. Ask the student to identify the error using only their screen reader and debugging tools. This directly tests real-world skill.

Feature extension: Give a working program and ask the student to add a specific feature. This is a more realistic assessment than writing from scratch and allows scaffolding.

Verbal design review: Present a programming problem and ask the student to describe their approach — data structures, functions, algorithm — without writing any code. Assess reasoning quality.

Portfolio review: Maintain a collection of all programs written during the course. Assess growth, consistency, commenting quality, and the student's ability to explain their own work.

Grading Considerations

- Do not penalize for missing visual formatting (e.g., a missing newline that only matters visually).
- Do reward for accessibility features the student adds above requirements: clear prompts, confirmation messages, help text.
- Recognize that a blind student may spend significantly more time debugging than a sighted peer on identical problems. Factor this into time expectations.
- Consider allowing oral explanations as alternatives to written code comments for students who find comment-writing slow.

---

18. Supporting Diverse Needs Within the Visually Impaired Population

For Students with Low Vision

- Use high-contrast VS Code themes: Dark High Contrast, Light High Contrast (available under Color Themes in settings).
- Increase editor font size: `Ctrl+Shift+P` -> "Preferences: Open Settings" -> `editor.fontSize`.
- Enable zoom magnification: `Ctrl++` to zoom the VS Code window.
- Test whether syntax highlighting is visible with their specific visual condition. Some students benefit from; others are confused by color information they cannot fully perceive.

For Students Who Are Deaf-Blind

- All communication with the teacher must be through the braille display or tactile signing — no spoken instruction.
- Screen reader audio verbosity settings must be managed by the student (since they cannot hear audio feedback during class).
- Consider pairing with a trained interpreter who knows both tactile communication and computing concepts.
- Review the Perkins School for the Blind's resources on deaf-blind education technology.

For Students with Physical Disabilities in Addition to Visual Impairment

- Switch access: VS Code is fully keyboard-navigable and can be operated with adapted keyboards, switch interfaces, or eye-gaze systems paired with virtual keyboards.

- Voice control: Dragon NaturallySpeaking and macOS Voice Control can both operate VS Code. Voice coding requires additional voice commands for symbols.
- Sip-and-puff devices: Can control keyboard input. Students using these will have different typing rates — extend all time-based tasks.

For Students with Cognitive Differences in Addition to Visual Impairment

- Reduce the amount of new vocabulary per session.
- Use consistent metaphors across the entire course — do not introduce a new metaphor for the same concept.
- Provide written summaries of every lesson that the student can review with their screen reader later.
- Break complex concepts into smaller steps; complete each step before introducing the next.
- Build in regular review sessions rather than only forward-moving instruction.

16

[16] AI code assistants (GitHub Copilot, Claude, ChatGPT) can generate code that *appears* correct but contains subtle logic errors or accessibility oversights. For VI students learning to code, the risk of uncritically accepting AI output is high. Teach a "trust but verify" workflow: read every line the AI generates before running it.

Framing AI for the Classroom Establish a clear classroom policy before students begin using AI tools:

Permitted: Using AI to explain error messages, generate a first draft for the student to critique and improve, explain a concept a different way, suggest test cases to consider.

Not permitted: Submitting AI-generated code as completed work without reading and understanding every line, using AI to complete assessments that are meant to evaluate individual skill.

Required when AI is used: The student must be able to explain any AI-generated code they submit as if they wrote it themselves. Inability to explain code is evidence it was not understood.

AI Tools and Blind Students — Specific Considerations **Potential accessibility benefit:** AI tools like GitHub Copilot are text-based — suggestions appear as text that a screen reader can read. Copilot's Accessible View integration in VS Code is improving.

Potential accessibility risk: AI may generate code that is visually clear but auditorily confusing — complex nested structures, long lines, or dense comprehensions that are harder to navigate with a screen reader than to read with eyes.

Teaching point: Ask students to evaluate AI-generated code for screen-reader accessibility:

- Are variable names descriptive?

- Are long lines split into readable pieces?
- Are print statements and prompts clear and complete?
- Does the code have docstrings?

If not, the AI-generated code needs to be modified before use — regardless of whether it runs correctly.

Thought Experiments on AI

1. *"An AI writes code that solves your problem but you don't understand it. You submit it. It passes tests. Have you learned anything? What is the difference between 'having a solution' and 'knowing how to solve problems'?"*
2. *"AI code generators are trained on billions of lines of code written mostly by sighted programmers. What biases might this introduce in the code they generate? Think specifically about: variable names, output design, prompts, error messages, documentation."*
3. *"If AI can write most code, which skills become more valuable for future programmers? Consider: understanding requirements, debugging, testing, communicating about code, evaluating solutions, and accessibility design."*

---

20. Alternative and Supplementary Resources

Textbooks and Reading

Resource	Level	Format	Accessibility
Python Crash Course, 3rd Ed. (Matthes)	Beginner	Print/ebook	Good; primary textbook for this curriculum
Python Programming: An Introduction to CS (Zelle)	Beginner	Print/ebook	Good; companion textbook
Automate the Boring Stuff with Python (Sweigart)	Beginner–Intermediate	Free HTML / print	Excellent; HTML version fully accessible
Think Python (Downey)	Beginner	Free HTML / print	Excellent HTML version
Learn Python 3 the Hard Way (Shaw)	Beginner	Print/ebook	Plain text exercises, highly accessible
Python for Everybody (Severance)	Beginner	Free HTML + video	Video lectures have transcripts; good accessibility
Fluent Python (Ramalho)	Advanced	Print/ebook	Dense but thorough; for motivated advanced students
Python Docs (python.org/3)	Reference	Free HTML	Excellent screen-reader accessibility; definitive

Online Courses and Platforms

Platform	Course	Screen Reader Compatibility	Notes
edX / CS50P (Harvard)	Introduction to Programming with Python	High — video transcripts, accessible exercises	Free audit. Very high quality.
Coursera	Python for Everybody (Michigan)	Medium — some interactive elements inaccessible	Free audit available
Codecademy	Learn Python 3	Partial — editor accessible, some UI not	Requires testing with specific screen reader
SoloLearn	Python Core	High — mobile app works well with VoiceOver/TalkBack	Good for mobile-first learners
freeCodeCamp Excellent for structured practice after basics	Python curriculum	Medium — varies by module	Free; large community
CodingBat	Python problems	High — plain HTML, browser-based	Good for logic practice; no complex UI
replit.com	Any Python project	Medium — test with student's setup	No-install option; accessibility variable

Video Resources

Resource	Description	Accessibility
CS50P (Harvard)	Full course lectures, free on YouTube and edX	Closed captions, transcripts; very high production quality
Corey Schafer Python tutorials (YouTube)	In-depth concept tutorials	Closed captions available
Tech With Tim Python (YouTube)	Project-based tutorials	Closed captions available
Sentdex Python tutorials (YouTube)	Large series from beginner to advanced	Captions available
Python.org Getting Started	Short official tutorials	Plain text, no video

Tools Specifically for Accessibility in Programming Education

Tool	Purpose	Notes
Python Tutor (pythontutor.com)	Step-by-step code execution with state display	Text-based state panel is screen-reader accessible
Mu Editor	Beginner-focused Python IDE	Simpler than VS Code; reasonable screen reader support
Thonny	Beginner IDE with step debugger	Variable viewer is navigable via keyboard
NVDA + Remote	Remote screen sharing for teacher support	Student's screen reader on their machine; teacher can see output
GitHub Codespaces	Cloud VS Code environment	Same accessibility as local VS Code; useful when local setup fails
Braille Institute Resources	General adaptive technology guidance	brailleinstitute.org
APH (American Printing House)	Educational materials for blind students	aph.org

Communities and Professional Organizations

Organization	Relevance
National Federation of the Blind (NFB) — Science & Engineering Division	Blind scientists and engineers; mentorship opportunities
American Council of the Blind (ACB) — Technology Interest Group	Accessible technology advocacy and resources
CSUN Assistive Technology Conference	Annual conference; many papers on computing education for blind students
SIGACCESS (ACM SIG on Computers and Accessibility)	Academic research on accessible computing
Ability Central	Technology resources for people with disabilities
Hadley Institute for the Blind	Professional development for educators of blind students
Perkins School for the Blind	Resources including STEM education for blind students
Carroll Center for the Blind	Technology training for blind adults; may have curriculum resources
Stack Overflow Blind/VI Community	Informal community; blind developers helping each other

---

21. Quick Reference Sheets

Error Message Reading Protocol When an error occurs, guide the student through this sequence:

```
1. Navigate to terminal output (Ctrl+` or click panel)
2. Read from the BOTTOM of the error output upward
3. Note the ERROR TYPE (last line): SyntaxError, NameError, TypeError, etc.
4. Note the ERROR MESSAGE: specific description of what went wrong
5. Note the LINE NUMBER in the traceback
6. Jump to that line with Ctrl+G -> [line number] -> Enter
7. Read the line and the lines immediately before and after it
8. Categorize the error (see table below)
9. Fix and re-run
```

Exercism is particularly well-suited for blind students: all exercises are solved locally in VS Code with a CLI tool, test-driven, and submitted via terminal. No web-based code editor required.

The bottom-up reading rule is the single most important

Accessibility Tip: When an error occurs, navigate to the *bottom* of the terminal output first—the last line of a Python traceback contains the most specific error message. Use `Ctrl+End` to jump to the end of the terminal buffer, then read upward to the line number.

Error Type Quick Reference

Error Type	Common Cause	First Thing to Check
`SyntaxError`	Missing `:`, `)`, `"`, `]`, or `=` instead of `==`	The exact column number in the error message
`IndentationError`	Inconsistent spaces/tabs	Line indentation; use `cat -A` or `show whitespace` in VS Code
`NameError`	Variable not defined, or misspelled	Check spelling; check if variable was defined before use
`TypeError`	Wrong data type	Use `type()` on each variable involved in the operation
`ValueError`	Right type, wrong value	Check what you're converting: `int('hello')` fails; `int('5')` works
`IndexError`	List index out of range	Print `len(list)` and check what index you're trying to use
`KeyError`	Dictionary key doesn't exist	Use `.get()` or check with `if key in dict` first
`FileNotFoundError`	File doesn't exist at that path	Print `os.getcwd()` and verify the file path
`AttributeError`	Called a method that doesn't exist	Check spelling and whether the object is the type you think it is
`ZeroDivisionError`	Dividing by zero	Add a check before the division: `if denominator != 0`
`RecursionError`	Infinite recursion	Verify base case exists and is reachable

Accessible Output Patterns

```python
print("Items in collection:")
for i, item in enumerate(my_list, 1):
    print(f"  {i}. {item}")
print(f"Total: {len(my_list)} items")

# Instead of: print(my_dict)
# Use:
print("Current settings:")
```

The bottom-up reading rule is the single most important error-debugging habit for screen reader users. Python tracebacks can be many lines long; the first line heard is always the least specific. Jump to the bottom first.

Accessible output rule: Never print a raw list

Accessibility Tip: When printing any collection, use `enumerate()` to number each item and print one per line. Numbered, labelled output is far easier to navigate with a screen reader than bracket-notation list output. Make this a habit from the first list lesson.

```
    for key, value in my_dict.items():
        print(f"  {key}: {value}")

    # Menu display:
    print("\n--- Main Menu ---")
    print("1. Add item")
    print("2. View all items")
    print("3. Search")
    print("4. Delete item")
    print("5. Save and quit")
    print("-----------------")
    choice = input("Enter option number (1-5): ")

    # Confirmation messages:
    print(f"Contact '{name}' added successfully.")
    print(f"Found {len(results)} matching records.")
    print("Data saved to contacts.json.")
    print("No matching records found.")
    print("Error: Name field cannot be empty. Please try again.")
```

VS Code Keyboard Shortcut Reference Card

```
NAVIGATION
  Ctrl+G           Go to specific line number
  Ctrl+Home        Jump to start of file
  Ctrl+End         Jump to end of file
  Ctrl+Shift+O     Go to function/class (symbol picker)
  F12              Go to Definition
  Alt+F12          Peek Definition
  Shift+F12        Find all References
  Ctrl+P           Quick open file by name

ERRORS AND DIAGNOSTICS
  F8               Next error/warning
  Shift+F8         Previous error/warning
  Ctrl+Shift+M     Open Problems panel
  Shift+Alt+F2     Accessible View (for hover/tooltip content)

EDITING
  Shift+Alt+F      Format document (auto-indent)
  Ctrl+/           Toggle comment on selected lines
  Tab              Indent selected lines
  Shift+Tab        Outdent selected lines
  Ctrl+D           Select next occurrence of word
  Ctrl+H           Find and replace

TERMINAL
  Ctrl+`           Toggle integrated terminal
```

```
  Ctrl+Shift+`    New terminal
  Up/Down arrows  Navigate command history
  Tab             Auto-complete filenames
  Ctrl+C          Stop running program

SCREEN READER
  Ctrl+Shift+P    Command Palette
  Ctrl+Shift+P -> "Toggle Screen Reader Optimized"
  Shift+Alt+F1    Accessibility help
```

Unit 0: Setting Up Your Accessible Python Development Environment

Overview A detailed environment setup is not merely a technical prerequisite; it empowers students and reduces dependency on external assistance. For visually impaired students, relying on external help for every setup or environment issue can be disempowering and slow down the learning process. This unit provides extremely detailed, screen-reader-friendly instructions for environment setup.

Target Audience: Students with no prior programming experience, relying on screen readers for all computer interactions.

Lesson 0.1: Installing Python and Visual Studio Code

Reading References

- Matthes: Chapter 1, pages 3–14
- Matthes: Appendix A, pages 463–468

Installing Python 3 Python is not always installed on a computer by default. Download the latest version from `https://www.python.org/downloads/`.

Windows

1. Locate the downloaded `.exe` file (usually in your Downloads folder). Activate it.
2. **CRUCIAL STEP:** When the installer window appears, check the **"Add Python to PATH"** checkbox. Use Tab to navigate to this checkbox and press Spacebar to check it.
3. Navigate to "Install Now" and activate it.
4. Follow any further prompts. Activate "Close" when finished.

macOS

1. Locate the downloaded `.pkg` file. Activate it.
2. Activate "Continue" through the steps.
3. Agree to the license terms. Choose the installation location (default is fine).
4. Enter your administrator password when prompted.
5. Activate "Install" and then "Close" when finished.

Linux Python 3 is often pre-installed. Verify in a terminal:

```
python3 --version
```

If not installed (Debian/Ubuntu):

```
sudo apt update
sudo apt install python3
sudo apt install python3-pip
```

Verifying Python Installation Open a terminal and type:

```
python3 --version
```

You should hear the version number (e.g., "Python 3.10.12"). Also check pip:

```
pip3 --version
```

Installing Visual Studio Code

1. Navigate to https://code.visualstudio.com/download.
2. Download the installer for your OS.

Windows

- **CRUCIAL STEP:** On the "Select Additional Tasks" screen, check **"Add to PATH"**.
- Activate "Next" and then "Install".

macOS

- Drag the "Visual Studio Code" application to your "Applications" folder.

Linux (Debian/Ubuntu)

```
cd Downloads
sudo dpkg -i code_*.deb
```

Verify VS Code

```
code --version
```

Skills Checklist

☐ I can successfully download the Python 3 installer for my operating system.
☐ I can run the Python installer and correctly select the "Add Python to PATH" option.
☐ I can successfully download the VS Code installer for my operating system.

- ☐ I can run the VS Code installer and correctly select the "Add to PATH" option.
- ☐ I can open my computer's system terminal.
- ☐ I can verify Python's installation by running `python3 --version` in the terminal.
- ☐ I can verify VS Code's installation by running `code --version` in the terminal.

Lesson 0.2: Configuring Your Screen Reader for VS Code

Reading References

- Matthes: Appendix B, pages 469–476
- Matthes: Appendix C, pages 477–482

General Screen Reader Tips for VS Code **1. Use Screen Reader Mode**

- Press `Ctrl + Shift + P` to open the Command Palette.
- Type "Toggle Screen Reader Optimized" and press Enter.

2. Accessible View

- Press `Shift + Alt + F2` (Windows/Linux) or `Shift + Option + F2` (macOS).
- Opens a flat, text-based representation of complex UI elements.
- Press `Esc` to close.

3. Keyboard-First Navigation

- `Tab` and `Shift+Tab`: Move focus between UI elements.
- Arrow Keys: Navigate within active panes.

4. Status Bar The status bar at the bottom provides current line/column number, file type, active Python environment, and error/warning counts.

Screen Reader Indentation Configuration Python relies heavily on indentation. Configure your screen reader to announce indentation changes.

NVDA (Windows)

1. Press `NVDA Key + N` -> Preferences -> Settings.
2. Find "Document Formatting".
3. Locate "Indentation" and set it to **"Say all"** or **"Beeps"**.
4. Activate "OK".

JAWS (Windows)

1. Press `JAWS Key + J` -> Alt+O for Options -> B for Basics.
2. Find "Text Processing" settings.
3. Configure indentation announcement.

VoiceOver (macOS) VoiceOver generally handles indentation well by default, announcing "4 spaces" or "tab" as you navigate.

Skills Checklist

☐ I can identify and toggle VS Code's "Screen Reader Optimized Mode".
☐ I can open and close the "Accessible View".
☐ I can navigate between VS Code panes using Tab and Shift+Tab.
☐ I can navigate line by line and character by character within the editor.
☐ I can locate and listen to information in the VS Code Status Bar.
☐ I can configure my screen reader to announce indentation changes.

Lesson 0.3: Setting Up Your VS Code Workspace

Reading References

- Matthes: Appendix B, pages 469–476
- Matthes: Chapter 8, pages 149–152

Installing VS Code Extensions

1. Open the Extensions View: `Ctrl + Shift + X` (Windows/Linux) or `Command + Shift + X` (macOS).
2. Search and install **"Python"** by Microsoft — the official and most important Python extension.
3. Search and install **"Pylance"** — provides excellent IntelliSense and type checking.
4. Optional: **"Code Runner"** — allows running code files with `Ctrl+Alt+N`.

Creating Your First Workspace Folder **Windows:** Open File Explorer (`Windows Key + E`). Navigate to Documents. Press `Ctrl + Shift + N` to create a new folder named `PythonProjects`.

macOS: Open Finder. Navigate to Documents. Press `Command + Shift + N` to create `PythonProjects`.

Opening the Folder in VS Code **Method 1 (Recommended):** Open terminal, navigate to the folder, then:

```
code .
```

Method 2: In VS Code, press `Ctrl + K, Ctrl + O` to "Open Folder".

VS Code Navigation Shortcuts

Area	Shortcut
Explorer	`Ctrl + Shift + E`
Search	`Ctrl + Shift + F`
Extensions	`Ctrl + Shift + X`
Toggle Sidebar	`Ctrl + B`
Command Palette	`Ctrl + Shift + P`
Problems View	`Ctrl + Shift + M`
Next Error	`F8`
Previous Error	`Shift + F8`
Integrated Terminal	'Ctrl + "

Skills Checklist

☐ I can open the Extensions View in VS Code.
☐ I can search for and install the "Python" extension.
☐ I can search for and install the "Pylance" extension.
☐ I can create a new folder for Python projects.
☐ I can open a specific folder as a workspace in VS Code.
☐ I can navigate the Activity Bar, Explorer, Editor, and Panel using keyboard shortcuts.
☐ I can effectively use the Command Palette (`Ctrl + Shift + P`).

Lesson 0.4: Python Environments and Terminal Usage

Reading References

- Matthes: Chapter 1, pages 3–14
- Matthes: Chapter 8, pages 149–152
- Matthes: Chapter 11, pages 210–211

Creating Your First Python File

1. Ensure Explorer is focused: `Ctrl + Shift + E`.

2. Create new file: `Ctrl + N`.
3. Save: `Ctrl + S`. Name it `hello.py`.
4. Type:

```
print("Hello from VS Code!")
```

Running Python Scripts **From System Terminal:**

```
cd ~/Documents/PythonProjects
python3 hello.py
```

From VS Code Integrated Terminal:

1. Open terminal: 'Ctrl + "
2. The terminal opens in your project folder automatically.
3. Run: `python3 hello.py`

Python Virtual Environments Virtual environments keep project dependencies separate, preventing conflicts.

Create a virtual environment:

```
python3 -m venv .venv
```

Activate:

- Windows (Command Prompt): `.venv\Scripts\activate.bat`
- Windows (PowerShell): `.venv\Scripts\Activate.ps1`
- macOS/Linux: `source ./.venv/bin/activate`

You will hear `(.venv)` at the beginning of your prompt when active.

Deactivate:

```
deactivate
```

Select Interpreter in VS Code:

1. Press `Ctrl + Shift + P`.
2. Type "Python: Select Interpreter" and press Enter.
3. Select the one that says "Python 3.x.x (.venv)".

Install packages:

```
pip install requests
pip list
```

Skills Checklist

☐ I can create a new Python file (.py) in VS Code.
☐ I can write a simple `print()` statement.
☐ I can run a Python script from the system terminal.
☐ I can run a Python script from the integrated terminal in VS Code.

☐ I understand why virtual environments are useful.
☐ I can create, activate, and deactivate a virtual environment.
☐ I can select the correct Python interpreter in VS Code.
☐ I can install Python packages using pip.

Lesson 0.5: General Accessibility Tools and Strategies

VS Code Accessibility Features

- **Accessible View** (`Shift+Alt+F2`): Reveals hover information and error details.
- **Focus on Errors** (`F8` / `Shift+F8`): Jump between errors without visual scanning.
- **Go to Definition** (`F12`): Navigate to where a function or variable is defined.
- **Go to Reference** (`Shift+F12`): Find all usages of a symbol.
- **Go to Line/Column** (`Ctrl+G`): Jump to a specific line number.

Python Documentation The official Python documentation is well-structured and accessible: https://docs.python.org/3/

Unit 0 Completion Checklist

☐ Python 3 is installed and `python3 --version` works in your system terminal.
☐ VS Code is installed and `code --version` works in your system terminal.
☐ Your screen reader is configured to announce indentation changes in VS Code.
☐ You can open your PythonProjects folder in VS Code.
☐ You have installed the official "Python" VS Code extension.
☐ You can create a new .py file, write `print("Hello!")`, and save it.
☐ You can run your hello.py file from both terminals.
☐ You can create and activate a virtual environment (.venv).
☐ You can select the virtual environment interpreter in VS Code.
☐ You can install a package using pip.
☐ You know how to use `Ctrl + Shift + P` (Command Palette) and `F8` (Go to next error).

Congratulations! You now have a powerful and accessible development environment ready for your Python journey.

Unit 1: Foundations of Computing and Getting Started with Python

Overview This unit covers basic computer science concepts, setting up the Python environment, and writing and executing simple Python programs.

Objectives:

- Understand what computer science is and how hardware relates to software.
- Write and run your first Python program.
- Use variables to store and manipulate simple data types.

Lesson 1.1: Welcome to Computer Science

Reading References

- Zelle: Chapter 1, pages 1–7
- Matthes: Chapter 1, pages 3–14

What Is Computer Science? A computer is a **universal machine** — it can be programmed to solve any problem that can be broken down into a series of precise, logical steps. The true power of a computer doesn't come from its hardware but from the software we create.

Computer science is the study of problem-solving, not just programming. It's an interdisciplinary field involving breaking down complex problems, designing algorithms, and implementing efficient solutions.

Hardware vs. Software

Component	Description
Hardware	Physical components you can touch: keyboard, screen, processor, memory
Software	Instructions that direct hardware: programs, operating systems, apps
CPU	The processor that executes instructions
Memory (RAM)	Temporary storage for active programs and data
Storage	Permanent storage for files

Analogy: Think of a kitchen. The stove, refrigerator, and pots are **hardware**. A cookie recipe is **software** — it provides precise steps for the "hardware" (the cook) to follow.

Why Python? Python is an ideal language for beginners because of its:

- **Readability** — English-like syntax makes code easy to understand.
- **Simplicity** — Less "boilerplate" code than other languages.
- **Versatility** — Used in web development, data science, AI, and more.

The Problem-Solving Approach

1. Understand the problem.
2. Break it down into smaller pieces.
3. Design a step-by-step solution (an algorithm).
4. Implement the solution in Python.
5. Test and refine your code.

Activities **Activity 1.1.1: "Human Computer" Role-Play**

- In pairs, one student is the "computer" and the other is the "programmer."
- The programmer provides simple, precise instructions for a common task (e.g., "make a peanut butter and jelly sandwich").
- The "computer" follows only what is literally said.
- Purpose: Illustrate the need for precise, unambiguous instructions.

Activity 1.1.2: Identify Hardware and Software

- List five items you use daily.
- For each, identify whether it is hardware or software and explain your choice.

Skills Checklist

☐ I can define what computer science is.
☐ I can distinguish between computer hardware and software.
☐ I can explain what a computer program is.
☐ I can identify Python as a programming language.

Lesson 1.2: Your First Python Program — "Hello World!"

Reading References

- Zelle: Chapter 1, pages 7–13
- Matthes: Chapter 1, pages 3–14

The `print()` Function

Accessible output rule: Never print a raw list or dict. Always iterate with `enumerate` and label each item. Raw collection output (`[item1, item2, ...]`) is structurally noisy when read aloud by a screen reader.

The `print()` Function The `print()` function is Python's primary method for displaying output to the user.

```
print("Hello, World!")
```

Expected output:

```
Hello, World!
```

Key Concepts

- **Strings** are text data enclosed in quotation marks (single or double).
- **Comments** start with `#` and are ignored by Python.
- Python executes code **line by line**, from top to bottom.

Examples **Example 1.2.1: Simple Greeting**

```
# This program prints a classic greeting.
print("Hello, World!")
```

Example 1.2.2: Personalized Message

```
# This program prints a message with your name.
print("Hello, Alice!")
print("Welcome to Python programming!")
```

Expected output:

```
Hello, Alice!
Welcome to Python programming!
```

Common Mistakes

```
print("Hello, World"        # Missing closing parenthesis --- SyntaxError
print "Hello"               # Python 2 style --- SyntaxError in Python 3
Print("Hello")              # Capital P --- NameError
```

Activities **Activity 1.2.1: My First Message**

1. Open VS Code and confirm the PythonProjects folder is open.
2. Create a new file as `my_message.py`.
3. Type:

   ```
   print("My favorite color is blue.")
   print("My favorite animal is a dog.")
   ```

4. Save and run the file.

Activity 1.2.2: Exploring Print Statements

1. Experiment with:
   - Printing numbers: `print(123)`
   - Printing sentences with punctuation.
   - Omitting quotes: `print(Hello)` — expect a `NameError`.

Skills Checklist

☐ I can write a `print()` statement in Python.
☐ I can save a Python file with the `.py` extension.
☐ I can execute a Python script from the terminal.
☐ I understand that the output of `print()` appears in the terminal.

Lesson 1.3: Variables and Simple Data Types

Reading References

- Zelle: Chapter 2, pages 24–26
- Matthes: Chapter 2, pages 15–32

What Is a Variable?

What Is a Variable?

What Is a Variable? A **variable** is a name that refers to a value stored in the computer's memory. The **assignment operator** `=` creates a connection between a name and a value.

```
age = 25
name = "Alice"
height = 5.6
```

Rules for variable names:

- Must start with a letter or underscore
- Can only contain letters, digits, and underscores
- Are case-sensitive (`age` is different from `Age`)
- Cannot be a Python keyword (`if`, `for`, `while`, etc.)

Best practices:

- Use descriptive names: `student_count`, not `sc`
- Use lowercase with underscores: `first_name`

Essential Data Types

Type	Description	Examples
`str` (String)	Text data	`"hello"`, `"Python"`
`int` (Integer)	Whole numbers	`10`, `-5`, `0`
`float` (Float)	Numbers with decimals	`3.14`, `9.99`
`bool` (Boolean)	True or False	`True`, `False`

Accessibility Tip: Every `print()` call produces one line of terminal output. Navigate output with the arrow keys in your terminal. Use `Ctrl+End` to jump to the most recent output, then arrow up to read previous lines. Always add a descriptive label: `print("Name:", name)` rather than just `print(name)`.

Accessibility Tip: Give every variable a descriptive name that says what it contains: `student_age` rather than `x`. When your screen reader announces variable names during debugging, descriptive names make the output immediately meaningful.

Type	Description	Examples

Examples **Example 1.3.1: Storing and Printing a String**

```
student_name = "Charlie"
print("Hello, " + student_name + "!")
```

Output:

```
Hello, Charlie!
```

Example 1.3.2: Storing and Printing Numbers

```
apple_count = 5
orange_count = 3
total_fruit = apple_count + orange_count
print("You have", total_fruit, "fruits in total.")

price_per_item = 2.50
quantity = 4
total_cost = price_per_item * quantity
print("The total cost is:", total_cost)
```

Output:

```
You have 8 fruits in total.
The total cost is: 10.0
```

Common Errors

```
2nd_number = 10          # Invalid variable name --- starts with digit
print("Result: " + 42)   # TypeError --- can't concatenate str and int
```

Activities **Activity 1.3.1: Personal Information Variables**

```
my_name = "Jamie"
favorite_number = 7
height_in_meters = 1.75
favorite_food = "pizza"

print("My name is:", my_name)
print("My favorite number is:", favorite_number)
print("My height is:", height_in_meters, "meters")
print("My favorite food is:", favorite_food)
```

Activity 1.3.2: Simple Math Challenge

```
num1 = 15
num2 = 7
sum_result = num1 + num2
difference = num1 - num2
product = num1 * num2
quotient = num1 / num2
```

```
print("Sum:", sum_result)
print("Difference:", difference)
print("Product:", product)
print("Quotient:", quotient)
```

Key Table: Python Data Types and Operations

Data Type	Description	Example Value	Common Operations
`str`	Text, words, sentences	`"Hello"`, `"Python"`	`+` (concatenation), `len()`, `.upper()`
`int`	Whole numbers	`10`, `-5`, `0`	`+`, `-`, `*`, `/`, `%`
`float`	Numbers with decimals	`3.14`, `9.99`, `0.0`	`+`, `-`, `*`, `/`

Skills Checklist

☐ I can define what a variable is in programming.
☐ I can create a valid variable name in Python.
☐ I can assign string, integer, and float values to variables.
☐ I can use `print()` to display the value of a variable.
☐ I can perform basic arithmetic operations with numbers in Python.

Unit 2: Writing Simple Programs and Numeric Computations

Overview This unit explores the software development process, more complex numeric operations, and how to obtain and use input from users.

Objectives:

- Understand the phases of software development.
- Work with different numeric data types (integers, floats).
- Perform type conversions.
- Use mathematical operators and the math library.
- Get user input with `input()`.

Lesson 2.1: The Software Development Process

Reading References

- Zelle: Chapter 2, pages 21–22 and pages 35–36

The Three-Phase Development Process **1. Design (Plan first)** Before writing any code, fully understand the problem:

- Identify **inputs** (information the program needs) and **outputs** (results it must produce).
- Write an **algorithm** — a step-by-step plan in plain English (**pseudocode**).
- Consider **edge cases** — unusual inputs that might break your program.

2. Implement (Write the code)

- Translate your design into Python syntax.
- Code **incrementally** — build in small pieces and test as you go.
- Follow coding standards and use descriptive variable names.

3. Test and Debug (Verify it works)

- Run with a variety of inputs, including normal values and edge cases.
- Read error messages carefully — they tell you the type of error and the line number.
- Use `print()` statements to check variable values during debugging.

Why This Process Matters

- **Reduces frustration** — Having a plan prevents getting stuck.
- **Improves code quality** — A thoughtful design leads to cleaner code.
- **Simplifies debugging** — Test each component in isolation.
- **Saves time** — Good design prevents time-consuming rewrites.

Activities **Activity 2.1.1: Planning a Tip Calculator** Write pseudocode before coding:

```
1. GET bill amount from user
2. CALCULATE tip as 15% of bill amount
3. CALCULATE total amount (bill + tip)
4. PRINT tip amount
5. PRINT total amount
```

Activity 2.1.2: Tracing a Simple Program Without running the code, trace the execution and predict output:

```
a = 10
b = 5
c = a * b
a = a + 2
print(c)
print(a)
```

Skills Checklist

☐ I can describe the purpose of the software development process.
☐ I can list and briefly explain the main phases of software development.
☐ I understand that problem-solving and design happen before writing code.

Lesson 2.2: Elements of Programs — Expressions and Assignment

Reading References

- Zelle: Chapter 2, pages 24–26

Expressions and Assignment An **expression** is any combination of values, variables, and operators that evaluates to a single result. Python follows the standard mathematical order of operations (PEMDAS).

```
result1 = 10 / 2 + 3  # 10/2 is 5, then 5+3 = 8.0
message = "Python" + " " + "is fun!"
```

Assignment:

```
score = 50
score = score + 20  # score is now 70
```

Examples **Example 2.2.1: Expression Evaluation**

```
result1 = 10 / 2 + 3
print("Result 1:", result1)   # Output: Result 1: 8.0

message = "Python" + " " + "is fun!"
print("Message:", message)    # Output: Message: Python is fun!
```

Example 2.2.2: Reassigning Variables

```
score = 50
print("Initial score:", score)
score = score + 20
print("New score:", score)     # Output: New score: 70
```

Skills Checklist

☐ I can identify an expression in Python code.
☐ I can predict the result of simple Python expressions.
☐ I can explain how the assignment operator = works.
☐ I can demonstrate how to update a variable's value.

Lesson 2.3: Input and Output Revisited

Reading References

- Zelle: Chapter 2, pages 27–33
- Matthes: Chapter 7, pages 114–116

The `input()` Function

Accessibility Tip: Give every variable a descriptive name that says what it contains: `student_age` rather than `x`. When your screen reader announces variable names during debugging, descriptive names make the output immediately meaningful.

The `input()` Function The `input()` function pauses program execution and waits for the user to type something and press Enter. **It always returns a string.**

```
user_name = input("What is your name? ")
print(f"Hello, {user_name}! Nice to meet you.")
```

F-Strings (Formatted String Literals) F-strings provide a clean way to embed variable values directly into a string:

```
name = "Alice"
age = 30
print(f"Hello, {name}! You are {age} years old.")
```

Converting Input for Calculations Since `input()` always returns a string, you must convert it for math:

```
age_str = input("How old are you? ")
age_int = int(age_str)
print(f"Next year you will be {age_int + 1} years old.")
```

Examples **Example 2.3.1: Basic Input**

```
user_name = input("What is your name? ")
print(f"Hello, {user_name}! Nice to meet you.")
```

Example 2.3.2: Numeric Input and Calculation

```
num1_str = input("Enter the first number: ")
num1 = int(num1_str)

num2_str = input("Enter the second number: ")
num2 = int(num2_str)

sum_result = num1 + num2
print(f"The sum of {num1} and {num2} is {sum_result}.")
```

Activities **Activity 2.3.1: Temperature Converter**

```
fahrenheit_str = input("Enter temperature in Fahrenheit: ")
fahrenheit = float(fahrenheit_str)
celsius = (fahrenheit - 32) * 5 / 9
print(f"{fahrenheit}°F is {celsius:.2f}°C")
```

Skills Checklist

☐ I can use the `input()` function to ask the user for information.
☐ I can provide a clear prompt message when using `input()`.
☐ I remember that `input()` returns a string, even for numbers.
☐ I can combine text and variable values in a `print()` statement using f-strings.

Lesson 2.4: Computing with Numbers and Type Conversions

Reading References

- Zelle: Chapter 3, pages 41–48
- Matthes: Chapter 2, pages 26–28

Integers and Floats

Type	Description	Example
`int`	Whole numbers (exact)	`5`, `-20`, `1000000`
`float`	Numbers with decimals	`3.14`, `-2.5`, `6.0`

Type	Description	Example

Key behaviors:

- Division `/` always returns a float: `10 / 2` -> `5.0`
- Floor division `//` returns the integer part: `10 // 3` -> `3`
- Mixed operations promote to float: `10 + 3.5` -> `13.5`

Type Conversion Functions

```
int("123")      # -> 123  (truncates: int(3.9) -> 3)
float("3.14")   # -> 3.14
str(42)         # -> "42"
```

Examples **Example 2.4.1: Type Conversion**

```
num_str = "123"
num_int = int(num_str)
print(f"String '{num_str}' as integer: {num_int}, type: {type(num_int)}")

num_float = float(num_str)
print(f"String '{num_str}' as float: {num_float}, type: {type(num_float)}")

my_number = 42
my_text = str(my_number)
print(f"Integer {my_number} as string: '{my_text}', type: {type(my_text)}")
```

Common Type Error:

```
# This will cause a TypeError:
# print("My age is " + 14)
# Fix: print("My age is " + str(14))
# Or: print(f"My age is {14}")
```

Activities **Activity 2.4.1: Mixed Bag Calculator**

```
age_str = input("Enter your age: ")
age = int(age_str)

weight_str = input("Enter your weight in kg: ")
weight = float(weight_str)

phrase = input("Enter a short phrase: ")

print(f"Age doubled: {age * 2}")
print(f"Weight + age: {weight + age}")
print(f"Phrase length: {len(phrase)} characters")
```

Skills Checklist

- ☐ I can identify when to use an integer versus a float.
- ☐ I can convert a string to an integer using `int()`.
- ☐ I can convert a string to a float using `float()`.
- ☐ I can convert a number to a string using `str()`.
- ☐ I can apply type conversions to perform calculations on user input.
- ☐ I understand and can apply the order of operations.

Lesson 2.5: Using the Math Library

Reading References

- Zelle: Chapter 3, pages 49–51

Importing the Math Module

```
import math
```

Key Math Functions and Constants

Function/Constant	Description	Example
`math.pi`	π ≈ 3.14159	`math.pi`
`math.e`	e ≈ 2.71828	`math.e`
`math.sqrt(x)`	Square root of x	`math.sqrt(25)` -> `5.0`
`math.pow(x, y)`	x raised to power y	`math.pow(2, 4)` -> `16.0`
`math.ceil(x)`	Round up	`math.ceil(4.2)` -> `5`
`math.floor(x)`	Round down	`math.floor(4.8)` -> `4`
`math.log(x, base)`	Logarithm	`math.log(100, 10)` -> `2.0`
`math.sin(x)`	Sine (in radians)	`math.sin(math.pi/2)` -> `1.0`
`math.radians(deg)`	Convert degrees to radians	`math.radians(90)`

Examples **Example 2.5.1: Circle Calculations**

```
import math

radius = 5
area = math.pi * radius ** 2
circumference = 2 * math.pi * radius

print(f"Area of circle with radius {radius}: {area:.2f}")
print(f"Circumference: {circumference:.2f}")

number = 81
square_root = math.sqrt(number)
```

```
print(f"Square root of {number}: {square_root}")
```

Activities **Activity 2.5.1: Circle Calculator**

```
import math

radius_str = input("Enter the radius: ")
radius = float(radius_str)

area = math.pi * radius ** 2
circumference = 2 * math.pi * radius

print(f"Area: {area:.2f}")
print(f"Circumference: {circumference:.2f}")
```

Activity 2.5.2: Distance Finder

```
import math

x = float(input("Enter x coordinate: "))
y = float(input("Enter y coordinate: "))
distance = math.sqrt(x**2 + y**2)
print(f"Distance from origin: {distance:.2f}")
```

Skills Checklist

☐ I can explain what a Python module is.
☐ I can import the `math` module into my program.
☐ I can use `math.pi` and `math.sqrt()`.
☐ I can use `math.ceil()` to round up and `math.floor()` to round down.

Unit 3: Control Flow — Decisions and Loops

Overview This unit introduces how programs make decisions using `if` statements and perform repetitive tasks with `for` loops, along with fundamental string manipulation techniques.

Objectives:

- Control program flow with `if`, `elif`, and `else`.
- Iterate over sequences using `for` loops.
- Understand and manipulate strings.

Lesson 3.1: Boolean Expressions and `if` Statements

Reading References

- Zelle: Chapter 7, pages 165–173
- Matthes: Chapter 5, pages 71–78

Comparison and Boolean Operators

Accessibility Tip: `input()` pauses your program and waits for you to type. Include a clear prompt describing exactly what to type: `input("Enter your age as a whole number: ")`. After pressing Enter, immediately call `print(type(variable))` to verify the value is the type you expect.

Comparison and Boolean Operators

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 5`	`True`
`!=`	Not equal to	`5 != 10`	`True`
`<`	Less than	`5 < 10`	`True`
`>`	Greater than	`10 > 5`	`True`
`<=`	Less than or equal to	`5 <= 5`	`True`
`>=`	Greater than or equal	`10 >= 5`	`True`

Boolean Operator	Description	Example	Result
`and`	True if both are true	`True and False`	`False`
`or`	True if at least one is true	`True or False`	`True`
`not`	Inverts the truth value	`not True`	`False`

The `if` Statement

Accessibility Tip: Pronounce comparison operators as words: `>` is "greater than," `==` is "equals equals" (never just "equals" — that is assignment). When reading a condition aloud: `if age >= 18 and verified:` becomes "if age is greater than or equal to 18 and verified."

The `if` Statement

```
age = 15
if age >= 18:
    print("You are old enough to vote.")
print("Program finished.")  # This runs regardless
```

Indentation defines which lines belong to the `if` block.

Examples **Example 3.1.1: Simple Age Check**

```
age = 17
if age >= 18:
    print("You are an adult.")
else:
    print("You are a minor.")
```

Example 3.1.2: Password Length Check

```
password = "mysecret"
if len(password) < 8:
    print("Weak password: Must be at least 8 characters.")
```

Activities **Activity 3.1.1: Number Comparison Game**

```
num1 = int(input("Enter the first number: "))
num2 = int(input("Enter the second number: "))

if num1 > num2:
    print(f"{num1} is greater than {num2}.")
if num1 == num2:
    print("The numbers are equal.")
if num1 < num2:
    print(f"{num1} is less than {num2}.")
```

Skills Checklist

☐ I can identify `True` and `False` as Boolean values.
☐ I can use comparison operators to create Boolean expressions.
☐ I can write a basic `if` statement.
☐ I can correctly indent the code block belonging to an `if` statement.

Lesson 3.2: Chained Conditionals (`elif`, `else`)

Reading References

- Zelle: Chapter 7, pages 173–176
- Matthes: Chapter 5, pages 79–84

`if-elif-else` Structure Python checks each condition in **sequential order**. As soon as a condition is `True`, it runs that block and skips the rest.

```
score = 85

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
else:
    print("Grade: F")
```

Output: `Grade: B`

Examples **Example 3.2.1: Stages of Life**

```
age = 25
if age < 2:
    print("You are a baby.")
elif age < 4:
    print("You are a toddler.")
elif age < 13:
    print("You are a kid.")
elif age < 20:
    print("You are a teenager.")
else:
    print("You are an adult.")
```

Activities **Activity 3.2.1: Movie Ticket Pricing**

```
age = int(input("Enter your age: "))

if age <= 5:
    print("Ticket price: Free")
elif age <= 12:
    print("Ticket price: $7")
elif age <= 64:
    print("Ticket price: $12")
else:
    print("Ticket price: $5")
```

Skills Checklist

☐ I can use an `else` statement to provide an alternative path.
☐ I can use `elif` to check multiple conditions in order.
☐ I can use `and`, `or`, and `not` operators to combine conditions.

Lesson 3.3: Introduction to Lists

Reading References

- Zelle: Chapter 5, pages 105–106
- Matthes: Chapter 3, pages 33–47

Creating and Accessing Lists

Creating and Accessing Lists

```
colors = ["red", "blue", "green", "yellow"]

print(colors[0])    # "red"    (first item)
print(colors[-1])   # "yellow" (last item)
print(len(colors)) # 4
```

Zero-based indexing: The first item is at index `0`.

Modifying Lists

```
fruits = ["apple", "banana", "cherry"]
fruits[1] = "grape"  # Change "banana" to "grape"
print(fruits)         # ["apple", "grape", "cherry"]
```

Activities **Activity 3.3.1: My Top 3 Hobbies**

```
hobbies = ["reading", "coding", "hiking"]
print("My hobbies:", hobbies)
print("My second hobby:", hobbies[1])
hobbies[2] = "swimming"
print("Updated hobbies:", hobbies)
```

Skills Checklist

☐ I can create a list in Python.
☐ I can access individual items in a list using their index.
☐ I can access list items from the end using negative indices.
☐ I can change the value of an item at a specific index.

Lesson 3.4: Iterating with `for` Loops

Reading References

- Zelle: Chapter 2, pages 34–36
- Matthes: Chapter 4, pages 49–60

The `for` Loop Structure

Accessibility Tip: Every `if`, `elif`, `else`, `for`, `while`, `def`, and `class` line must end with a colon. When your screen reader navigates into an indented block, it announces the indent level. If it does not, your indentation announcement is not configured—stop and fix this before writing any more code.

Accessibility Tip: Never use `print(my_list)` to inspect a list. Use `for i, item in enumerate(my_list, 1): print(f"{i}. {item}")` instead. This gives numbered, one-item-per-line output that is easy to navigate with arrow keys in the terminal.

```
for item in sequence:
    # code that runs for each item
```

Examples **Example 3.4.1: Looping Through a List**

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(f"I like {fruit}.")
print("Finished listing fruits.")
```

Example 3.4.2: Looping Through a String

```
word = "Python"
for char in word:
    print(char)
```

Example 3.4.3: Using `range()`

```
for i in range(5):       # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 8):    # 2, 3, 4, 5, 6, 7
    print(i)

for i in range(0, 10, 2):  # 0, 2, 4, 6, 8
    print(i)
```

Activities **Activity 3.4.1: My Favorite Things**

```
favorite_things = ["sunshine", "coffee", "books", "music", "mountains"]
for thing in favorite_things:
    print(f"I love {thing}!")
```

Activity 3.4.2: Countdown Timer

```
start = int(input("Enter a starting number: "))
for i in range(start, 0, -1):
    print(i)
print("Blast off!")
```

Skills Checklist

☐ I can explain the purpose of a `for` loop.
☐ I can write a `for` loop to iterate over items in a list.
☐ I can write a `for` loop to iterate over characters in a string.
☐ I can use the `range()` function to control how many times a loop runs.

Lesson 3.5: Basic String Operations

Reading References

- Zelle: Chapter 5, pages 99–115
- Matthes: Chapter 2, pages 19–25

Common String Methods

```
message = "  Hello Python!  "

message.strip()          # "Hello Python!"   --- removes whitespace
message.upper()          # "  HELLO PYTHON!  "
message.lower()          # "  hello python!  "
message.replace("Hello", "Hi")  # "  Hi Python!  "
len(message)             # 17
```

Important: String methods return a **new string** — they do not modify the original.

Examples **Example 3.5.1: String Manipulation**

```
message = "  Hello Python  "
print(f"Original: '{message}'")
print(f"Length: {len(message)}")
print(f"Uppercase: '{message.upper()}'")
print(f"Stripped: '{message.strip()}'")
```

Example 3.5.2: Combining Strings

```
first_name = "Alice"
last_name = "Smith"
full_name = first_name + " " + last_name
print(f"Full name: {full_name}")
```

Activities **Activity 3.5.1: Name Formatter**

```
first_name = input("Enter your first name: ")
last_name = input("Enter your last name: ")
full_name = first_name + " " + last_name
print(f"Full name: {full_name}")
print(f"Uppercase: {full_name.upper()}")
print(f"Total characters: {len(full_name)}")
```

Skills Checklist

☐ I can combine strings using concatenation.
☐ I can find the length of a string using `len()`.
☐ I can convert a string to uppercase/lowercase.
☐ I can remove extra spaces using `.strip()`.
☐ I can replace parts of a string using `.replace()`.

Unit 4: Data Structures — Lists and Dictionaries

Overview This unit deepens understanding of lists and introduces dictionaries as powerful mechanisms for organizing data.

Objectives:

- Add, remove, and organize list elements.
- Work with tuples (immutable lists).
- Use dictionaries for key-value pair storage.
- Loop through dictionaries.

Lesson 4.1: More on Lists

Adding and Removing Elements

```
my_list = [10, 20, 30]

my_list.append(40)          # Add to end -> [10, 20, 30, 40]
my_list.insert(1, 15)       # Insert at index 1 -> [10, 15, 20, 30, 40]
del my_list[0]              # Delete by index -> [15, 20, 30, 40]
removed = my_list.pop()     # Remove and return last -> removed=40, list=[15, 20, 30]
my_list.remove(20)          # Remove by value -> [15, 30]
```

Example

```
todo_list = ["Buy groceries", "Walk the dog"]

todo_list.append("Call grandma")
todo_list.insert(0, "Wake up early")
del todo_list[1]
completed_task = todo_list.pop()
todo_list.remove("Walk the dog")
print(todo_list)
```

Skills Checklist

☐ I can add an item to the end of a list using `.append()`.
☐ I can insert an item at a specific position using `.insert()`.
☐ I can remove an item by index using `del`.
☐ I can remove an item and retrieve its value using `.pop()`.
☐ I can remove the first occurrence of a value using `.remove()`.

Lesson 4.2: Organizing Lists

Sorting and Reversing

```
numbers = [5, 2, 9, 1]
numbers.sort()                          # Permanent sort in-place
print(numbers)                          # [1, 2, 5, 9]

fruits = ["orange", "apple", "banana"]
sorted_fruits = sorted(fruits)          # Temporary sort --- original unchanged
print(fruits)                           # ['orange', 'apple', 'banana']
print(sorted_fruits)                    # ['apple', 'banana', 'orange']

numbers.reverse()                       # Reverse in-place
numbers.sort(reverse=True)              # Sort descending
print(len(numbers))                     # Length
```

Skills Checklist

☐ I can sort a list permanently using `.sort()`.
☐ I can get a temporarily sorted version using `sorted()`.
☐ I can reverse a list using `.reverse()`.
☐ I can find how many items are in a list using `len()`.

Lesson 4.3: Tuples — Immutable Lists

Tuples Tuples are defined with parentheses and **cannot be changed** after creation.

```
coordinates = (10, 20, 30)
print(coordinates[0])     # 10
print(coordinates[-1])    # 30

# This would cause a TypeError:
# coordinates[1] = 25
```

When to use tuples: For data that should remain constant (coordinates, RGB values, fixed settings).

Skills Checklist

☐ I can create a tuple in Python.
☐ I can explain the main difference between a list and a tuple (mutability).
☐ I can access items in a tuple by their index.

Lesson 4.4: Introducing Dictionaries

Reading References

- Zelle: Chapter 11, pages 307–309
- Matthes: Chapter 6, pages 91–98

Dictionary Basics

Dictionary Basics A dictionary stores **key-value pairs**. Keys must be unique and immutable (strings, numbers, tuples). Values can be anything.

```
student_profile = {}
    "name": "Emily",
    "age": 13,
    "grade": "7th",
    "favorite_subject": "Science"

print(student_profile["name"])      # "Emily"
student_profile["city"] = "Springfield"   # Add new key
student_profile["age"] = 14               # Modify existing key
del student_profile["grade"]              # Remove key
```

Examples **Example 4.4.1: Book Information Dictionary**

```
my_book = {}
    "title": "The Hitchhiker's Guide to the Galaxy",
    "author": "Douglas Adams",
    "publication_year": 1979,
    "genre": "Science Fiction"

print(f"Title: {my_book['title']}")
print(f"Author: {my_book['author']}")

my_book["number_of_pages"] = 193
my_book["genre"] = "Comedy Science Fiction"
print(my_book)
```

Skills Checklist

- ☐ I can create a dictionary in Python.
- ☐ I can explain what a key-value pair is.
- ☐ I can access a value in a dictionary using its key.
- ☐ I can add, modify, and remove key-value pairs.

Lesson 4.5: Looping Through Dictionaries

Dictionary Iteration Methods

```
inventory = {}"apples": 50, "bananas": 30, "oranges": 40
# Loop through keys (default)
for item_name in inventory:
    print(item_name)

# Loop through values only
```

Accessibility Tip: Before running a `for` loop, trace it aloud: state what variable changes, what range or list it covers, and what the body does each iteration. This verbal trace catches errors before you hear many lines of wrong output.

```python
for count in inventory.values():
    print(count)

# Loop through key-value pairs
for item_name, count in inventory.items():
    print(f"{item_name}: {count} in stock")
```

Skills Checklist

☐ I can loop through all the keys in a dictionary.
☐ I can loop through all the values in a dictionary.
☐ I can loop through both keys and values at the same time.

Unit 5: User Input and `while` Loops

Overview This unit focuses on obtaining diverse input from users and controlling program execution through indefinite loops.

Objectives:

- Implement indefinite loops using `while` statements.
- Control `while` loop execution with `break` and `continue`.
- Avoid infinite loops.

Lesson 5.1: How the `input()` Function Works

Key Reminder `input()` **always returns a string**. Convert to numeric types as needed.

```python
age_input = input("How old are you? ")
print(f"Type: {type(age_input)}")    # <class 'str'>

age_int = int(age_input)
print(f"In 5 years: {age_int + 5}")
```

Skills Checklist

☐ I can effectively use `input()` to get user data.
☐ I can confirm that `input()`'s return value is always a string.
☐ I can convert user input to a numeric type when necessary.

Lesson 5.2: The `while` Loop in Action

`for` vs. `while` Loop Comparison

Accessibility Tip: Use `my_dict.get(key, "not found")` instead of `my_dict[key]`. If the key does not exist, `my_dict[key]` raises `KeyError` and crashes; `.get()` returns your default value silently. Crashes mid-session can be hard to detect with a screen reader.

`for` vs. `while` Loop Comparison

Feature	`for` Loop	`while` Loop
Purpose	Definite iteration (known count)	Indefinite iteration (condition-based)
When to Use	Iterating over lists, ranges	Repeating until a condition is met
Control Variable	Automatically updated	Must be manually updated
Risk	Less prone to infinite loops	High risk if condition never becomes false

Examples **Example 5.2.1: Simple Counter**

```
count = 1
while count <= 5:
    print(count)
    count += 1
print("Counting finished!")
```

Example 5.2.2: User Input Loop (Sentinel Value)

```
message = ""
while message != 'quit':
    message = input("Enter a message, or type 'quit' to exit: ")
    if message != 'quit':
        print(f"You said: {message}")
print("Goodbye!")
```

Skills Checklist

☐ I can explain the difference between a `for` loop and a `while` loop.
☐ I can write a basic `while` loop.
☐ I can make sure my `while` loop has a way to stop.

Lesson 5.3: Using Flags and `break` to Exit Loops

Flag Variable

```
active = True
while active:
    message = input("Say something (or 'quit' to stop): ")
    if message == 'quit':
        active = False
    else:
        print(f"You said: {message}")
```

`break` Statement

```python
while True:
    city = input("Enter a city (or 'done' to finish): ")
    if city == 'done':
        break
    print(f"I've heard {city} is a great place!")
```

Skills Checklist

☐ I can use a Boolean "flag" variable to control a `while` loop.
☐ I can use the `break` statement to exit a loop early.

Lesson 5.4: Using `continue` in Loops

`continue` — Skip the Current Iteration

```python
number = 0
while number < 10:
    number += 1
    if number % 2 == 0:
        continue        # Skip even numbers
    print(number)       # Prints 1, 3, 5, 7, 9
```

`continue` vs. `break`

- `break` — **exits the loop entirely**
- `continue` — **skips only the current iteration**, loop continues

Skills Checklist

☐ I can use `continue` to skip the rest of the current loop iteration.
☐ I can identify scenarios where `continue` would be useful.

Lesson 5.5: Avoiding Infinite Loops An **infinite loop** never ends because its condition always remains `True`.

Common causes:

1. Forgetting to update the loop variable.
2. Updating the variable in the wrong direction.
3. Using an incorrect logical condition.

```python
# INFINITE LOOP --- Don't run!
count = 1
while count <= 5:
    print(count)
    # Missing: count += 1

# CORRECT
```

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

17 18

Skills Checklist

☐ I can define what an infinite loop is.
☐ I can identify common causes of infinite loops.
☐ I know how to stop a running program in an infinite loop.

Unit 6: Functions

Overview This unit introduces functions, enabling organization of code into reusable blocks, passing arguments, and returning values.

Objectives:

- Define and call functions.
- Pass arguments to functions.
- Return values from functions.
- Understand the concept of scope.

Lesson 6.1: Defining Functions

Reading References

- Zelle: Chapter 6, pages 137–142
- Matthes: Chapter 8, pages 129–130

Function Syntax

Function Syntax

```
def function_name(parameter1, parameter2):
    """Docstring describing what the function does."""
    # Function body
    print(f"Hello, {parameter1}!")
```

Examples **Example 6.1.1: Simple Greeting Function**

```
def greet_user():
    """Prints a simple greeting message."""
    print("Hello, Python programmer!")
```

Accessibility Tip: Use `for` when you know the number of iterations in advance. Use `while` when you are waiting for a condition to change (user input, file end, sensor value). Every `while` loop must update the condition variable inside the loop body or it will run forever.

[17] `Ctrl+C` sends a `SIGINT`

Accessibility Tip: When a program hangs or enters an infinite loop, press `Ctrl+C` in the terminal to stop it immediately. If the screen reader does not announce the new prompt after stopping, press Enter once to restore focus. Then use the Up arrow to recall the last command.

[18] `Ctrl+C` sends a `SIGINT` (interrupt signal) to the running process. On Windows terminals with some screen readers, focus may not return to the prompt automatically after `Ctrl+C`—press Enter once if the cursor seems unresponsive.

```
greet_user()   # Call the function
greet_user()   # Call it again
```

Example 6.1.2: Function with a Parameter

```
def greet_name(name):
    """Prints a personalized greeting."""
    print(f"Hello, {name}!")

greet_name("Alice")
greet_name("Bob")
```

Skills Checklist

☐ I can explain why functions are useful.
☐ I can define a simple function in Python.
☐ I can use parameters to specify function inputs.
☐ I can call a function by its name.

Lesson 6.2: Passing Arguments

Positional vs. Keyword Arguments

```
def describe_animal(animal_type, animal_name):
    print(f"I have a {animal_type} named {animal_name}.")

# Positional arguments (order matters)
describe_animal("dog", "Buddy")

# Keyword arguments (order doesn't matter)
describe_animal(animal_name="Max", animal_type="cat")
```

Default Values

```
def make_shirt(size, message="I love Python"):
    print(f"Making a {size}-sized shirt: '{message}'")

make_shirt("large")                      # Uses default message
make_shirt("medium", "Hello World")      # Custom message
```

Skills Checklist

☐ I can differentiate between a parameter and an argument.
☐ I can pass arguments using positional order.
☐ I can pass arguments using keyword arguments.

Lesson 6.3: Return Values

The `return` Statement

```
def add_numbers(num1, num2):
    """Adds two numbers and returns the result."""
    total = num1 + num2
    return total

result = add_numbers(5, 3)
print(f"The sum is: {result}")   # 8
```

Accessibility Tip: Every function you write should have a docstring as its first line: `"""Describe what this function does."""`. When IntelliSense shows the function, it reads the docstring aloud. This is how you document a function for both yourself and anyone else who uses it.

This distinction trips up nearly every beginner. A function that `print()`s its result

Accessibility Tip: When testing a function, call it and print the result: `print(my_function(5))`. If the function uses `return`, you must capture and print the value yourself. If the function uses `print()` internally, calling it will display output automatically—but the value cannot be reused.

This distinction trips up nearly every beginner. A function that `print()`s its result cannot be composed with other functions—the value disappears into the terminal. A function that `return`s its result can be stored, passed, or tested.

Skills Checklist

- ☐ I can use `return` to send a value back from a function.
- ☐ I can explain the difference between `print()` and `return`.
- ☐ I can store the returned value of a function in a variable.

Lesson 6.4: Functions with Lists and Dictionaries [19]

```
def add_new_user(user_list, new_user):
    user_list.append(new_user)

users = ['Alice', 'Bob']
add_new_user(users, 'Charlie')
print(users)   # ['Alice', 'Bob', 'Charlie']
```

To prevent modification, pass a copy:

```
add_new_user(users[:], 'Charlie')   # Original unchanged
```

Skills Checklist

- ☐ I can pass a list or dictionary as an argument to a function.
- ☐ I understand that modifying a list inside a function affects the original.
- ☐ I can create a copy of a list to avoid unintended modifications.

Lesson 6.5: Arbitrary Number of Arguments

`*args` — Arbitrary Positional Arguments

```
def make_pizza(size, *toppings):
    print(f"Making a {size}-inch pizza with:")
    for topping in toppings:
        print(f"  - {topping}")

make_pizza(12, 'pepperoni')
make_pizza(16, 'mushrooms', 'green peppers', 'extra cheese')
```

`**kwargs` — Arbitrary Keyword Arguments

```
def build_profile(first, last, **user_info):
    user_info['first_name'] = first
    user_info['last_name'] = last
    return user_info

profile = build_profile('albert', 'einstein',
                        location='princeton',
                        field='physics')
print(profile)
```

Skills Checklist

- ☐ I can define a function using `*args` for arbitrary positional arguments.
- ☐ I can define a function using `**kwargs` for arbitrary keyword arguments.

Unit 7: Classes and Object-Oriented Programming

Overview This unit introduces the fundamental concepts of Object-Oriented Programming (OOP) in Python, focusing on defining and using classes, attributes, methods, and inheritance.

Objectives:

- Define and instantiate classes.
- Work with attributes and methods.
- Understand inheritance and composition.

Lesson 7.1: Why Learn About Classes?

Reading References

- Zelle: Chapter 4, pages 65–67; Chapter 10, pages 247–248
- Matthes: Chapter 9, page 157

Classes as Blueprints A **class** is a blueprint or template for creating objects. An **object** is a concrete instance created from that blueprint.

Analogy: A "Car" class defines that every car has a `color` and a `speed` (attributes) and can `accelerate()` and `brake()` (methods). Your red sedan and your friend's blue pickup are two separate **objects** of the same "Car" class.

Key OOP Concepts

- **Encapsulation** — Bundling data and methods into a single unit.
- **Abstraction** — Hiding complexity from the outside.
- **Inheritance** — New class reuses functionality of existing class.
- **Polymorphism** — Same method name behaves differently for different classes.

Skills Checklist

☐ I can explain the basic idea of Object-Oriented Programming.
☐ I can describe a class as a blueprint and an object as an instance.

Lesson 7.2: Creating and Using a Class

Reading References

- Zelle: Chapter 10, pages 248–254
- Matthes: Chapter 9, pages 158–162

Class Syntax

```
class Dog:
    """A simple attempt to model a dog."""

    def __init__(self, name, age):
        """Initialize name and age attributes."""
        self.name = name
        self.age = age
        print(f"A new dog named {self.name} has been created!")

# Create instances
my_dog = Dog("Buddy", 3)
your_dog = Dog("Lucy", 1)

print(f"My dog's name is {my_dog.name}.")
print(f"Your dog's name is {your_dog.name}.")
```

The `__init__()` method runs automatically when a new object is created. `self` refers to the specific instance being created.

Skills Checklist

☐ I can define a basic Python class.
☐ I can define the `__init__` method within a class.
☐ I understand the purpose of the `self` parameter.
☐ I can create an instance (object) of a class.

Lesson 7.3: Working with Attributes and Methods

Reading References

- Matthes: Chapter 9, pages 162–166

Calling Methods

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def sit(self):
        """Simulate a dog sitting."""
        print(f"{self.name} is now sitting.")

    def roll_over(self):
        """Simulate a dog rolling over."""
        print(f"{self.name} rolled over!")

    def get_older(self, years_to_add):
        """Add years to the dog's age."""
```

```
        self.age += years_to_add
        print(f"{self.name} is now {self.age} years old.")

my_dog = Dog("Buddy", 3)
my_dog.sit()
my_dog.roll_over()
my_dog.age = 4          # Direct attribute modification
my_dog.get_older(2)     # Method-based modification
```

Skills Checklist

☐ I can access an object's attributes using dot notation.
☐ I can define and call methods within a class.
☐ I can use a method to change an object's attribute.

Lesson 7.4: Inheritance

Reading References

- Zelle: Chapter 12, pages 351–352
- Matthes: Chapter 9, pages 167–170

Creating a Child Class

```
class Car:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year
        self.odometer_reading = 0

    def get_descriptive_name(self):
        return f"{self.year} {self.make} {self.model}".title()

    def read_odometer(self):
        print(f"This car has {self.odometer_reading} miles on it.")


class ElectricCar(Car):
    def __init__(self, make, model, year):
        super().__init__(make, model, year)    # Call parent's __init__
        self.battery_size = 75

    def describe_battery(self):
        print(f"This car has a {self.battery_size}-kWh battery.")


my_ev = ElectricCar('ford', 'mustang mach e', 2023)
```

```
print(my_ev.get_descriptive_name())    # Inherited method
my_ev.describe_battery()               # New method
my_ev.read_odometer()                  # Inherited method
```

`super()` Function `super().__init__()` calls the parent class's `__init__()` method, ensuring inherited attributes are properly initialized.

Skills Checklist

☐ I can define a child class that inherits from a parent class.
☐ I can use `super().__init__()` to properly initialize inherited attributes.
☐ I can add new attributes and methods to a child class.
☐ I can override a parent method in a child class.

Lesson 7.5: Instances as Attributes (Composition)

Composition — "Has-A" Relationship Instead of inheriting, a class **contains an instance of another class** as an attribute.

```
class Battery:
    def __init__(self, battery_size=75):
        self.battery_size = battery_size

    def describe_battery(self):
        print(f"This car has a {self.battery_size}-kWh battery.")

    def get_range(self):
        if self.battery_size == 75:
            range_miles = 260
        else:
            range_miles = 310
        print(f"About {range_miles} miles of range.")


class ElectricCar:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year
        self.battery = Battery()    # Composition: ElectricCar HAS A Battery


my_ev = ElectricCar('ford', 'mustang mach e', 2023)
my_ev.battery.describe_battery()
my_ev.battery.get_range()
```

Skills Checklist

☐ I can explain the concept of composition in OOP.

☐ I can create an instance of one class and assign it as an attribute of another.
☐ I can access attributes and methods of a nested object.

Unit 8: Files and Exceptions

Overview This unit covers interacting with text files for reading and writing data, handling common errors gracefully using exceptions, and storing data using JSON.

Objectives:

- Read from and write to files.
- Handle exceptions with `try-except` blocks.
- Store and load data using JSON.

Lesson 8.1: Reading from a File

Reading References

- Zelle: Chapter 5, pages 125–127
- Matthes: Chapter 10, pages 184–189

Reading Files

Reading Files

```
# Reading entire file
with open('my_notes.txt', 'r') as file_object:
    contents = file_object.read()
print(contents)

# Reading line by line
with open('my_notes.txt', 'r') as file_object:
    for line in file_object:
        print(line.strip())

# Reading all lines into a list
with open('my_notes.txt', 'r') as file_object:
    lines = file_object.readlines()
```

The `with` statement **automatically closes the file** even if an error occurs.

File Modes

[19] Mutability is one of Python's most common sources of subtle bugs. If you want a function to work on a *copy* of a list, pass `my_list[:]` or `list(my_list)` instead of the original. This is especially important in tests where the same list is reused across multiple test cases.

Mode	Description
`'r'`	Read (default)
`'w'`	Write (creates new or overwrites)
`'a'`	Append (adds to end)

Skills Checklist

☐ I can open a file for reading using `with open()`.
☐ I can read the entire content of a file into a string.
☐ I can read a file line by line using a `for` loop.
☐ I can remove whitespace/newlines using `.strip()`.

Lesson 8.2: Writing to a File

Reading References

- Zelle: Chapter 5, pages 127–128
- Matthes: Chapter 10, pages 190–192

```python
# Write mode (overwrites existing content)
with open('programming_message.txt', 'w') as f:
    f.write("I love programming in Python!\n")

# Append mode (adds to end)
with open('programming_message.txt', 'a') as f:
    f.write("It's a very versatile language.\n")
    f.write("I'm excited to learn more!\n")
```

Important: The `.write()` method does not add newlines automatically. Use `\n`.

Skills Checklist

☐ I can open a file for writing using `"w"` mode.
☐ I can open a file for appending using `"a"` mode.
☐ I can write a string to a file.
☐ I can add newlines to a file using `\n`.

Lesson 8.3: Exceptions — Handling Errors

Reading References

- Zelle: Chapter 7, pages 177–179
- Matthes: Chapter 10, pages 192–199

`try-except` Block

`try-except` Block

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Error: You cannot divide by zero!")
print("Program continues after error handling.")
```

Accessibility Tip: Always use `with open(filename) as f:` to open files. The `with` block guarantees the file is closed when you are done, even if an error occurs. Forgetting to close files can cause data loss that produces no error message.

Common Exception Types

Exception	When it occurs
`ValueError`	Invalid value (e.g., `int("abc")`)
`ZeroDivisionError`	Division by zero
`FileNotFoundError`	File doesn't exist
`IndexError`	List index out of range
`KeyError`	Dictionary key doesn't exist
`TypeError`	Wrong type (e.g., `"5" + 5`)

Examples **Example 8.3.1: Handling ZeroDivisionError**

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Error: Cannot divide by zero!")
```

Example 8.3.2: Handling ValueError

```
try:
    num = int(input("Enter a number: "))
    print(f"You entered: {num}")
except ValueError:
    print("Error: That was not a valid number.")
```

Skills Checklist

☐ I can use a `try-except` block to handle errors.
☐ I can name common exception types and what causes them.
☐ I can catch specific exception types.

Lesson 8.4: The `else` and `finally` Blocks

`else` Block — Runs only on success

```
try:
    num = int(input("Enter a number: "))
except ValueError:
    print("That's not a number.")
else:
    print(f"You entered {num}.")    # Only runs if no exception
```

`finally` Block — Always runs (cleanup)

```
file_handle = None
try:
    file_handle = open('data.txt', 'w')
    file_handle.write("Some data.")
except TypeError:
    print("A type error occurred.")
finally:
    if file_handle:
        file_handle.close()      # Always closes the file
        print("File closed.")
```

Skills Checklist

☐ I can use an `else` block for code that runs only on success.
☐ I can use a `finally` block for code that always runs.

Lesson 8.5: Storing Data with JSON

Reading References

- Matthes: Chapter 10, pages 200–204

The `json` Module JSON (JavaScript Object Notation) is a lightweight format for storing and exchanging data. Python's `json` module converts between Python objects and JSON.

```
import json

# Save data
user_settings = {}
    "username": "coder_kid",
    "theme": "dark",
    "notifications_on": True,
    "favorite_numbers": [7, 13, 42]

with open('user_settings.json', 'w') as f:
    json.dump(user_settings, f, indent=4)
```

```python
# Load data
try:
    with open('user_settings.json', 'r') as f:
        loaded_settings = json.load(f)
    print(f"Username: {loaded_settings['username']}")
    print(f"Theme: {loaded_settings['theme']}")
except FileNotFoundError:
    print("Settings file not found.")
except json.JSONDecodeError:
    print("Error: Could not decode settings file.")
```

JSON<->Python Mapping

JSON	Python
Object `{}`	Dictionary `{}`
Array `[]`	List `[]`
String	`str`
Number	`int` or `float`
Boolean	`bool`
null	`None`

Skills Checklist

☐ I can explain what JSON is and why it's used.
☐ I can use `json.dump()` to save Python data to a JSON file.
☐ I can use `json.load()` to load JSON data from a file.
☐ I can save and load lists of dictionaries using JSON.

Unit 9: Testing and Data Analysis

Overview This unit introduces testing code for correctness and explores basic text-based data analysis.

Lesson 9.1: Testing a Function

Reading References

- Matthes: Chapter 11, pages 209–217

Unit Testing with `unittest`

```python
# my_functions.py
def add_two_numbers(a, b):
    return a + b
```

```python
# test_my_functions.py
import unittest
from my_functions import add_two_numbers

class TestAddNumbers(unittest.TestCase):
    def test_positive_numbers(self):
        result = add_two_numbers(5, 3)
        self.assertEqual(result, 8)

    def test_negative_numbers(self):
        result = add_two_numbers(-5, -3)
        self.assertEqual(result, -8)

    def test_mixed_numbers(self):
        result = add_two_numbers(10, -4)
        self.assertEqual(result, 6)

if __name__ == '__main__':
    unittest.main()
```

Common Assertion Methods

Method	Description
`assertEqual(a, b)`	Check a == b
`assertTrue(x)`	Check x is True
`assertFalse(x)`	Check x is False
`assertIn(a, b)`	Check a in b
`assertIsNone(x)`	Check x is None

Skills Checklist

☐ I can create a basic test class using `unittest.TestCase`.
☐ I can define test methods within a test class.
☐ I can use `assertEqual()` to check for expected results.
☐ I can run unit tests from the terminal.

Lesson 9.2: Testing a Class

The `setUp()` Method `setUp()` runs **before every test method**, providing a fresh instance each time:

```
class TestEmployee(unittest.TestCase):
    def setUp(self):
        """Create an employee instance for all test methods."""
        self.employee = Employee('janis', 'joplin', 60000)

    def test_give_default_raise(self):
        self.employee.give_raise()
        self.assertEqual(self.employee.salary, 65000)

    def test_give_custom_raise(self):
        self.employee.give_raise(10000)
        self.assertEqual(self.employee.salary, 70000)
```

Skills Checklist

☐ I can write tests for a Python class.
☐ I can use `setUp()` to prepare my tests.

Lesson 9.3: Introduction to Random Numbers

```
import random

# Random integer (inclusive)
die_roll = random.randint(1, 6)
print(f"You rolled: {die_roll}")

# Random float between 0.0 and 1.0
rand_float = random.random()

# Random choice from list
colors = ["red", "blue", "green"]
chosen = random.choice(colors)
print(f"Chosen color: {chosen}")
```

Skills Checklist

☐ I can import the `random` module.
☐ I can generate a random integer within a range.
☐ I can select a random item from a list.

Lesson 9.4: Analyzing Text Data

```
sentence = "the quick brown fox jumps over the lazy dog the quick brown fox"
words = sentence.lower().split()

word_frequencies = {}
for word in words:
    if word in word_frequencies:
```

```
        word_frequencies[word] += 1
    else:
        word_frequencies[word] = 1

print("Word Frequencies:")
for word, count in word_frequencies.items():
    print(f"'{word}': {count}")
```

Skills Checklist

☐ I can read text content from a file.
☐ I can split a string of text into individual words.
☐ I can use a dictionary to count the frequency of words.

Lesson 9.5: Working with CSV Data

```
import csv

filename = 'students.csv'
try:
    with open(filename, 'r') as file_object:
        reader = csv.reader(file_object)
        header_row = next(reader)
        print(f"Columns: {header_row}")
        for row in reader:
            print(row)
except FileNotFoundError:
    print(f"Error: The file {filename} was not found.")
```

Manual CSV Parsing

```
with open('students.csv', 'r') as f:
    header = f.readline().strip()
    for line in f:
        data = line.strip().split(',')
        name = data[0]
        age = data[1]
        grade = data[2]
        print(f"Name: {name}, Age: {age}, Grade: {grade}")
```

Skills Checklist

☐ I can explain what CSV data is.
☐ I can read a CSV file line by line.
☐ I can split a line of CSV data into individual values.

Unit 10: Project-Based Learning — Building Text-Based Applications

Overview This unit guides students through designing and building larger, interactive text-based applications, integrating all previously learned concepts.

Objectives:

- Practice modular programming and code organization.
- Develop a text-based interactive program.
- Apply top-down design principles.

Lesson 10.1: Designing Your Project

Reading References

- Zelle: Chapter 9, pages 225–235
- Matthes: Part II: Projects, pages 225–226

Top-Down Design **Top-down design** (stepwise refinement) breaks a complex problem into smaller, more manageable subproblems.

The Design Process:

1. Analyze the Problem

- Identify **inputs** (information the program needs).
- Identify **outputs** (results it must produce).
- Define the scope of the project.

2. Determine the Major Components Write a high-level algorithm in plain English:

```
show_greeting()
get_user_input()
calculate_cost()
print_ticket()
```

3. Refine Each Component Repeat the design process for each component, adding detail until simple enough to code.

Benefits of Top-Down Design

- **Reduces Complexity** — Focus on one piece at a time.
- **Encourages Reusability** — Functions written for one part may be useful elsewhere.
- **Simplifies Debugging** — Test each component in isolation.
- **Improves Collaboration** — Clear plan for assigning work.

Accessibility Considerations for I/O Design

- Provide **clear, descriptive prompts** for all input.
- Provide **explicit, labeled output** — avoid relying on visual formatting.
- Print **confirmation messages** after every action.
- Use consistent **output separators** like `---` to group information.

Skills Checklist

☐ I can break down a larger problem into smaller, manageable parts.
☐ I can identify the key features required for a project.
☐ I can plan which data structures will be most suitable for my project's data.
☐ I can outline the main steps or flow of my program.

Lesson 10.2: Building a Text-Based Adventure Game (Part 1)

Reading References

- Matthes: Chapter 6, pages 105–111

Room Definition and Movement

```
room_descriptions = {}
    "start_room": {
        "description": "You are in a dimly lit cave. There's a narrow passage to the NORTH.",
        "exits": {}"north": "tunnel"}

    "tunnel": {
        "description": "The tunnel is damp and cold. You can go SOUTH back to the cave or EAST to a clearing.",
        "exits": {}"south": "start_room", "east": "clearing"}

    "clearing": {
        "description": "You've reached a small, sunny clearing. A faint path leads WEST.",
        "exits": {}"west": "tunnel"}
    }

current_room = "start_room"

while True:
    print("---")
    print(room_descriptions[current_room]["description"])
    command = input("What do you want to do? ").lower()

    if command in room_descriptions[current_room]["exits"]:
        current_room = room_descriptions[current_room]["exits"][command]
    elif command == "quit":
        print("Exiting game. Goodbye!")
        break
```

```
    else:
        print("I don't understand that command or you can't go that way.")
```

Skills Checklist

☐ I can design a way to represent different locations in a text-based game.
☐ I can implement player movement based on user commands.
☐ I can update and track the player's current location.
☐ I can parse simple commands from user input.

Lesson 10.3: Building a Text-Based Adventure Game (Part 2)

Adding Items, Inventory, and Puzzles

```
rooms = {
    "start": {
        "description": "A dark cave. There's a SHINY KEY on the ground.",
        "exits": {}"north": "locked_door_room"},
        "items": ["shiny key"]

    "locked_door_room": {
        "description": "A small chamber with a heavy locked door to the NORTH.",
        "exits": {}"south": "start"},
        "items": []
    }

current_room = "start"
player_inventory = []
door_locked = True

while True:
    print("\n" + rooms[current_room]["description"])
    if rooms[current_room]["items"]:
        print(f"You see: {', '.join(rooms[current_room]['items'])}")
    print(f"Your inventory: {', '.join(player_inventory) if player_inventory else 'empty'}")

    command = input("What do you want to do? ").lower()

    if command == "take shiny key" and "shiny key" in rooms[current_room]["items"]:
        player_inventory.append("shiny key")
        rooms[current_room]["items"].remove("shiny key")
        print("You picked up the shiny key.")
    elif command == "use shiny key" and current_room == "locked_door_room" and "shiny key" in player_inventory and door_locked:
        door_locked = False
        print("You unlocked the door!")
        rooms["locked_door_room"]["exits"]["north"] = "treasure_room"
    elif command in rooms[current_room]["exits"]:
        if command == "north" and current_room == "locked_door_room" and door_locked:
            print("The door is locked! You need a key.")
```

```
        else:
            current_room = rooms[current_room]["exits"][command]
    elif command == "quit":
        break
    else:
        print("I don't understand that command.")
```

Skills Checklist

☐ I can add items to locations and manage a player's inventory.
☐ I can implement commands for picking up and dropping items.
☐ I can design and implement a simple puzzle.
☐ I can define win/lose conditions for the game.

Lesson 10.4: Developing a Command-Line Tool

Simple To-Do List Tool

```
import json

TODO_FILE = 'todos.json'
todos = []


def load_todos():
    global todos
    try:
        with open(TODO_FILE, 'r') as f:
            todos = json.load(f)
        print("Tasks loaded.")
    except FileNotFoundError:
        print("No existing tasks found.")
        todos = []


def save_todos():
    with open(TODO_FILE, 'w') as f:
        json.dump(todos, f, indent=4)
    print("Tasks saved.")


def add_task(task_description):
    todos.append({"description": task_description, "completed": False}
    print(f"Task '{task_description}' added.")


def view_tasks():
    if not todos:
        print("No tasks in your list.")
        return
    print("--- Your To-Do List ---")
```

```
    for i, task in enumerate(todos):
        status = "[X]" if task["completed"] else "[ ]"
        print(f"{i + 1}. {status} {task['description']}")
    print("-----------------------")


def mark_completed(task_number):
    try:
        index = int(task_number) - 1
        if 0 <= index < len(todos):
            todos[index]["completed"] = True
            print(f"Task '{todos[index]['description']}' marked as complete.")
        else:
            print("Invalid task number.")
    except ValueError:
        print("Please enter a valid number.")


def main_menu():
    print("\n--- To-Do List Menu ---")
    print("1. Add Task")
    print("2. View Tasks")
    print("3. Mark Task Complete")
    print("4. Save Tasks")
    print("5. Load Tasks")
    print("6. Quit")
    print("-----------------------")


load_todos()

while True:
    main_menu()
    choice = input("Enter your choice (1-6): ")

    if choice == '1':
        desc = input("Enter task description: ")
        add_task(desc)
    elif choice == '2':
        view_tasks()
    elif choice == '3':
        view_tasks()
        task_num = input("Enter task number to mark complete: ")
        mark_completed(task_num)
    elif choice == '4':
        save_todos()
    elif choice == '5':
        load_todos()
    elif choice == '6':
        save_todos()
        print("Goodbye!")
        break
    else:
```

```
        print("Invalid choice. Please try again.")
```

Skills Checklist

☐ I can design an interactive menu system for a tool.
☐ I can use a `while` loop to maintain continuous user interaction.
☐ I can break down the tool's functionality into separate functions.
☐ I can implement file I/O to save and load data.

Lesson 10.5: Refinement and User Experience

Principles for Accessible Text Programs

1. **Clear prompts** — Tell the user exactly what to enter and in what format.
2. **Consistent output** — Use the same format for similar information.
3. **Graceful error handling** — Never let the program crash on bad input.
4. **Helpful error messages** — Tell the user what went wrong and how to fix it.
5. **"Help" commands** — Provide a way to list available commands.
6. **Confirmation messages** — Confirm every action taken.

Robust Input Validation

```
def get_positive_number(prompt):
    """Prompts user until a positive integer is entered."""
    while True:
        try:
            num = int(input(prompt))
            if num > 0:
                return num
            else:
                print("Please enter a positive number.")
        except ValueError:
            print("Invalid input. Please enter a whole number.")


age = get_positive_number("Enter your age: ")
print(f"You entered age: {age}")
```

Skills Checklist

☐ I can write clear, descriptive prompts for user input.
☐ I can handle invalid input gracefully using `try-except`.
☐ I can provide helpful error messages.

Unit 11: Capstone Project — Accessible Personal Data Manager

Overview Students will develop a fully functional, screen-reader-accessible graphical application using NiceGUI that allows users to manage a collection of structured personal data.

Project Goal: Build a simple application to manage a list of structured items.

Project Options:

- Contact Manager: name, phone, email, notes
- Book/Movie Tracker: title, author/director, genre, status
- Simple To-Do List: task description, due date, status
- Simple Inventory: item name, quantity, location, notes

Core Functionality Requirements

1. **Add New Item** — A form to input details for a new item.
2. **View All Items** — Display items in a clear, accessible list format.
3. **Edit Item** — Select an existing item and modify its details.
4. **Delete Item** — Remove an item from the collection.
5. **Save/Load Data** — Persist the data to a JSON file.

Setup

```
pip install nicegui
```

Guided Implementation

Lesson 11.1: Introduction to NiceGUI **Task 1:** Run the `app_skeleton.py` file and navigate using the keyboard and screen reader to understand the existing structure.

Task 2: Define your data model class:

```
class Contact:
    def __init__(self, name, phone, email, notes):
        self.name = name
        self.phone = phone
        self.email = email
        self.notes = notes
```

Task 3: Implement data persistence using the `json` module within the `load_data()` and `save_data()` functions.

Lesson 11.2: Data Model and Storage **Task 1:** Complete the `add_item_logic()` function:

1. Get values from input fields using `.value` attribute.
2. Validate (e.g., name cannot be empty).
3. Provide accessible feedback: `ui.notify("Name is required!", type='negative')`.

Task 2: Create a new item object and add it to `all_items`.
Task 3: Clear input fields and provide success notification.

Lesson 11.3–11.5: Displaying and Managing Items **Implement `update_item_display()`:**

1. Clear the display: `items_display_column.clear()`.
2. Loop through `all_items` and create `ui.card` for each.
3. Update `item_selection_dropdown.options`.

Implement `update_item_logic()`:

1. Find the item in `all_items`.
2. Update attributes from edit form fields.
3. Call `update_item_display()` and `save_data()`.

Implement `delete_selected_item_logic()`:

1. Find and remove the item from `all_items`.
2. Call `update_item_display()`.
3. Provide accessible feedback.

Assessment Criteria

Functionality (60%)

☐ Ability to add new items with all specified attributes.
☐ Correct display of all current items.
☐ Correct editing of existing items.
☐ Correct deletion of items.
☐ Accurate saving and loading to/from `data.json`.
☐ No major bugs or crashes during typical usage.

Code Quality & Structure (20%)

☐ Clear and concise Python code.
☐ Appropriate use of functions and/or classes.
☐ Meaningful variable and function names.
☐ Adequate comments where complex logic exists.
☐ Proper indentation.

Accessibility & User Experience (15%)

☐ All interactive elements are correctly labeled for screen readers.
☐ Clear and consistent user feedback via `ui.notify()`.
☐ Logical flow and presentation of information.
☐ No visual-only cues for critical information.

README and Reflection (5%)

☐ Complete and clear README.
☐ Thoughtful reflection on development and accessibility challenges.

Deliverables

1. `app.py` — Complete NiceGUI application code.
2. `data.json` — Example JSON file demonstrating saved data.
3. `README.md` — Including:
   - Project title and student name.
   - Description of the application.
   - Instructions on how to run: `python app.py`.
   - Instructions for interacting with a screen reader.
   - Reflection on challenges faced.

Supplemental: AI Tools and Version Control

AI-Powered Coding Tools **Benefits:**

- Increased productivity and speed.
- Syntax and idiom discovery.
- Rapid prototyping.

Critical Warning: AI tools can produce code that looks correct but contains subtle bugs. **Always:**

1. Read every line of AI-generated code.
2. Verify the logic and intent.
3. Look for familiar error types (indentation, name errors, type errors).
4. Run and test incrementally.
5. Add your own comments and docstrings.

The Golden Rule: You are responsible for your code. Treat AI suggestions as a first draft that needs meticulous editing.

Git and Version Control **Git** is the most widely used distributed version control system, invented in 2005 by Linus Torvalds for the Linux kernel project.

Core workflow:

1. **Working directory** — Make changes to files.
2. **Staging area** — `git add` — Stage changes.
3. **Repository** — `git commit` — Save a permanent snapshot.
4. **Remote** — `git push` / `git pull` — Sync with GitHub.

Why Git is especially valuable with AI code:

- **Rollback** — If AI-generated code breaks your app, revert instantly.
- **`git blame`** — See exactly which commit introduced a bug.
- **Branches** — Test AI experiments in isolation without affecting main code.
- **Pull requests** — Review AI-generated code before merging.

Basic commands:

- `git init` — Initialize a repository.
- `git add .` — Stage all changes.
- `git commit -m "Add feature"` — Save a snapshot.
- `git log` — View commit history.
- `git diff` — See what changed.
- `git checkout -- file.py` — Discard changes to a file.

Accessibility Tip: Always catch specific exception types: `except ValueError:` not bare `except:`. Print the error in the except block: `except ValueError as e: print("Error:", e)`. A bare `except` silences all errors—including ones you did not expect—making bugs invisible.

Version control is critical for blind coders using AI, because AI frequently introduces bugs that are difficult to detect aurally.

Part II

JavaScript Curriculum

Introduction to Computer Science with JavaScript

For Visually Impaired Students

Course Overview This curriculum introduces computer science concepts using JavaScript, focusing on Node.js for server-side/terminal execution. All exercises are text-based and terminal-focused — no browser required for core concepts.

Target Audience: Students with no prior programming experience using screen readers (NVDA, JAWS, VoiceOver, or Windows Narrator).

Why JavaScript? JavaScript is one of the most widely used programming languages in the world. It powers:

- Web development (front-end and back-end)
- Server applications (Node.js)
- Mobile apps (React Native)
- Desktop apps (Electron)

Learning JavaScript opens doors to a vast ecosystem of tools and opportunities.

Tools Required

- **Node.js 18 LTS+** — https://nodejs.org/
- **Visual Studio Code** with the following extensions:
  - JavaScript (ES6) code snippets
  - ESLint
  - Prettier
- A screen reader (NVDA, JAWS, or VoiceOver)

Accessibility Notes

- All program output is text-based (Node.js terminal/REPL).
- JavaScript uses curly braces `{}` for code blocks, like Java.
- VS Code's Screen Reader Optimized Mode works well with JavaScript.
- Use `Ctrl + Shift + P` -> "Toggle Screen Reader Optimized" in VS Code.
- Use `F8` / `Shift+F8` to navigate between errors.

Units

Unit	Title
Unit 0	Setting Up Your JavaScript Development Environment
Unit 1	Foundations — Getting Started with JavaScript
Unit 2	Numbers, Input, and the Math Object
Unit 3	Control Flow — Decisions and Loops
Unit 4	Data Structures — Arrays and Objects
Unit 5	User Input in Node.js
Unit 6	Functions
Unit 7	Classes and Object-Oriented Programming
Unit 8	Files and Error Handling
Unit 9	Modules, Collections, and Testing
Unit 10	Project-Based Learning
Unit 11	Capstone Project

References

- MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/JavaScript
- Eloquent JavaScript (free online): https://eloquentjavascript.net/
- Node.js Documentation: https://nodejs.org/docs/

Teacher's Guide: Introduction to Computer Science with JavaScript

For Students with Visual Impairments — Complete Instructor Edition

---

Table of Contents

1. Philosophy and Why JavaScript
2. JavaScript-Specific Accessibility Challenges
3. Classroom Setup and Configuration
4. Spoken JavaScript Conventions
5. Unit 0–1: Setup and Foundations
6. Unit 2: Numbers, Input, and Math
7. Unit 3: Control Flow
8. Unit 4: Arrays and Objects
9. Unit 5: Strings
10. Unit 6: Functions
11. Unit 7: Classes and OOP
12. Unit 8: Files, Modules, and Error Handling
13. Unit 9: Collections, Analysis, and Testing
14. Units 10–11: Projects and Capstone
15. Bridging from Python/Java to JavaScript
16. Assessment Strategies
17. Supporting Diverse Needs
18. Alternative and Supplementary Resources
19. Quick Reference Sheets

---

1. Philosophy and Why JavaScript

The Case for Teaching JavaScript to Blind Students JavaScript has a unique profile of benefits and challenges for visually impaired learners:

Universal relevance. JavaScript is the only language that runs natively in every web browser. A student who learns JavaScript can eventually build tools that improve their own accessibility experience — custom browser extensions, accessible widgets, automation scripts. This self-advocacy potential is meaningful. [20]

Node.js enables accessible terminal programs. While JavaScript began in browsers **Node.js enables accessible terminal programs.** While JavaScript began in browsers, Node.js allows running JavaScript programs in the terminal — the same accessible text-based environment used for Python. This curriculum focuses on Node.js to keep all I/O in the terminal where screen readers excel.

[20] Several prominent screen reader users and accessibility advocates are also JavaScript developers who built or extended tools for their own use: custom NVDA add-ons, browser scripts that fix inaccessible pages, and automated testing tools. JavaScript is uniquely positioned as the language of accessibility tooling.

Accessibility Tip: Run all JavaScript with `node filename.js` from the terminal rather than in a browser console. The terminal is fully accessible to any screen reader; browser developer consoles have variable and often poor screen reader support.

The browser is not an accessible first environment for learning JavaScript. Browser developer tools, the console, and the DOM all require navigating complex, partially-accessible UIs. Node.js keeps everything in the terminal where screen readers excel. This curriculum deliberately avoids browser-

Asynchronous programming is a real-world skill. JavaScript's `async/await` model reflects how most modern software actually works — programs that wait for I/O, network requests, and user events. Learning this early prepares students for professional development.

The npm ecosystem. Over 2 million packages on npm provide pre-built solutions for almost any problem. [21] This is a double-edged concern for beginners (dependency complexity), but the ability to say `npm install` and gain powerful functionality is professionally relevant.

[21] npm package quality and accessibility vary enormously. Before recommending any package, verify it works in a Node.js terminal context (no browser DOM required) and produces text output that is screen-reader navigable. Packages that produce progress bars, spinners, or colour-coded terminal output can be disruptive for screen reader users.

JSON is native. `JSON.parse()` and `JSON.stringify()` are built into JavaScript with no import required. For data persistence projects, this reduces friction.

An Honest Account of Challenges **JavaScript's permissiveness is a silent-failure risk.** Unlike Python (which raises exceptions for type mismatches) and Java (which rejects them at compile time), JavaScript silently converts types. `"5" + 3` = `"53"`. `"5" - 3` = `2`. `0 == false` = `true`. Without visual scanning of intermediate results, these silent failures can propagate through a program undetected. This must be addressed through defensive programming habits taught from day one.

Asynchronous programming is conceptually hard. The `async/await` pattern required for user input is a genuine hurdle for beginners. There is no way to eliminate it from a Node.js input-based curriculum. The approach in this guide is to scaffold it carefully and defer full explanation of the event loop to later units.

Multiple valid syntaxes. Function declarations, function expressions, arrow functions, methods — all do essentially the same thing. `var`, `let`, `const` — three variable declarations. `==` and `===` — two equality operators. This variety, which professionals call "JavaScript fatigue," creates unnecessary cognitive overhead for beginners. The approach in this curriculum is to teach one style consistently (`const/let`, arrow functions, `===`) and acknowledge alternatives exist without teaching them in parallel.

The one-style-consistently approach is even more important for VI students than for sighted ones. When a student encounters unfamiliar syntax in tutorial code or Stack Overflow answers, they must navigate it without being able to scan the structure visually. Teach recognition of variants as a separate explicit lesson, not by exposure.

---

2. JavaScript-Specific Accessibility Challenges

Silent Type Coercion This is the greatest accessibility risk in JavaScript for blind students. Unlike Python's `TypeError` and Java's compile-time type check, JavaScript silently converts types in ways that can produce wrong answers with no error:

Expression	Result	Expected by Programmer	Problem
`"5" + 3`	`"53"`	`8`	`+` concatenates when one operand is a string
`"5" - 3`	`2`	`2`	`-` converts string to number
`"5" * "3"`	`15`	`15`	Multiplication converts both
`"" == false`	`true`	Maybe?	Loose equality coercion
`0 == false`	`true`	Maybe?	Same
`null == undefined`	`true`	Probably not	
`NaN === NaN`	`false`	`true`	NaN is not equal to itself

Mitigation: Require `Number()` before any arithmetic on user input. **Mitigation:** Require `Number()` before any arithmetic on user input. Require `===` always. Teach `typeof` and `isNaN()` as defensive checks. Make logging intermediate values an explicit exercise before running any calculation.

Accessibility Tip: Wrap every value from user input in `Number()` before performing arithmetic: `const n = Number(await askQuestion("Enter number: "))`. Without this, `"5" + 3` silently produces `"53"` instead of `8`—a wrong answer with no error.

`undefined` vs. `null` vs. `NaN` vs. `false` vs. `0` vs. `""` All six of these are "falsy" in JavaScript. Aurally, they are indistinguishable unless you call `typeof` or check explicitly. A student who hears `console.log(result)` and the screen reader announces "false" cannot know whether `result` is `false`, `0`, `""`, `null`, `undefined`, or `NaN` without further inspection.

Teaching approach: Establish a diagnostic template:

```
console.log("Value:", result, "Type:", typeof result);
```

This outputs both the value and its type, providing audio context that `console.log(result)` alone does not. [22] [23]

[22] The `"Value:", result, "Type:", typeof result` diagnostic pattern

Accessibility Tip: When debugging a JavaScript value, always log both value and type: `console.log("Value:", x, "Type:", typeof x)`. A screen reader announces both, giving you type information that is invisible when only the value is logged.

[23] The `"Value:", result, "Type:", typeof result` diagnostic pattern should become as automatic for JavaScript students as `type()` is in Python. Both serve the same function: converting invisible type information into audible output.

Callback Hell and Asynchronous Confusion Before `async/await`, Node.js input used nested callbacks:

```
// Confusing nesting --- avoid in teaching
rl.question("Name? ", (name) => {}
  rl.question("Age? ", (age) => {
    rl.question("City? ", (city) => {
      console.log(`$name, $age, $city`);
    )};
  )};
);
```

This nesting produces code that is confusing to read AND to hear — the indentation deepens with each question. The `async/await` pattern linearizes this:

The linearisation that `async/await` provides is not just aesthetically cleaner—it is a direct accessibility improvement. Deeply nested callbacks produce code with increasing indentation levels that a screen reader announces at every line. `async/await` keeps indentation flat.

```
// Clear and linear --- use this exclusively
const name = await askQuestion("Name? ");
const age = await askQuestion("Age? ");
const city = await askQuestion("City? ");
console.log(`${name}, ${age}, ${city}`);
```

Teach `async/await` from the start and avoid callback patterns entirely in this course. Teach `async/await` from the start and avoid callback patterns entirely in this course.

Accessibility Tip: Always use `async/await` for any operation that involves waiting (user input, file reads, network requests). The resulting linear code reads top-to-bottom the same way it executes, making it straightforward to trace with a screen reader without tracking nested callback indentation.

---

3. Classroom Setup and Configuration

VS Code for JavaScript **Required extensions:**

- **ESLint** — catches type issues, undefined variables, and style violations before running code before running code
- **Prettier** — auto-formats code consistently (run on save with `editor.formatOnSave: true`)
- **JavaScript (ES6) code snippets** — tab-complete common patterns

ESLint is a first-class accessibility tool for blind JavaScript developers.

Accessibility Tip: Install ESLint and configure it to run automatically. In VS Code with Screen Reader Optimized Mode, ESLint warnings are announced as you type. This gives you real-time audio feedback about type errors, undefined variables, and style violations—equivalent to visual syntax highlighting for sighted developers.

ESLint is a first-class accessibility tool for blind JavaScript developers. It converts silent runtime failures (wrong type, undefined variable, loose equality) into audible editor warnings. Configure it before writing a single line of JavaScript—not as an optional style tool but as a safety layer.

Recommended ESLint configuration (`.eslintrc.json` in project root):

```
{
  "env": { "node": true, "es2021": true },
  "extends": "eslint:recommended",
  "rules": {
    "eqeqeq": "error",
    "no-unused-vars": "warn",
    "no-undef": "error",
    "prefer-const": "warn",
    "no-console": "off"
  }
}
```

The `eqeqeq: error` rule makes `==` a linting error — it must be `===`. This is an automated enforcement of the rule that prevents the most common silent bugs.

Node.js REPL accessibility:

```
node                    # Start REPL
.help                   # List REPL commands
.exit                   # Quit (or Ctrl+D)
```

The Node.js REPL is excellent for early exploration. Results are printed immediately after each expression, providing audio feedback without the compile-run-read cycle.

Alternative JavaScript Environments

Environment	Platform	Accessibility	Notes
Node.js + VS Code	All	High	Recommended setup
Node.js + terminal only	All	High	For students who prefer minimal UI
Replit.com	Web	Medium	No-install; test with student's setup
CodeSandbox	Web	Medium	Cloud editor; variable accessibility
GitHub Codespaces	Web	High	Cloud VS Code; same accessibility as local
Glitch.com	Web	Medium	Project-based; variable accessibility
Observable	Web	Low	Notebook-style; primarily visual
Deno	Runtime	High	Alternative to Node.js; similar accessibility

---

4. Spoken JavaScript Conventions

Symbol/Pattern	Spoken Form
`const`	"const"
`let`	"let"
`var`	"var" (avoid; legacy)
`===`	"triple equals" or "strict equal"
`!==`	"strict not equal"
`==`	"loose equal" (should hear this rarely)
`=>`	"arrow" or "fat arrow"
`...arr`	"spread arr"
`...args`	"rest args"
`async`	"async"
`await`	"await"
`undefined`	"undefined"
`null`	"null"
`NaN`	"NaN" (say each letter N-A-N)
`typeof`	"typeof"
`instanceof`	"instanceof"
`new`	"new"
`this`	"this"
`class`	"class"
`extends`	"extends"
`super`	"super"
`import`	"import"
`export`	"export"
`require()`	"require"

Symbol/Pattern	Spoken Form
`module.exports`	"module dot exports"
`Promise`	"Promise"
`async function`	"async function"
`() => {}`	"arrow function"
`(x) => x * 2`	"arrow function taking x, returning x times 2"
`#field`	"hash field" or "private field"
`` `template ${var}` ``	"backtick template literal with variable var"

Reading an arrow function:

```
const doubleIfPositive = (n) => (n > 0 ? n * 2 : n);
```

"const doubleIfPositive gets arrow function taking n, returning: if n greater than 0, n times 2, else n"

---

5. Unit 0–1: Setup and Foundations

Lesson 0.1: Installing Node.js

Instructional Cues **Verify installation at every step:**

```
node --version    # Should print v18.x.x or later
npm --version     # Should print a version number
npx --version     # Should print a version number (npx is bundled with npm)
```

All three must work. If `npm` is missing, the student may have installed only the Node.js runtime without npm. Reinstall from nodejs.org (npm is included in the standard installer).

REPL first — files second: Begin with `node` in the terminal. Let the student type expressions and hear results immediately. This builds confidence before introducing the file -> run -> read cycle:

```
$ node
> 5 + 3
8
> "Hello " + "World"
'Hello World'
> typeof 42
'number'
> typeof "hello"
'string'
> .exit
```

The npm init conversation: When initializing a project, `npm init -y` generates `package.json` automatically. Walk the student through reading the generated file:

```
npm init -y
cat package.json   # or type package.json on Windows
```

`package.json` is plain JSON text — fully readable with a screen reader. Understanding its structure is important for later lessons about dependencies and scripts.

Non-Obvious Accessibility Strategies

- **Node.js version managers:** `nvm` (Node Version Manager) allows multiple Node.js versions. For a student who may work on multiple machines or projects, `nvm use 18` is a consistent way to set the Node.js version. `nvm` is command-line-only and fully accessible.
- **Global vs. local packages:** `npm install -g` installs globally; `npm install` installs locally (in `node_modules`). Teach local installation as the default. Global packages can interfere across projects.

Thought Experiments

1. *"JavaScript was created in 10 days in 1995. What marks of that hasty design are still visible in modern JavaScript? (Think about `typeof null === 'object'`, `NaN !== NaN`, `0 == false`.) Why haven't these been fixed?"*
2. *"Node.js uses a single-threaded event loop to handle many concurrent operations. A traditional multi-threaded approach uses multiple threads running in parallel. For a web server handling thousands of requests, what are the tradeoffs of each approach? Which tends to be more memory efficient?"*
3. *"npm has millions of packages. The average Node.js project depends on hundreds of packages, many maintained by single developers with no organizational backing. The 'left-pad incident' in 2016 broke thousands of projects when one 11-line package was unpublished. What does this incident reveal about the risks and benefits of the npm ecosystem?"*
4. *"JavaScript runs in browsers, in Node.js servers, in mobile apps via React Native, in desktop apps via Electron, in IoT devices, and even in serverless cloud functions. What does this ubiquity tell you about the relationship between a language's design and its adoption?"*

Lesson 1.3: Variables and Data Types

Instructional Cues **`const` vs. `let` — establish the rule on day one:**

"Use `const` for everything. If you get an error saying you're trying to reassign a `const`, THEN change it to `let`. This approach produces cleaner code and prevents accidental reassignment bugs."

Demonstrate the error:

```
const x = 5;
x = 10; // TypeError: Assignment to constant variable.
```

"This error is helpful. `const` is protecting you from changing something you said was fixed."

Dynamic typing with auditory ambiguity:

Run this sequence in the REPL and ask the student to predict each output before pressing Enter:

```
typeof 42; // "number"
typeof "42"; // "string"
typeof true; // "boolean"
typeof null; // "object"  <- the famous bug
typeof undefined; // "undefined"
typeof {}; // "object"
typeof []; // "object"  <- arrays are objects!
typeof function () {}; // "function"
```

"Notice that both `{}` (object) and `[]` (array) produce 'object'. How do you tell them apart? `Array.isArray([])` returns `true`. `Array.isArray({})` returns `false`."

Template literals vs. concatenation:

```
const name = "Alice";
const age = 15;

// Concatenation --- confusing order of operations when read aloud
console.log("Hello, " + name + "! You are " + age + " years old.");

// Template literal --- linear, clean, easy to hear
console.log(`Hello, ${name}! You are ${age} years old.`);
```

Read both versions aloud with the student. Ask which is easier to follow aurally. Template literals are almost universally clearer when heard, because the variable values appear where they belong in the sentence.

Thought Experiments

1. *"JavaScript has only one number type for integers and decimals. Python has separate `int` and `float`. Java has `int`, `long`, `double`, `float`. What precision problems arise from having a single number type? When does `0.1 + 0.2 = 0.30000000000000004` matter, and when doesn't it?"*

2. *"JavaScript has six falsy values: `false`, `0`, `""`, `null`, `undefined`, `NaN`. Everything else is truthy — including `"false"` (a non-empty string), `[]` (an*

empty array), and `{}` (an empty object). Design a quiz: ask the student whether ten different expressions are truthy or falsy. Which answers are surprising?"

3. *"`var` has function scope; `let` and `const` have block scope. What does 'scope' mean? How does a variable declared inside an `if` block with `var` vs. `let` behave differently? Which behavior is more predictable and why?"*

---

6. Unit 2: Numbers, Input, and Math

Lesson 2.2: User Input in Node.js

Instructional Cues **The input helper module pattern:**

Provide the `input.js` module in every project skeleton. Students should not need to re-create this boilerplate:

```
// input.js
const readline = require("readline");
const rl = readline.createInterface({}
  input: process.stdin,
  output: process.stdout,
);

function askQuestion(prompt) {}
  return new Promise((resolve) => rl.question(prompt, resolve));

function close() {}
  rl.close();

module.exports = { askQuestion}, close            OperatorTok;
```

Walk through what each line does at least once so students understand the structure, but do not require students to write it from scratch until they're comfortable with async programming.

The `async`/`await` mental model — teach this story:

"When your code hits `await`, imagine it puts up a 'please wait' sign and steps aside. Other things can happen while you wait. When the answer arrives, your code picks up exactly where it paused. The `async` keyword on the function declaration is your promise that 'this function may pause at some point.'"

```
async function main() {}
  console.log("Starting..."); // 1: runs immediately
  const name = await askQuestion("Name? "); // 2: pauses here until user types
  console.log("Got name:", name); // 3: runs after user answers
  const age = await askQuestion("Age? "); // 4: pauses again
  console.log(`${name} is ${age}`); // 5: runs after user answers again
  close();
```

```
main(); // Kick off the async function
```

Ask: "What line is running while the program waits for user input?" (None — the function is paused. The event loop can handle other things.)

Why `main()` must be called at the bottom: `async function main()` defines the function but doesn't run it. The `main()` call at the end actually starts execution. Without the call, nothing happens. This trips up beginners — demonstrate it.

Non-Obvious Accessibility Strategies

- **`rl.close()` is mandatory:** Without `rl.close()`, the Node.js process continues running after `main()` finishes. The terminal appears to hang — cursor blinks but no prompt. For a screen reader user, this is confusing. Teach `close()` as the final step of every program that uses input.
- **Prompt trailing space:** Always include a trailing space in prompt strings: `"Enter your name: "`. Without the space, the cursor appears immediately after the colon with no separation, which some screen readers render as "Enter your name:cursor" — an awkward audio experience.
- **Input validation loop:** Without validation, a blank Enter or a non-numeric string for a number causes silent failure. Provide this reusable pattern:

```
async function getNumber(prompt) {}
  while (true) {}
    const input = await askQuestion(prompt);
    const num = Number(input.trim());
    if (!isNaN(num)) return num;
    console.log("Invalid input: please enter a number.");
  }
```

Teach this pattern and require students to use it whenever they need numeric input.

Thought Experiments

1. *"The `readline` module uses an 'event listener' pattern — `rl.question(prompt, callback)` means 'when the user answers, call this function.' The `async/await` pattern wraps this in a Promise. What problem does the Promise solve that the raw callback doesn't? (Think about reading the code linearly.)"*
2. *"Node.js uses a single-threaded event loop. While your program is waiting for user input (`await askQuestion`), the thread is free. What else could the*

thread be doing? In a real application, this might mean handling a different request or updating state. What does this tell you about why `async`/`await` is important for scalable programs?"

3. *"Synchronous input (blocking until the user responds) and asynchronous input (doing other things while waiting) represent fundamentally different programming models. For a single-user command-line program, which matters? For a server handling 10,000 simultaneous users, which matters more? Why?"*

---

7. Unit 3: Control Flow

Lessons 3.1–3.3

Instructional Cues **`===` enforcement — make it automatic:**

With ESLint configured to flag `==`, the student will hear a warning whenever they use loose equality. Walk through the first warning together:

```
if (userInput == 5) {   // ESLint: Expected '===' and instead saw '=='
```

"ESLint is protecting you from a common silent bug. Change this to `===`. The only exception is checking for null-or-undefined: `x == null` is sometimes used as a shorthand for `x === null || x === undefined` — but avoid this until you understand it." [24]

Truthy/falsy in input validation:

```
const name = await askQuestion("Enter your name: ");
if (!name.trim()) {
  console.log("Name cannot be empty.");
  // re-prompt...
}
```

`!name.trim()` is true when the trimmed string is empty (`""` is falsy). This is idiomatic JavaScript. Read it aloud: "if not name dot trim" — ask: "What does 'not an empty string' mean? Why is an empty string falsy?"

Ternary operator — limit its use:

The ternary `condition ? valueIfTrue : valueIfFalse` is concise but can be hard to follow when read aloud:

```
const status = score >= 60 ? "pass" : "fail";
```

Read: *"const status gets: if score greater-than-or-equal 60, quote pass, else quote fail"*

Teach it but establish this rule: "If the ternary is easy to read aloud in one breath, use it. If it requires re-reading, use if-else."

[24] `===` vs `==` is the JavaScript equivalent of Python's type confusion between `"42"` and `42`. Just as blind students are trained to use `type()` in Python, they should use `===` as the default in JavaScript to avoid coercion surprises that are invisible when reading code without running it.

Thought Experiments

1. *"JavaScript's `==` performs type coercion before comparing. TypeScript (a typed superset of JavaScript) was partly created to address this. What does TypeScript's existence tell you about the real-world cost of JavaScript's permissiveness? What did developers gain from TypeScript that JavaScript didn't provide?"*

2. *"Consider: `if (userName)` vs. `if (userName !== '' && userName !== null && userName !== undefined)`. The first uses JavaScript's truthiness; the second is explicit. In a code review, which would you prefer seeing? What are the readability and correctness tradeoffs?"*

3. *"JavaScript's optional chaining (`?.`) and nullish coalescing (`??`) were added to the language to handle null/undefined more gracefully. `user?.address?.city` returns `undefined` instead of throwing if `user` or `address` is null. Why did this feature take decades to add to a language created in 1995?"*

4. *"The `switch` statement falls through by default in JavaScript. Modern JavaScript has been moving away from `switch` toward mapping objects for dispatch tables. `const actions = { 'add': addFunc, 'delete': deleteFunc }; actions[command]?.();`. What are the accessibility implications of this pattern vs. a `switch` statement when read with a screen reader?"*

---

8. Unit 4: Arrays and Objects

Lesson 4.1: Arrays and Functional Methods

Instructional Cues **Introduce `map`, `filter`, `reduce` as a trio:**

"These three methods each take a function and apply it to every element in an array. `map` transforms elements. `filter` keeps or removes elements. `reduce` combines elements into one value. Together they cover most array processing."

Then give one shared example that uses all three:

`map`, `filter`, and `reduce` originate in functional programming (Lisp, Haskell). JavaScript borrowed them from these traditions. Teaching the trio together as a conceptual unit—transform, select, aggregate—is more durable than teaching individual method signatures.

```
const students = [
  { }name: "Alice", score: 88 },
  { name: "Bob", score: 62 },
  { name: "Charlie", score: 95 },
  { name: "Diana", score: 71 },
];

// 1. Filter passing students (score >= 70)
const passing = students.filter((s) => s.score >= 70);

// 2. Map to get just names
const passingNames = passing.map((s) => s.name);

// 3. Reduce to build a report string
```

```
const report = passingNames.reduce(
  (acc, name) => acc + name + ", ",
  "Passing: ",
);

console.log(report); // "Passing: Alice, Charlie, Diana, "
console.log("Count:", passing.length);
```

Accessible printing for arrays of objects:

```
// Accessible: numbered, labeled fields
students.forEach((student, index) => {
  console.log(`${index + 1}. ${student.name}: ${student.score}`);
);

// Less accessible: raw object output
console.log(students); // Produces cluttered bracket notation when read aloud
```

Chaining — build up incrementally:

Show chaining as a composition of steps, each of which you can trace separately:

```
const result = numbers
  .filter((n) => n % 2 === 0) // Step 1: keep evens
  .map((n) => n ** 2) // Step 2: square each
  .reduce((sum, n) => sum + n, 0); // Step 3: sum

// DEBUG: break the chain to trace each step
const step1 = numbers.filter((n) => n % 2 === 0);
console.log("After filter:", step1);
const step2 = step1.map((n) => n ** 2);
console.log("After map:", step2);
const step3 = step2.reduce((sum, n) => sum + n, 0);
console.log("Final sum:", step3);
```

The debug version (step-by-step) is more accessible during development. Consolidate to the chain after confirming correctness.

Thought Experiments

1. *"`map`, `filter`, and `reduce` come from functional programming — a paradigm based on mathematical functions that have no side effects. JavaScript arrays also have `forEach` which is for side effects only. What is the philosophical distinction between `map` and `forEach`? Why does this distinction matter for reasoning about code correctness?"*

2. *"The `sort` method in JavaScript sorts in-place and mutates the original array. `filter`, `map`, and `reduce` return new arrays, leaving the original unchanged. Why the asymmetry? What bugs could arise from `sort`'s mutation? How would you sort without mutating: `[...arr].sort(...)`?"*

3. _"Functional composition: `const double = n => n _ 2; const addOne = n => n + 1; const doubleAndAdd = n => addOne(double(n))`. How

does function composition scale? What is the advantage of composing small, focused functions over writing one large function that does everything?"*

4. *"The spread operator `[...arr]` creates a shallow copy of an array. For an array of primitives `[1, 2, 3]`, the spread copy is fully independent. For an array of objects `[{name: 'A'}, {name: 'B'}]`, the objects inside are still shared. What does 'shallow copy' mean? When does this distinction cause bugs?"*

Lesson 4.2: Objects

Instructional Cues **Destructuring — introduce before objects are complex:**

```
const person = { }name: "Alice", age: 15, city: "Portland" };

// Without destructuring:
const name = person.name;
const age = person.age;

// With destructuring:
const { name, age } = person;

// In function parameter:
function introduce({ name, age ) {
  console.log(`Hi, I'm ${name}, ${age} years old.`);
}
introduce(person);
```

object access code , which in turn makes [25] programs cleaner to read and hear.

`Object.keys()`, `Object.values()`, `Object.entries()` as navigation tools:

```
const config = { }theme: "dark", fontSize: 14, notifications: true };

// What properties does this object have?
console.log(Object.keys(config).join(", "));

// What are all the values?
Object.values(config).forEach((v) => console.log(v));

// Both together:
Object.entries(config).forEach(([key, value]) => {
  console.log(`${key}: ${value}`);
);
```

Optional chaining for safe nested access:

[25] Destructuring also appears in function parameters (`function f({ name, age })`), in `for...of` loops, and in import statements (`import { useState } from 'react'`). Introducing the concept once and pointing out these variations as they arise is more efficient than re-teaching each form separately.

`Object.keys()`, `.values()`, and `.entries()` are the JS equivalent

Accessibility Tip: When you receive an unfamiliar object, run `console.log(Object.keys(obj).join(", "))` to hear all its property names. This replaces the visual object inspection that sighted developers do by expanding an object in the browser console.

`Object.keys()`, `.values()`, and `.entries()` are the JS equivalent of Python's `dict.keys()`, `.values()`, and `.items()`. Students moving between the two languages should be shown this correspondence explicitly.

```
const user = { profile: { }address: null } };

// Without optional chaining --- crashes if address is null
const city = user.profile.address.city; // TypeError!

// With optional chaining --- returns undefined instead
const city = user?.profile?.address?.city; // undefined, no crash

// With nullish coalescing for a default
const city = user?.profile?.address?.city ?? "Unknown city";
```

This pattern is essential for working with real-world data where fields may be missing.

Thought Experiments

1. *"JavaScript objects can have properties added or removed at any time.* `user.newField = 'value'` *just works. Java objects have a fixed set of fields defined at compile time. Which approach is better for representing data from an API that might add new fields in future versions? Which is better for ensuring all required fields are present?"*
2. *"*`Object.freeze(obj)` *makes an object immutable — you can't add, remove, or modify properties. This mimics the behavior of a Java* `final` *field or Python's* `frozenset`*. When would immutable objects be useful? What patterns become possible when you can guarantee an object won't change?"*
3. *"Prototype-based inheritance means JavaScript objects can inherit from other objects directly, without classes. Classes (introduced in ES6) are 'syntactic sugar' over this prototype system. What does this mean? What can you do with prototypes that you cannot easily express with class syntax?"*
4. *"JSON (JavaScript Object Notation) is the universal data exchange format. It looks almost like JavaScript object literals but has stricter rules: all keys must be double-quoted strings, no functions, no comments. Why must JSON be stricter than JavaScript object literals? What security problems would arise if JSON allowed functions?"*

---

9. Unit 5: Strings

Instructional Cues **Template literals for multiline strings:**

```
const menu = `
--- Main Menu ---
1. Add item
2. View all
3. Search
4. Delete
5. Quit
```

```
  ----------------`;

  console.log(menu);
```

Template literals with newlines are much cleaner for building multiline output than repeated `console.log()` calls. Teach this pattern for menus and reports.

Regular expression basics — connect to real need:

Before introducing regex syntax, establish the problem it solves:

"You have user input that might have extra spaces, uppercase, or punctuation. You want just the words. Without regex, you'd call `.trim().toLowerCase().replace(',', '').replace('.', '')...` for every punctuation mark. Regex lets you express 'any non-letter character' once."

```
const messy = "  Hello, World!   How are you?  ";
const words = messy
  .trim()
  .toLowerCase()
  .split(/\W+/)
  .filter((w) => w.length > 0);
console.log(words); // ['hello', 'world', 'how', 'are', 'you']
```

Teach `\w+` (word characters), `\d+` (digits), `\s+` (whitespace), and the `g` and `i` flags. Don't attempt to teach full regex — a basic vocabulary is enough for data cleaning tasks.

Thought Experiments

1. *"Template literals use backticks `` ` `` while regular strings use `"` or `'`. Backticks on a keyboard are typically in the upper-left corner. For touch typists, backticks are less natural than quotes. What keyboard shortcuts or text expansions might a blind programmer use to reduce the friction of frequently-needed but awkward characters?"*

2. *"String immutability in JavaScript (same as Python) means every `.toUpperCase()`, `.replace()`, and `.slice()` creates a new string. For a loop that builds a long string incrementally, what is the efficient alternative? (Hint: consider `Array.push()` + `.join()` as a form of StringBuilder.)"*

---

10. Unit 6: Functions

Extended Instructional Cues **The four function forms — teach one style, acknowledge the rest:**

```
// 1. Function declaration (hoisted --- can be called before declaration)
function add(a, b) {
  return a + b;
}

// 2. Function expression (not hoisted)
const multiply = function (a, b) {
  return a * b;
};

// 3. Arrow function --- SHORT BODY (implicit return)
const square = (n) => n * n;

// 4. Arrow function --- LONG BODY (explicit return required)
const absolute = (n) => {
  if (n < 0) return -n;
  return n;
};
```

Teach style 3 (short arrow) and style 4 (long arrow) as the primary forms. Acknowledge 1 and 2 exist and how to read them. Do not attempt to teach all four simultaneously.

The implicit return trap — a critical accessibility issue:

```
// These DO return a value:
const double = (n) => n * 2; // [x] implicit return
const greet = (name) => `Hi ${name}`; // [x] implicit return

// This does NOT return a value (the curly brace requires explicit return):
const broken = (n) => {
  n * 2; // No return statement! Returns undefined.
};

// Fixed:
const fixed = (n) => {
  return n * 2;
};
```

When heard aloud, `n => { n * 2; }` and `n => { return n * 2; }` sound different — the `return` keyword is present in one and absent in the other. Teach students to listen for `return` inside block-bodied arrow functions.

Higher-order functions — connect to previous array methods:

```
// map() takes a function as argument
const doubled = [1, 2, 3].map((n) => n * 2);

// We can also pass a named function
const double = (n) => n * 2;
const doubled2 = [1, 2, 3].map(double);
```

```
// This works because map expects a function --- it doesn't care if it's
// named or anonymous, arrow or traditional
```

"Every time you used `array.map(...)`, you were passing a function as an argument. Higher-order functions — functions that take other functions — are normal JavaScript."

Closures — establish the mental model before the code:

"A closure is a function that remembers the environment where it was created, even after that environment is gone. Imagine a letter in an envelope. The letter is the function. The envelope is the environment — it contains variables the function needs. Even if the person who wrote the letter is gone, the letter still has access to everything in its envelope."

```
function makeCounter(startValue = 0) {
  let count = startValue; // This is the "envelope"

  return {
    increment: () => ++count,
    decrement: () => --count,
    reset: () => {
      count = startValue;
    },
    getCount: () => count,
  };
}

const counter = makeCounter(10);
counter.increment();
counter.increment();
counter.increment();
console.log(counter.getCount()); // 13 --- count persists!
```

Thought Experiments

1. *"Arrow functions don't have their own `this` — they inherit `this` from the surrounding scope. Regular functions create their own `this`. This was a major source of bugs in pre-ES6 JavaScript. How many of the following do you think involved confusing `this`? (a) Typos, (b) Variable scope errors, (c) Async timing bugs, (d) `this` binding bugs. In JavaScript specifically, (d) is disproportionately common — why?"*

2. *"Pure functions always produce the same output for the same input and have no side effects. `Math.sin(x)` is pure. `console.log(x)` is not (it has a side effect). Why are pure functions easier to test? Why are pure functions easier to reason about in a screen-reader debugging session?"*

3. *"Memoization caches the results of function calls so repeated calls with the same arguments return the cached result. Write a description of a memoized `fibonacci(n)` function. What is saved? What is the tradeoff? When would memoization be harmful rather than helpful?"*

4. *"JavaScript is a multi-paradigm language: you can write imperative code (explicit step-by-step), object-oriented code (classes and methods), or functional code (pure functions and immutable data). For the accessible data manager project, which paradigm produces the most maintainable code? The most testable? The most readable when heard aloud?"*

---

11. Unit 7: Classes and OOP

Instructional Cues **`toString()` is mandatory:**

```
class BankAccount {
  #balance;

  constructor(owner, initialBalance = 0) {
    this.owner = owner;
    this.#balance = initialBalance;
  }

  toString() {
    return `BankAccount[owner=${this.owner}, balance=$${this.#balance.toFixed(2)}]`;
  }
}

const account = new BankAccount("Alice", 1000);
console.log(account.toString()); // BankAccount[owner=Alice, balance=$1000.00]
console.log(`${account}`); // Also calls toString() implicitly
```

Require `toString()` in every class. Explain: "When your screen reader reads `console.log(myAccount)`, it reads `toString()`. Without it, you hear `[object Object]` — completely useless. Define `toString()` to hear meaningful information."

Private fields `#` — establish as the default:

```
class Dog {
  #name; // Private --- cannot be accessed outside the class
  #age;

  constructor(name, age) {
    this.#name = name;
    this.#age = age;
  }

  get name() {
    return this.#name;
  } // Getter --- controlled access
  get age() {
    return this.#age;
  }

  set age(value) {
```

```
    if (value < 0) throw new Error("Age cannot be negative");
    this.#age = value;
  }

  bark() {
    console.log(`${this.#name} says: Woof!`);
  }

  toString() {
    return `Dog[name=${this.#name}, age=${this.#age}]`;
  }
}
```

"Private fields starting with `#` are truly private — not just conventional privacy. `myDog.#name` outside the class throws a SyntaxError. This is real encapsulation."

Thought Experiments

1. *"JavaScript classes are 'syntactic sugar' over prototype-based inheritance. TypeScript adds static typing on top of JavaScript classes. With three layers (prototypes -> ES6 classes -> TypeScript classes), is JavaScript's OOP model more or less trustworthy than Java's, which has only one layer? What does 'syntactic sugar' imply about the behavior of the underlying mechanism?"*
2. *"The `class` keyword was added to JavaScript in 2015, 20 years after the language was created. In those 20 years, JavaScript developers built massive applications without classes using only prototypes and closures. What does this tell you about the role of syntax vs. capability in programming languages?"*
3. *"Mixins are a way to compose behavior from multiple sources without inheritance. Instead of `class A extends B`, you copy methods from B onto A's prototype. JavaScript supports this; Java does not (without interfaces). When would mixins be more flexible than single inheritance? What problems could they cause?"*

---

12. Unit 8: Files, Modules, and Error Handling

Lesson 8.1: Files

Instructional Cues **`__dirname` and path construction:**

```
const path = require("path");
const fs = require("fs");

// __dirname is the directory of the current file
const dataFile = path.join(__dirname, "data", "contacts.json");
```

```
// Path joining prevents slash vs. backslash issues on Windows/macOS/Linux
console.log("Data file location:", dataFile);
```

For a student who cannot visually navigate a file tree, `console.log(__dirname)` and `console.log(dataFile)` are essential diagnostic tools. Teach them to print file paths as the first debugging step when a file isn't found.

`fs.existsSync()` before file operations:

```
if (fs.existsSync(dataFile)) {
  const data = fs.readFileSync(dataFile, "utf8");
  return JSON.parse(data);
} else {
  console.log("No data file found. Starting with empty data.");
  return [];
}
```

Always check existence before reading. The alternative — catching `ENOENT` in a try-catch — works but puts normal "first run" behavior in the error path.

Lesson 8.3: CommonJS Modules **Module design for accessibility:**

A well-designed module has a clear exports interface. Teach students to think of their module's exports as its "public API" — what other files can use:

```
// contacts.js --- exports only what other files need
const path = require("path");
const fs = require("fs");

const DATA_FILE = path.join(__dirname, "..", "data", "contacts.json");

function loadContacts() {
  /* ... */
}
function saveContacts(contacts) {
  /* ... */
}
function findContact(contacts, name) {
  /* ... */
}

// Export only the public interface
module.exports = { loadContacts, saveContacts, findContact };
```

"Everything not in `module.exports` is private to this file. Other files can use `loadContacts` and `saveContacts`, but they cannot directly access `DATA_FILE`. This is encapsulation at the module level."

Thought Experiments

1. *"Node.js's CommonJS (`require`) and the ES Module standard (`import`) are two different module systems in JavaScript. This creates confusion — some packages use one, some use the other. TypeScript can use both. Why did two module systems emerge? What does this history say about how standards develop in open-source ecosystems?"*

2. *"Synchronous file I/O (`readFileSync`) blocks execution until the file is read. Asynchronous file I/O (`readFile` or `fs.promises.readFile`) allows other code to run while waiting. For a startup script that reads configuration before doing anything else, which makes more sense? For a web server that handles thousands of requests, which matters more?"*

3. *"Custom error classes allow code to throw and catch specific, named errors: `throw new ValidationError('email', 'Invalid format')`. What is the advantage of this over throwing `new Error('Validation failed: email field invalid format')`? Think about code that catches and handles the error."*

---

13. Unit 9: Collections, Analysis, and Testing

Jest Testing

Instructional Cues **Jest output is accessible:**

Run `npx jest --verbose` for maximum accessibility. The `--verbose` flag produces:

```
PASS src/math.test.js
  add function
    [x] adds two positive numbers (3ms)
    [x] adds negative numbers (1ms)
    [x] handles zero (0ms)
  getGrade function
    [x] returns A for score >= 90 (0ms)
    [x] returns F for score < 60 (0ms)

Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
```

Each test name is announced. Failures include file path, test name, expected vs. received values, and a diff. All of this is plain text — fully screen-reader-accessible.

Add to `package.json` scripts:

```
"scripts": {
    "test": "jest --verbose",
    "test:watch": "jest --verbose --watch"
}
```

`npm test` runs all tests. `npm run test:watch` reruns tests whenever a file changes — useful during active development.

Test naming convention:

```
describe("BankAccount", () => {
  describe("deposit", () => {
    test("increases balance by deposited amount", () => {
      /* ... */
    });
    test("rejects negative amounts", () => {
      /* ... */
    });
    test("rejects zero amount", () => {
      /* ... */
    });
  );
  describe("withdraw", () => {
    test("decreases balance by withdrawn amount", () => {}
      /* ... */
    });
    test("throws InsufficientFundsError when balance is too low", () => {
      /* ... */
    });
  );
)};
```

The nested `describe` / `test` structure, when announced by Jest with `--verbose`, reads as complete sentences: "BankAccount > deposit > increases balance by deposited amount."

Thought Experiments

1. *"Jest's `expect(value).toEqual(expected)` performs deep comparison — it checks that two objects have the same structure and values, not that they are the same object in memory. Why is deep comparison usually what you want when testing? When would you specifically want reference equality (`toBe`)?"*

2. *"Snapshot testing in Jest captures the output of a component or function the first time, then checks future runs against that snapshot. What are the advantages? What are the risks? When might a snapshot become 'stale' and mislead you into thinking code is correct when it isn't?"*

3. *"Code coverage measures what percentage of your code is executed by tests. 100% coverage is a goal but not a guarantee of correctness. Design a function with 100% test coverage that still has a bug. (Hint: think about boundary conditions — what if the test covers the case where x=5 but not x=0?)"*

---

14. Units 10–11: Projects and Capstone

JavaScript Project Structure

```
my-data-manager/
|--- src/
|   |--- models/
|   |   +--- contact.js
|   |--- storage/
|   |   +--- dataManager.js
|   |--- ui/
|   |   +--- consoleUI.js
|   +--- utils/
|       |--- input.js          (the readline helper)
|       +--- validation.js     (reusable validation functions)
|--- data/
|   +--- contacts.json
|--- tests/
|   |--- models/
|   |   +--- contact.test.js
|   +--- storage/
|       +--- dataManager.test.js
|--- .eslintrc.json
|--- package.json
+--- index.js
```

`package.json` scripts — teach as accessibility tools:

```
{
  "scripts": {
    "start": "node index.js",
    "test": "jest --verbose",
    "test:watch": "jest --verbose --watch",
    "lint": "eslint src/",
    "lint:fix": "eslint src/ --fix"
  }
}
```

`npm start` — consistent command regardless of entry point filename. `npm test` — runs all tests. `npm run lint` — checks for errors without running the program.

`npm run lint` as a pre-run check: Teach students to run `npm run lint` before running the program. ESLint catches undefined variables, unused variables, and `==` before the program runs — much faster than finding runtime errors.

Accessible Output Design for Capstone Require students to design their program's output to be navigable and self-describing:

```
// Navigation landmark prints
function printSeparator(title) {
  const line = "-".repeat(40);
  console.log(line);
  if (title) console.log(`  ${title.toUpperCase()}`);
  console.log(line);
}

// Numbered list with clear field labels
function printContactList(contacts) {
  if (contacts.length === 0) {}
    console.log("No contacts found.");
    return;
  }
  printSeparator("contacts");
  contacts.forEach((c, i) => {
    console.log(`${i + 1}. ${c.name}`);
    console.log(`   Phone: ${c.phone || "not provided"}`);
    console.log(`   Email: ${c.email || "not provided"}`);
  );
  console.log(`Total: ${contacts.length} contact(s)`);
}

// Confirmation messages --- every action must produce one
function confirmAction(action, details) {
  console.log(`[x] ${action}: ${details}`);
}
```

Thought Experiments

1. *"Your capstone is a command-line program. Eventually, it could become a web application. What would need to change? What would transfer directly? Write a two-column list: 'Can reuse as-is' and 'Must rewrite.'"*

2. *"The accessible design principle 'no visual-only cues for critical information' means every status, error, or confirmation must be communicated through text, not color or icon alone. Where in your program have you relied on visual formatting? How would you replace it?"*

3. *"Open-source your capstone project. Write a README that explains: what the program does, how to install it, how to run it, and specifically how to use it with a screen reader. What information would a sighted user find obvious but a blind user needs explicitly stated?"*

---

15. Bridging from Python/Java to JavaScript

From Python to JavaScript

Python	JavaScript	Key Difference
`print("hello")`	`console.log("hello")`	Method vs. function
`input("prompt")`	`await askQuestion("prompt")`	Async in JavaScript
`f"Hello {name}"`	`` `Hello ${name}` ``	f-string vs. template literal
`def func(x):`	`const func = x => ...`	Arrow function style
`True/False`	`true/false`	Lowercase in JavaScript
`None`	`null` and `undefined`	JS has two "nothing" values
`my_list`	`myList`	JavaScript uses camelCase
`len(x)`	`x.length`	Property, not function
`x is None`	`x === null` or `x === undefined`	Different null checks
`import json`	`const fs = require('fs')`	Built-in `JSON` object, no import
`if __name__ == '__main__':`	No direct equivalent (just call `main()`)	

Thought experiments for Python->JS students:

1. *"In Python, you use spaces to define blocks (indentation). In JavaScript, you use `{}`. Python's blocks are invisible — they're just the alignment of characters. JavaScript's blocks are explicitly marked. Which feels more natural? Which provides clearer information when read aloud?"*

2. *"Python's `None` and JavaScript's `null`/`undefined` both represent 'nothing,' but they behave differently. Python has one 'nothing'; JavaScript has two. When is `null` appropriate? When is `undefined` appropriate? Under what circumstances would you encounter one vs. the other?"*

3. *"Python's type system and JavaScript's both use dynamic typing, but JavaScript's is more permissive — it coerces types rather than raising errors. After using both, which approach do you find more helpful for learning? Which is more appropriate for production software?"*

---

16. Assessment Strategies

Alternative Assessments for JavaScript **Console output analysis:** Provide a JavaScript program and 3–4 sample inputs. Ask the student to predict what the program prints for each input before running it. This tests reading comprehension without requiring writing.

Linting-first review: Submit code that is free of all ESLint warnings. A student who achieves this has internalized JavaScript's best practices at the syntax level.

Refactoring exercises: Give a working but poorly structured program. Ask the student to refactor it to use functions, modules, or classes appropriately. Assess the quality of the architecture, not just whether it runs.

Test-first implementation: Give the student a set of failing tests. Ask them to write the implementation that makes the tests pass. This tests code comprehension and problem-solving without a blank-page start.

---

17. Supporting Diverse Needs

For Students Who Are New to Async Programming Async/await is conceptually hard. For students who struggle:

1. Provide the complete `input.js` helper and instruct them to use it without modifying it.
2. For the first several lessons, treat `async function main() { ... } main();` as boilerplate, not content.
3. Introduce the conceptual explanation of async/await only after students are comfortable with the rest of the language.
4. Consider deferring async until after basic functions are mastered.

For Students with Additional Cognitive Considerations

- ESLint provides immediate text feedback for common errors — use it as a learning tool. Hearing "no-unused-vars" and then explaining what that means is a teaching moment triggered by the student's own code.
- JavaScript's many syntactic variations can be overwhelming. Enforce a single consistent style (arrow functions, `const`/`let`, `===`) and defer alternatives.
- Use the REPL liberally for short experiments so the student gets immediate feedback without file management overhead.

---

18. Alternative and Supplementary Resources

Textbooks and Reading

Resource	Level	Format	Accessibility
Eloquent JavaScript (Haverbeke, free online)	Beginner–Intermediate	Free HTML	Excellent screen-reader-friendly HTML

Resource	Level	Format	Accessibility
You Don't Know JS (Simpson, free on GitHub)	Intermediate	Free HTML/PDF	In-depth on closures, scope, types
JavaScript: The Good Parts (Crockford)	Intermediate	Print	Short; focuses on the reliable subset of JS
JavaScript: The Definitive Guide (Flanagan)	Reference	Print/ebook	Comprehensive; good for looking up specifics
MDN Web Docs (developer.mozilla.org)	Reference	Free HTML	Excellent HTML accessibility; definitive JS reference
Node.js Documentation (nodejs.org/docs)	Reference	Free HTML	Official Node.js API reference
JavaScript.info	Beginner–Advanced	Free HTML	Very well-organized; plain HTML; comprehensive

javascript.info is especially recommended — it's a comprehensive free tutorial organized from basics to advanced topics, all in clean, accessible HTML. It's often more readable than MDN for learning (vs. reference), and screen readers navigate it very well.

Online Courses and Platforms

Platform	Course	Accessibility	Notes
freeCodeCamp	JavaScript Algorithms & Data Structures	Medium	Certification path; some UI inaccessible
Codecademy	Learn JavaScript	Partial	Editor accessible; some UI elements not
Exercism.io	JavaScript Track	High	Terminal-based; mentor feedback
JavaScript30 (Wesbos)	30 vanilla JS projects	Medium	Video-based; transcripts available
edX	Various JavaScript courses	High	Video transcripts; accessible exercises
Scrimba	Learn JavaScript	Low	Interactive video; poor screen reader support
Khan Academy	JavaScript course	Medium	Browser-based editor; variable accessibility
NodeSchool.io	Node.js workshops	High	Command-line workshopper format; fully accessible

NodeSchool.io is especially recommended — workshops are run entirely in the terminal via a command-line tool. Students work through exercises, get feedback from the tool, and progress at their own pace. The terminal-first design makes it highly accessible.

Video and Audio Resources

Resource	Description	Accessibility
Traversy Media (YouTube)	JavaScript crash courses and projects	Captions available
The Net Ninja (YouTube)	In-depth JavaScript series	Captions available
Academind (YouTube)	Deep dives into JS/Node	Captions available
Kevin Powell (YouTube)	JavaScript and web fundamentals	Captions; clear explanations
CS50W (Harvard)	Web programming with Python and JavaScript	Full transcripts; high quality
JavaScript: Understanding the Weird Parts (Udemy)	Deep JS internals	Captions

JavaScript-Specific Accessibility Tools

Tool	Purpose	Notes
ESLint	Code quality and error prevention	Command-line; fully accessible
Prettier	Consistent code formatting	Run on save; no screen interaction needed
Jest	Testing framework	Accessible output; `--verbose` flag
jshint.com	Online JS linter	Plain HTML; basic linting
node-inspect	Node.js debugger	Terminal-based debugging
0x	Node.js profiler	Terminal output
nvm	Node version manager	Command-line; accessible

Communities

Community	Relevance
Node.js Community (nodejs.org/community)	Official Node.js community resources
JavaScript Weekly (newsletter)	Weekly digest of JS news; plain HTML email
Stack Overflow JavaScript	Q&A community; plain HTML; excellent accessibility
Dev.to	Developer blog platform; reasonable accessibility
GitHub Discussions on popular JS projects	Community support; text-based
NFB Jernigan Institute	Resources for blind STEM students
SIGACCESS	Research on accessible computing including programming

19. Quick Reference Sheets

JavaScript vs Python vs Java at a Glance

Concept	Python	Java	JavaScript
Print	`print("x")`	`System.out.println("x")`	`console.log("x")`
Variable	`x = 5`	`int x = 5;`	`const x = 5;`
String format	`f"Hi {name}"`	`String.format("Hi %s", name)`	`` `Hi ${name}` ``
User input	`input("? ")`	`scanner.nextLine()`	`await askQuestion("? ")`
Array/List	`[1, 2, 3]`	`int[] arr = {1,2,3}`	`[1, 2, 3]`
Dictionary/Map	`{"a": 1}`	`new HashMap<>()`	`{ a: 1 }`
Function	`def f(x):`	`public int f(int x) {`	`const f = x => ...`
Class	`class Dog:`	`public class Dog {`	`class Dog {`
Exception	`try/except`	`try/catch`	`try/catch`
Null	`None`	`null`	`null/undefined`
Boolean	`True/False`	`true/false`	`true/false`
Length	`len(x)`	`x.length`	`x.length`
For each	`for x in list:`	`for (T x : list) {`	`for (const x of list) {`
Import	`import math`	`import java.util.*;`	`const x = require('./x')`

Accessible Output Patterns

```
// Standard separator
console.log("-".repeat(40));

// Section header
console.log(`\n=== ${title.toUpperCase()} ===`);

// Numbered list with details
items.forEach((item, i) => {
  console.log(`${i + 1}. ${item.name}`);
  console.log(`   ${item.detail}`);
)};
console.log(`Total: ${items.length}`);

// Menu
const MENU = `
--- Main Menu ---
1. Add item
2. View all items
3. Search
4. Edit item
5. Delete item
6. Save
7. Quit
----------------`;
```

```
// Confirmation
console.log(`[x] Added: "${name}"`);
console.log(`[x] Deleted: "${name}"`);
console.log(`[x] Saved to ${filename}`);

// Error messages
console.log(`Error: Name cannot be empty. Please try again.`);
console.log(`Error: "${filename}" not found. Starting with empty data.`);
console.log(`Error: Task number must be between 1 and ${count}.`);
```

Common JavaScript Error Quick Reference

Error	Common Cause	First Check
`ReferenceError: x is not defined`	Typo or not declared	Spelling; scope
`TypeError: Cannot read property of undefined`	Accessing property on `undefined`	Which variable is undefined; optional chaining
`TypeError: x is not a function`	Calling non-function	`typeof x` before calling
`SyntaxError: Unexpected token`	Syntax mistake	Line number; check brackets, commas
`SyntaxError: Cannot use import statement`	ES module syntax in CommonJS	Use `require()` or add `"type": "module"`
`Error: ENOENT: no such file or directory`	File not found	`console.log(__dirname)` and check path
`Error: EACCES: permission denied`	No write permission	Check file permissions
`RangeError: Maximum call stack size exceeded`	Infinite recursion	Check recursive function's base case
`UnhandledPromiseRejectionWarning`	Missing `await` or `.catch()`	Add `try/catch` around `await` calls
`NaN` (not an error, but wrong value)	Arithmetic on non-number	`console.log(typeof variable)` before math

Input Helper Module (Copy into every project)

```
// utils/input.js
const readline = require("readline");

const rl = readline.createInterface({}
  input: process.stdin,
  output: process.stdout,
});

/**
 * Prompts the user and returns their response as a string.
 * @param {string} prompt - The question to display
 * @returns {Promise<string>} The user's trimmed response
 */
function askQuestion(prompt) {
  return new Promise((resolve) => {
    rl.question(prompt, (answer) => resolve(answer));
```

```
  );
}

/**
 * Prompts until a valid number is entered.
 * @param {string} prompt - The question to display
 * @returns {Promise<number>} A valid number
 */
async function askNumber(prompt) {
  while (true) {}
    const input = await askQuestion(prompt);
    const num = Number(input.trim());
    if (!isNaN(num)) return num;
    console.log("Invalid input: please enter a number.");
  }
}

/**
 * Closes the readline interface. Call at the end of every program.
 */
function close() {
  rl.close();
}

module.exports = { askQuestion, askNumber, close };
```

Unit 0: Setting Up Your JavaScript Development Environment

Lesson 0.1: Installing Node.js Node.js lets you run JavaScript on your computer (outside a browser) using the terminal.

Installing Node.js Download **Node.js 18 LTS** from `https://nodejs.org/`.

Windows

1. Download the `.msi` installer.
2. **CRUCIAL STEP:** Ensure "Add to PATH" is checked during installation.
3. Activate "Install" and "Finish".

macOS

1. Download the `.pkg` installer and follow the prompts.
2. Or use Homebrew: `brew install node`

Linux (Debian/Ubuntu)

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install nodejs
```

Verify Installation

```
node --version      # Should show v18.x.x or later
npm --version       # Should show npm version
```

Skills Checklist

☐ Running `node --version` shows the version number.
☐ Running `npm --version` shows the npm version.

Lesson 0.2: Setting Up VS Code

1. Install VS Code from `https://code.visualstudio.com/`.
2. Press `Ctrl + Shift + X` to open Extensions. Install:
   - **JavaScript (ES6) code snippets** — Handy code shortcuts.
   - **ESLint** — Catches common errors.
3. Enable Screen Reader Mode: `Ctrl + Shift + P` -> "Toggle Screen Reader Optimized".

VS Code Shortcuts for JavaScript

Action	Shortcut
Run file (via terminal)	`Ctrl + \`` then`node filename.js| | Command Palette |Ctrl + Shift + P| | Problems View |Ctrl + Shift + M| | Next Error |F8| | Format Code |Shift + Alt + F| | Go to Definition |F12‘

Lesson 0.3: Your First JavaScript File

Hello World Create `hello.js`:

```
console.log("Hello, World!");
console.log("Welcome to JavaScript!");
```

Run it:

```
node hello.js
```

The Node.js REPL For quick experiments, open an interactive session:

```
node
```

Type JavaScript directly and press Enter to execute. Type `.exit` to quit.

Key Differences from Python

Python	JavaScript
`print()`	`console.log()`
`#` for comments	`//` for single-line, `/* */` for multi-line
No semicolons required	Semicolons optional but recommended
Indentation defines blocks	Curly braces `{}` define blocks
`def` for functions	`function` or `=>` for functions
`True`/`False`	`true`/`false` (lowercase)
`None`	`null` and `undefined`
`len(x)`	`x.length`

Skills Checklist

☐ I can create a `.js` file and run it with `node filename.js`.
☐ I can use the Node.js REPL for quick experiments.

Unit 1: Foundations — Getting Started with JavaScript

Overview This unit covers JavaScript fundamentals: variables, data types, and basic output.

Lesson 1.1: Welcome to Computer Science (See Python Unit 1.1 for conceptual content — hardware, software, algorithms, and problem-solving are language-agnostic.)

JavaScript's Role

JavaScript was designed in ten days. Many of its quirks—`typeof null`, loose equality, hoisting—are direct consequences of that original sprint. Node.js, created by Ryan Dahl, moved JavaScript to the server in 2009 and changed its role from scripting language to general-purpose runtime.

Lesson 1.2: Variables and Output

`console.log()`

`console.log()`

```
console.log("Hello, World!");
console.log(42);
console.log(true);
console.log("Value:", 42, "and", "text");
```

Accessibility Tip: Always include a label in `console.log()` calls: `console.log("Result:", value)` rather than `console.log(value)`. The label is spoken first, giving context before the value is announced. This prevents the common confusion of hearing a number with no idea what it represents.

Declaring Variables

Declaring Variables JavaScript has three ways to declare variables:

Accessibility Tip: Use `const` for every variable by default. If you later try to reassign a `const`, JavaScript throws a `TypeError` immediately, giving you an audible signal that you have made an error. Change to `let` only when you genuinely need to reassign.

Keyword	Scope	Reassignable	Use When
`const`	Block	No	Value won't change (default choice)
`let`	Block	Yes	Value will change
`var`	Function	Yes	Legacy — avoid in modern code

```
const PI = 3.14159; // Cannot be reassigned
let score = 100; // Can change
score = 150; // OK

// var is function-scoped (legacy)
```

```
var oldWay = "avoid this";
```

`var` is still valid JavaScript and appears in most legacy code.

Template Literals

```
const name = "Alice";
const age = 15;

// Template literal (backticks --- preferred)
console.log(`Hello, ${name}! You are ${age} years old.`);

// Concatenation (older style)
console.log("Hello, " + name + "! You are " + age + " years old.");
```

Accessibility Tip: Use `const` for every variable declaration unless you know the value will change. If you try to reassign a `const`, JavaScript throws a `TypeError` immediately—an audible error that helps you catch accidental reassignments that would otherwise be silent bugs with `let` or `var`.

`var` is still valid JavaScript and appears in most legacy code. Students will encounter it in tutorials and Stack Overflow answers. Teach `const`/`let` as the current standard but make sure students can *read* `var` without confusion.

Skills Checklist

- ☐ I can use `console.log()` to display output.
- ☐ I can declare variables using `const` and `let`.
- ☐ I can use template literals with `${}`.

Lesson 1.3: Data Types

Primitive Types

```
// Number (integers and floats use the same type)
let integer = 42;
let decimal = 3.14;
let negative = -10;

// String
let name = "Alice";
let greeting = "Hello";
let multiLine = `Line 1
Line 2`;

// Boolean
let isRaining = false;
let isLoggedIn = true;

// null (intentional absence of value)
let emptyValue = null;

// undefined (value not assigned)
let notAssigned;
console.log(notAssigned); // undefined

// Check type
console.log(typeof 42); // "number"
console.log(typeof "hello"); // "string"
console.log(typeof true); // "boolean"
console.log(typeof null); // "object" (known quirk!)
```

```
console.log(typeof undefined); // "undefined"
```

26

[26] `typeof null === "object"` is a decades-old bug in JavaScript that was never fixed for backward compatibility. The correct test for null is `value === null` (strict equality), not `typeof`. This is one of several JavaScript "gotchas" documented in depth by Douglas Crockford's *JavaScript: The Good Parts*.

Arithmetic Operators

```
console.log(5 + 3); // 8
console.log(10 - 4); // 6
console.log(7 * 6); // 42
console.log(15 / 4); // 3.75  (always decimal, unlike Python 2 or Java)
console.log(10 % 3); // 1  (remainder)
console.log(2 ** 10); // 1024  (exponentiation)

// Math.floor for integer division equivalent
console.log(Math.floor(15 / 4)); // 3
```

Type Conversion

```
// String to Number
let str = "123";
let num = Number(str); // 123
let numParsed = parseInt(str); // 123
let floatParsed = parseFloat("3.14"); // 3.14

// Number to String
let n = 42;
let s = String(n); // "42"
let s2 = n.toString(); // "42"
let s3 = `${n}`; // "42" (template literal)

// Checking
console.log(isNaN("abc")); // true --- "not a number"
console.log(isNaN(42)); // false
console.log(Number("abc")); // NaN
```

Constants

```
const MAX_SIZE = 100;
const APP_NAME = "My App";
const PI = Math.PI; // Use Math.PI --- already defined
```

Examples **Example 1.3.1: Personal Information**

```
const myName = "Jamie";
const favoriteNumber = 7;
const heightInMeters = 1.75;
const likesPizza = true;

console.log(`Name: ${myName}`);
console.log(`Favorite number: ${favoriteNumber}`);
console.log(`Height: ${heightInMeters} meters`);
console.log(`Likes pizza: ${likesPizza}`);
```

Example 1.3.2: Simple Math

```
const num1 = 15;
const num2 = 7;

console.log(`Sum: ${num1 + num2}`);
console.log(`Difference: ${num1 - num2}`);
console.log(`Product: ${num1 * num2}`);
console.log(`Quotient: ${(num1 / num2).toFixed(2)}`);
console.log(`Remainder: ${num1 % num2}`);
```

Activities **Activity 1.3.1: Area Calculator**

```
const length = 12;
const width = 5;
const height = 8;

const rectangleArea = length * width;
const boxVolume = length * width * height;
const circleArea = Math.PI * (length / 2) ** 2;

console.log(`Rectangle area: ${rectangleArea}`);
console.log(`Box volume: ${boxVolume}`);
console.log(`Circle area: ${circleArea.toFixed(2)}`);
```

Skills Checklist

☐ I can identify JavaScript's primitive data types.
☐ I can perform arithmetic operations in JavaScript.
☐ I understand that JavaScript's `/` always gives a decimal result.
☐ I can convert between types using `Number()`, `String()`, `parseInt()`.
☐ I understand `const` vs. `let`.

Unit 2: Numbers, Input, and the Math Object

Lesson 2.1: The Math Object JavaScript's built-in `Math` object provides mathematical functions and constants:

```
// Constants
console.log(Math.PI); // 3.14159265358979...
console.log(Math.E); // 2.71828...

// Functions
console.log(Math.sqrt(25)); // 5
console.log(Math.pow(2, 10)); // 1024
console.log(Math.abs(-7.5)); // 7.5
console.log(Math.ceil(4.2)); // 5
console.log(Math.floor(4.8)); // 4
console.log(Math.round(4.5)); // 5
console.log(Math.round(4.4)); // 4
console.log(Math.max(3, 7, 1, 9, 5)); // 9
console.log(Math.min(3, 7, 1, 9, 5)); // 1

// Random number between 0 (inclusive) and 1 (exclusive)
console.log(Math.random());

// Random integer between min and max (inclusive)
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
console.log(randomInt(1, 6)); // Simulates a dice roll
```

Lesson 2.2: Getting User Input in Node.js Getting user input in Node.js requires the built-in `readline` module.

Using `readline` (Standard Library)

```
const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

rl.question("What is your name? ", (name) => {
  console.log(`Hello, ${name}!`);
  rl.close();
);
```

Promisified Input (Modern Approach)

```
const readline = require("readline");

const rl = readline.createInterface({}
  input: process.stdin,
  output: process.stdout,
});

function askQuestion(prompt) {
  return new Promise((resolve) => {
    rl.question(prompt, (answer) => {
      resolve(answer);
    });
  );
}

async function main() {
  const name = await askQuestion("What is your name? ");
  const ageStr = await askQuestion("How old are you? ");
  const age = parseInt(ageStr);

  console.log(`Hello, ${name}!`);
  console.log(`Next year you will be ${age + 1} years old.`);

  rl.close();
}

main();
```

Helper Module for Input (Reusable) Create `input.js` once and reuse across projects:

```
// input.js --- save this file in your project
const readline = require("readline");

const rl = readline.createInterface({}
  input: process.stdin,
  output: process.stdout,
});

function askQuestion(prompt) {
  return new Promise((resolve) => {
    rl.question(prompt, resolve);
  });
}

function close() {
  rl.close();
}

module.exports = { askQuestion, close };
```

Usage in your main file:

```
// main.js
const { askQuestion, close } = require("./input");

async function main() {
  const name = await askQuestion("Enter your name: ");
  const numStr = await askQuestion("Enter a number: ");
  const num = parseFloat(numStr);

  console.log(`Hello, ${name}!`);
  console.log(`Your number squared is: ${num ** 2}`);

  close();
}

main();
```

Examples **Example 2.2.1: Temperature Converter**

```
const { askQuestion, close } = require("./input");

async function main() {
  const fahrenheitStr = await askQuestion("Enter temperature in Fahrenheit: ");
  const fahrenheit = parseFloat(fahrenheitStr);

  const celsius = ((fahrenheit - 32) * 5) / 9;

  console.log(`${fahrenheit}°F is ${celsius.toFixed(1)}°C`);
  close();
}

main();
```

Example 2.2.2: Circle Calculator

```
const { askQuestion, close } = require("./input");

async function main() {
  const radiusStr = await askQuestion("Enter the radius: ");
  const radius = parseFloat(radiusStr);

  const area = Math.PI * radius ** 2;
  const circumference = 2 * Math.PI * radius;

  console.log(`Area: ${area.toFixed(2)}`);
  console.log(`Circumference: ${circumference.toFixed(2)}`);
  close();
}

main();
```

Skills Checklist

☐ I can use `Math` object methods and constants.

☐ I can get user input using the `readline` module.
☐ I understand `async`/`await` for asynchronous input.
☐ I can convert string input to numbers using `parseInt()` and `parseFloat()`.

Unit 3: Control Flow — Decisions and Loops

Lesson 3.1: Boolean Expressions and `if` Statements

Comparison Operators

Operator	Meaning	Note
`===`	Strict equal	Checks value AND type (preferred)
`!==`	Strict not equal	Checks value AND type (preferred)
`==`	Loose equal	Type coercion (avoid)
`!=`	Loose not equal	Type coercion (avoid)
`<`	Less than	
`>`	Greater than	
`<=`	Less than or equal	
`>=`	Greater than or equal	

Always use `===` and `!==` — `==` does unexpected type conversions.

Logical Operators

Operator	Meaning	Example
`&&`	AND	`age >= 18 && hasId`
`||`	OR	`day === 'Sat' || day === 'Sun'`
`!`	NOT	`!isRaining`

`if-else if-else`

```
const score = 85;

if (score >= 90) {
  console.log("Grade: A");
} else if (score >= 80) {
  console.log("Grade: B");
} else if (score >= 70) {
  console.log("Grade: C");
} else {
  console.log("Grade: F");
```

```
}
```

Ternary Operator

```
const age = 20;
const status = age >= 18 ? "adult" : "minor";
console.log(status); // "adult"
```

`switch` Statement

```
const day = "Wednesday";
switch (day) {
  case "Monday":
  case "Tuesday":
  case "Wednesday":
  case "Thursday":
  case "Friday":
    console.log("Weekday");
    break;
  case "Saturday":
  case "Sunday":
    console.log("Weekend");
    break;
  default:
    console.log("Unknown day");
}
```

Truthy and Falsy Values In JavaScript, the following values are **falsy** (evaluate to `false` in conditions):

- `false`, `0`, `""` (empty string), `null`, `undefined`, `NaN`

  Everything else is **truthy**.

```
const name = "";
if (name) {
  console.log("Name is set");
} else {
  console.log("Name is empty"); // This runs
}
```

Examples **Example 3.1.1: Grade Calculator**

```
const { askQuestion, close } = require("./input");

async function main() {
  const scoreStr = await askQuestion("Enter your score (0-100): ");
  const score = parseInt(scoreStr);

  let grade;
  if (score >= 90) grade = "A";
  else if (score >= 80) grade = "B";
  else if (score >= 70) grade = "C";
  else if (score >= 60) grade = "D";
```

```
  else grade = "F";

  console.log(`Your grade: ${grade}`);
  close();
}

main();
```

Skills Checklist

☐ I use `===` and `!==` for comparisons in JavaScript.
☐ I can write `if`, `else if`, and `else` statements.
☐ I understand truthy and falsy values.
☐ I can write a `switch` statement.

Lesson 3.2: `for` Loops

```
// Basic for loop
for (let i = 0; i < 5; i++) {
  console.log(`i = ${i}`);
}

// Counting down
for (let i = 10; i > 0; i--) {
  console.log(i);
}
console.log("Blast off!");

// for...of --- iterate over array elements (like Python's for-in)
const fruits = ["apple", "banana", "cherry"];
for (const fruit of fruits) {
  console.log(`I like ${fruit}.`);
}

// for...in --- iterate over object keys
const person = { name: "Alice", age: 15, city: "Portland" };
for (const key in person) {
  console.log(`${key}: ${person[key]}`);
}
```

`Array.forEach()`

```
const numbers = [1, 2, 3, 4, 5];

numbers.forEach((num) => {
  console.log(num * 2);
});

// With index
```

```
numbers.forEach((num, index) => {
  console.log(`Index ${index}: ${num}`);
);
```

Examples **Example 3.2.1: Multiplication Table**

```
const n = 5;
console.log(`Multiplication table for ${n}:`);
for (let i = 1; i <= 10; i++) {
  console.log(`${n} x ${i} = ${n * i}`);
}
```

Skills Checklist

☐ I can write a `for` loop with initialization, condition, and update.
☐ I can use `for...of` to iterate over arrays.
☐ I can use `for...in` to iterate over object keys.
☐ I can use `Array.forEach()`.

Lesson 3.3: `while` Loops

```
let count = 1;
while (count <= 5) {
  console.log(`Count: ${count}`);
  count++;
}

// do-while --- runs at least once
let num;
do {
  num = Math.floor(Math.random() * 10);
  console.log(`Rolled: ${num}`);
 }while (num < 5);
```

`break` and `continue`

```
// break --- exit entirely
for (let i = 0; i < 10; i++) {
  if (i === 5) break;
  console.log(i); // Prints 0-4
}

// continue --- skip iteration
for (let i = 0; i < 10; i++) {}
  if (i % 2 === 0) continue;
  console.log(i); // Prints 1, 3, 5, 7, 9
}
```

Skills Checklist

☐ I can write `while` and `do-while` loops.

☐ I can use `break` and `continue` appropriately.

Unit 4: Data Structures — Arrays and Objects

Lesson 4.1: Arrays

Lesson 4.1: Arrays

Accessibility Tip: To inspect a JavaScript array accessibly, use: `arr.forEach((item, i) => console.log(`${i+1}. ${item}`))`. This prints each item numbered on its own line. Never use `console.log(arr)` alone—the bracket notation it produces is hard to navigate when spoken by a screen reader.

Creating and Accessing Arrays

```
const colors = ["red", "blue", "green", "yellow"];

// Access
console.log(colors[0]); // "red"
console.log(colors[colors.length - 1]); // "yellow"

// Modify
colors[1] = "purple";

// Check
console.log(colors.length); // 4
console.log(colors.includes("red")); // true
console.log(colors.indexOf("green")); // 2
```

Array Methods

```
const fruits = ["apple", "banana"];

// Adding
fruits.push("cherry"); // Add to end -> ["apple", "banana", "cherry"]
fruits.unshift("avocado"); // Add to start -> ["avocado", "apple", "banana", "cherry"]

// Removing
const last = fruits.pop(); // Remove and return last
const first = fruits.shift(); // Remove and return first

// Slicing (non-destructive)
const sub = fruits.slice(1, 3); // Returns new array, original unchanged

// Splicing (destructive --- modifies original)
fruits.splice(1, 1); // Remove 1 element at index 1
fruits.splice(1, 0, "mango"); // Insert "mango" at index 1

// Joining
const str = fruits.join(", "); // "apple, mango, cherry"

// Sorting
const nums = [3, 1, 4, 1, 5, 9, 2, 6];
nums.sort((a, b) => a - b); // Ascending: [1, 1, 2, 3, 4, 5, 6, 9]
```

```
nums.sort((a, b) => b - a); // Descending

// Reversing
const reversed = [...fruits].reverse(); // Use spread to avoid mutating original

// Concatenating
const combined = fruits.concat(nums);
```

Functional Array Methods These are powerful for data transformation:

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

// map --- transform each element, returns new array
const doubled = numbers.map((n) => n * 2);
console.log(doubled); // [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

// filter --- keep elements matching condition
const evens = numbers.filter((n) => n % 2 === 0);
console.log(evens); // [2, 4, 6, 8, 10]

// reduce --- combine elements into single value
const sum = numbers.reduce((accumulator, current) => accumulator + current, 0);
console.log(sum); // 55

// find --- return first matching element
const firstOver5 = numbers.find((n) => n > 5);
console.log(firstOver5); // 6

// some --- true if any element matches
const hasEven = numbers.some((n) => n % 2 === 0);
console.log(hasEven); // true

// every --- true if all elements match
const allPositive = numbers.every((n) => n > 0);
console.log(allPositive); // true

// Chaining
const result = numbers
  .filter((n) => n % 2 === 0)
  .map((n) => n ** 2)
  .reduce((sum, n) => sum + n, 0);
console.log(result); // 220  (4+16+36+64+100)
```

Examples **Example 4.1.1: Array Analysis**

```
const scores = [85, 92, 78, 95, 88, 72, 90, 83];

const average = scores.reduce((sum, s) => sum + s, 0) / scores.length;
const max = Math.max(...scores);
const min = Math.min(...scores);
const passing = scores.filter((s) => s >= 80);

console.log(`Scores: ${scores.join(", ")}`);
```

```
console.log(`Average: ${average.toFixed(1)}`);
console.log(`Highest: ${max}`);
console.log(`Lowest: ${min}`);
console.log(`Passing (>=80): ${passing.length} students`);
```

Skills Checklist

☐ I can create, access, and modify arrays.
☐ I can use `push`, `pop`, `shift`, `unshift`, `splice`, and `slice`.
☐ I can use `map`, `filter`, and `reduce`.
☐ I can sort arrays using a comparator function.

Lesson 4.2: Objects

Creating and Using Objects Objects in JavaScript are like Python dictionaries and Java Maps combined with class instances.

```
// Object literal
const studentProfile = {
  name: "Emily",
  age: 13,
  grade: "7th",
  favoriteSubject: "Science",
};

// Access properties
console.log(studentProfile.name); // "Emily" (dot notation)
console.log(studentProfile["age"]); // 13 (bracket notation)

// Add/modify
studentProfile.city = "Springfield";
studentProfile.age = 14;

// Delete
delete studentProfile.grade;

// Check if property exists
console.log("name" in studentProfile); // true
console.log(studentProfile.hasOwnProperty("city")); // true

// Get keys, values, entries
console.log(Object.keys(studentProfile));
console.log(Object.values(studentProfile));
console.log(Object.entries(studentProfile));
```

Iterating Over Objects

```
const inventory = { apples: 50, bananas: 30, oranges: 40 };

// for...in
for (const key in inventory) {
  console.log(`${key}: ${inventory[key]}`);
}

// Object.entries() with destructuring
for (const [key, value] of Object.entries(inventory)) {
  console.log(`${key}: ${value} in stock`);
}
```

Nested Objects and Arrays

```
const library = {
  name: "City Library",
  location: "Downtown",
  books: [
    { title: "Dune", author: "Frank Herbert", available: true },
    { title: "1984", author: "George Orwell", available: false },
  ],
};

// Access nested data
console.log(library.books[0].title); // "Dune"
console.log(library.books[1].author); // "George Orwell"

// Filter available books
const available = library.books.filter((book) => book.available);
console.log(`Available: ${available.map((b) => b.title).join(", ")}`);
```

Destructuring

```
// Object destructuring
const { name, age } = studentProfile;
console.log(name, age); // "Emily" 13

// With renaming
const { name: studentName, favoriteSubject: subject } = studentProfile;

// Array destructuring
const [first, second, ...rest] = [1, 2, 3, 4, 5];
console.log(first); // 1
console.log(rest); // [3, 4, 5]

// In function parameters
function printStudent({ name, age, grade } ) {
  console.log(`${name} | Age: ${age} | Grade: ${grade}`);
}
printStudent(studentProfile);
```

Spread and Rest Operators

```
// Spread --- expand array or object
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];
const combined = [...arr1, ...arr2]; // [1, 2, 3, 4, 5, 6]

const obj1 = { }a: 1, b: 2 };
const obj2 = { c: 3, d: 4 };
const merged = { ...obj1, ...obj2 }; // { a: 1, b: 2, c: 3, d: 4 }

// Copy without mutating
const originalArr = [1, 2, 3];
const copyArr = [...originalArr];

const originalObj = { name: "Alice" };
const copyObj = { ...originalObj };
```

Skills Checklist

- ☐ I can create, access, modify, and delete object properties.
- ☐ I can iterate over objects using `for...in` and `Object.entries()`.
- ☐ I can work with nested objects and arrays.
- ☐ I can use destructuring assignment.
- ☐ I can use spread and rest operators.

Unit 5: Strings and String Methods

Lesson 5.1: Working with Strings

```
const name = "Alice Smith";

// Properties
console.log(name.length); // 11

// Accessing characters
console.log(name[0]); // "A"
console.log(name.charAt(0)); // "A"
console.log(name[name.length - 1]); // "h"

// Substrings
console.log(name.slice(6)); // "Smith"
console.log(name.slice(0, 5)); // "Alice"
console.log(name.substring(6, 11)); // "Smith"

// Case
console.log(name.toUpperCase()); // "ALICE SMITH"
console.log(name.toLowerCase()); // "alice smith"

// Trimming
const padded = "  hello  ";
console.log(padded.trim()); // "hello"
console.log(padded.trimStart()); // "hello  "
console.log(padded.trimEnd()); // "  hello"

// Searching
console.log(name.indexOf("Smith")); // 6
console.log(name.lastIndexOf("l")); // 3
console.log(name.includes("Alice")); // true
console.log(name.startsWith("Ali")); // true
console.log(name.endsWith("ith")); // true

// Replacing
const result = name.replace("Alice", "Bob"); // "Bob Smith"
const allReplaced = "a,b,a,c,a".replaceAll("a", "x"); // "x,b,x,c,x"

// Splitting
const sentence = "the quick brown fox";
const words = sentence.split(" ");
console.log(words); // ["the", "quick", "brown", "fox"]

const csv = "Alice,15,Portland";
const parts = csv.split(",");
console.log(parts); // ["Alice", "15", "Portland"]

// Repeating and Padding
console.log("ha".repeat(3)); // "hahaha"
console.log("5".padStart(3, "0")); // "005"
```

```
console.log("5".padEnd(3, "0")); // "500"

// Template literals (multi-line)
const multiLine = `
Name: ${name}
Length: ${name.length}
Uppercase: ${name.toUpperCase()}
`;
console.log(multiLine);
```

Regular Expressions (Basics)

```
const text = "Hello World! Hello JavaScript!";

// Test if pattern exists
console.log(/hello/i.test(text)); // true (i = case-insensitive)

// Find matches
const matches = text.match(/Hello/g); // g = global (find all)
console.log(matches); // ["Hello", "Hello"]

// Replace with regex
const cleaned = text.replace(/[!]/g, "."); // "Hello World. Hello JavaScript."

// Split with regex
const words2 = "one  two   three".split(/\s+/);
console.log(words2); // ["one", "two", "three"]
```

Skills Checklist

☐ I can use common `String` methods in JavaScript.
☐ I can use template literals for multi-line strings and interpolation.
☐ I can split strings into arrays and join arrays into strings.
☐ I understand basic regular expressions.

Unit 6: Functions

Lesson 6.1: Defining Functions JavaScript has multiple ways to define functions:

Function Declaration

Function Declaration

Function Declaration

Accessibility Tip: Write a comment above every function describing what it does, its parameters, and its return value. When IntelliSense shows the function in a completion list, it displays this comment as documentation that is read aloud by the screen reader.
Accessibility Tip: Write a JSDoc comment above every function: `/** @param {number} x - The input. @returns {number} The result. */`. IntelliSense reads this aloud when you type the function name, giving you parameter and return type information without having to navigate to the definition.

```
function greetUser() {
  console.log("Hello, JavaScript programmer!");
}

greetUser();
greetUser();
```

Function Expression

```
const greetByName = function (name) {
  console.log(`Hello, ${name}!`);
};

greetByName("Alice");
```

Arrow Functions (ES6) — Modern Preferred Style

```
// Full arrow function
const add = (a, b) => {
  return a + b;
};

// Concise body (implicit return for single expressions)
const addShort = (a, b) => a + b;

// Single parameter (no parentheses needed)
const double = (n) => n * 2;

// No parameters
const sayHello = () => console.log("Hello!");

console.log(add(5, 3)); // 8
console.log(addShort(5, 3)); // 8
console.log(double(7)); // 14
sayHello();
```

Default Parameters

```
function makeShirt(size, message = "I love JavaScript") {
  console.log(`Making a ${size}-sized shirt: '${message}'`);
}

makeShirt("large"); // Uses default
makeShirt("medium", "Hello World"); // Custom message
```

Rest Parameters

```
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
```

```
console.log(sum(1, 2, 3)); // 6
console.log(sum(10, 20, 30, 40)); // 100
```

Examples **Example 6.1.1: Calculator Functions**

```
const add = (a, b) => a + b;
const subtract = (a, b) => a - b;
const multiply = (a, b) => a * b;
const divide = (a, b) => {
  if (b === 0) throw new Error("Cannot divide by zero");
  return a / b;
};

const circleArea = (radius) => Math.PI * radius ** 2;
const getGrade = (score) => {
  if (score >= 90) return "A";
  if (score >= 80) return "B";
  if (score >= 70) return "C";
  if (score >= 60) return "D";
  return "F";
};

console.log(add(5, 3));
console.log(circleArea(5).toFixed(2));
console.log(getGrade(85));
```

Example 6.1.2: Higher-Order Functions

```
// Functions that take or return other functions

// Map using our own function
function doubleAll(numbers) {
  return numbers.map((n) => n * 2);
}

// Function that returns a function (closure)
function makeAdder(x) {
  return (y) => x + y;
}

const addFive = makeAdder(5);
const addTen = makeAdder(10);

console.log(addFive(3)); // 8
console.log(addTen(3)); // 13
console.log(doubleAll([1, 2, 3, 4, 5]));
```

Closures

```
function makeCounter() {
  let count = 0; // Private variable

  return {
    increment: () => ++count,
```

```
    decrement: () => --count,
    getCount: () => count,
  };
}

const counter = makeCounter();
counter.increment();
counter.increment();
counter.increment();
counter.decrement();
console.log(counter.getCount()); // 2
```

Skills Checklist

- ☐ I can define functions using declarations, expressions, and arrow syntax.
- ☐ I can use default and rest parameters.
- ☐ I understand closures and how they capture variables.
- ☐ I can pass functions as arguments and return them.

Unit 7: Classes and Object-Oriented Programming

Lesson 7.1: Classes in JavaScript JavaScript uses prototype-based inheritance, but ES6 `class` syntax provides a familiar OOP style.

Class Syntax

```
class Dog {
  // Instance variables declared in constructor
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  // Methods
  sit() {
    console.log(`${this.name} is now sitting.`);
  }

  rollOver() {
    console.log(`${this.name} rolled over!`);
  }

  getOlder(years) {
    this.age += years;
    console.log(`${this.name} is now ${this.age} years old.`);
  }
```

Accessibility Tip: Override `toString()` on every class you create. When `console.log(myObject)` is called, JavaScript uses `toString()` for the output. Without it you hear `[object Object]`. Return a descriptive string: `return \`Dog[name=${this.name}, age=${this.age}]\`;`

```
  // Getter
  get info() {
    return `${this.name} (${this.age} years old)`;
  }

  // toString equivalent
  toString() {
    return `Dog { name: '${this.name}', age: ${this.age} }`;
  }
}

// Create instances
const myDog = new Dog("Buddy", 3);
const yourDog = new Dog("Lucy", 1);

myDog.sit();
myDog.getOlder(2);
console.log(myDog.info); // Uses getter
console.log(myDog.toString());
```

Private Fields (ES2022)

```
class BankAccount {
  #balance; // Private field --- only accessible within the class

  constructor(initialBalance) {
    this.#balance = initialBalance;
  }

  deposit(amount) {
    if (amount > 0) {
      this.#balance += amount;
      console.log(`Deposited $${amount}. New balance: $${this.#balance}`);
    }
  }

  withdraw(amount) {
    if (amount > this.#balance) {
      console.log("Insufficient funds!");
    } else {
      this.#balance -= amount;
      console.log(`Withdrew $${amount}. New balance: $${this.#balance}`);
    }
  }

  get balance() {
    return this.#balance;
  }
}

const account = new BankAccount(1000);
account.deposit(500);
account.withdraw(200);
```

```
console.log(`Balance: $${account.balance}`);
```

Static Methods

```
class MathHelper {
  static add(a, b) {
    return a + b;
  }
  static multiply(a, b) {
    return a * b;
  }
  static PI = 3.14159265358979;
}

// Call without creating an instance
console.log(MathHelper.add(5, 3));
console.log(MathHelper.PI);
```

Lesson 7.2: Inheritance

```
class Vehicle {
  constructor(make, model, year) {
    this.make = make;
    this.model = model;
    this.year = year;
    this.odometer = 0;
  }

  getDescriptiveName() {
    return `${this.year} ${this.make} ${this.model}`;
  }

  readOdometer() {
    console.log(`This vehicle has ${this.odometer} miles on it.`);
  }
}

class Car extends Vehicle {
  constructor(make, model, year) {
    super(make, model, year); // Call parent constructor
    this.numDoors = 4;
  }

  readOdometer() {
    // Override parent method
    console.log(`This car has ${this.odometer} miles on it.`);
  }
}

class ElectricCar extends Car {
  #batterySize;
```

```
  constructor(make, model, year) {
    super(make, model, year);
    this.#batterySize = 75;
  }

  describeBattery() {
    console.log(`This car has a ${this.#batterySize}-kWh battery.`);
  }
}

const myEV = new ElectricCar("Ford", "Mustang Mach-E", 2023);
console.log(myEV.getDescriptiveName()); // Inherited from Vehicle
myEV.readOdometer(); // Overridden in Car
myEV.describeBattery(); // Defined in ElectricCar
```

Skills Checklist

- ☐ I can define a JavaScript class with a constructor and methods.
- ☐ I can use `extends` to create a subclass.
- ☐ I can use `super()` to call the parent constructor.
- ☐ I can use `#` for private fields.
- ☐ I can define static methods and properties.

Unit 8: Files, Modules, and Error Handling

Lesson 8.1: Reading and Writing Files

Accessibility Tip: Wrap all file and async operations in `try/catch` and always log the error: `catch (e) { console.log("Error:", e.message); }`. Without this, failed operations are silently swallowed—the program continues with missing data and no audio signal that anything went wrong.

```
const fs = require("fs");
const path = require("path");

// Synchronous reading (simple but blocks execution)
try {
  const content = fs.readFileSync("my_notes.txt", "utf8");
  console.log(content);
} catch (err) {
  console.log(`Error: ${err.message}`);
}

// Reading line by line
const lines = content.split("\n");
lines.forEach((line, index) => {
  console.log(`Line ${index + 1}: ${line}`);
);

// Writing (overwrites existing content)
try {
  fs.writeFileSync("output.txt", "Hello, Node.js file I/O!\n");
  console.log("File written successfully.");
 }catch (err) {
  console.log(`Error writing file: ${err.message}`);
```

```
}

// Appending
try {
  fs.appendFileSync("output.txt", "This line is appended.\n");
  console.log("Line appended.");
 }catch (err) {
  console.log(`Error appending: ${err.message}`);
}

// Check if file exists
if (fs.existsSync("my_notes.txt")) {}
  console.log("File exists.");
}
```

Asynchronous File I/O (Modern Approach)

```
const fs = require("fs").promises; // Promise-based fs

async function readFile() {
  try {
    const content = await fs.readFile("my_notes.txt", "utf8");
    console.log(content);
  } catch (err) {
    console.log(`Error reading file: ${err.message}`);
  }
}

async function writeFile() {
  try {
    await fs.writeFile("output.txt", "Async write!\n");
    await fs.appendFile("output.txt", "Appended!\n");
    console.log("Files written.");
   catch (err) {
    console.log(`Error: ${err.message}`);
  }
}

readFile();
writeFile();
```

Lesson 8.2: Error Handling

```
// try-catch-finally
try {
  const result = 10 / 0; // Doesn't throw in JS (gives Infinity)
  const parsed = JSON.parse("invalid json"); // This throws
 }catch (err) {
  console.log(`Error type: ${err.constructor.name}`);
  console.log(`Error message: ${err.message}`);
 }finally {
  console.log("This always runs.");
```

```
}

// Throwing custom errors
function divide(a, b) {
  if (b === 0) {
    throw new Error("Division by zero is not allowed.");
  }
  return a / b;
}

try {
  console.log(divide(10, 2)); // 5
  console.log(divide(10, 0)); // Throws
} catch (err) {
  console.log(`Caught: ${err.message}`);
}
```

Custom Error Classes

```
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = "ValidationError";
    this.field = field;
  }
}

class InsufficientFundsError extends Error {
  constructor(needed) {
    super(`Insufficient funds. Need $${needed} more.`);
    this.name = "InsufficientFundsError";
    this.needed = needed;
  }
}

try {
  throw new ValidationError("email", "Invalid email format.");
} catch (err) {
  if (err instanceof ValidationError) {
    console.log(`Validation failed on '${err.field}': ${err.message}`);
   else {
    throw err; // Re-throw if not our custom error
  }
}
```

Lesson 8.3: JSON in Node.js

```
const fs = require("fs");

// JavaScript object to JSON string
const settings = {}
  username: "coder_kid",
```

```
  theme: "dark",
  notificationsOn: true,
  favoriteNumbers: [7, 13, 42],
};

const jsonString = JSON.stringify(settings, null, 2); // Pretty-printed
console.log(jsonString);

// Save to file
fs.writeFileSync("settings.json", jsonString);

// Load from file
try {
  const loaded = JSON.parse(fs.readFileSync("settings.json", "utf8"));
  console.log(`Username: ${loaded.username}`);
  console.log(`Theme: ${loaded.theme}`);
  console.log(`Numbers: ${loaded.favoriteNumbers}`);
 catch (err) {
  if (err instanceof SyntaxError) {}
    console.log("Error: JSON file is corrupted.");
  } else if (err.code === "ENOENT") {
    console.log("Error: File not found.");
  } else {
    console.log(`Error: ${err.message}`);
  }
}
```

Skills Checklist

- ☐ I can read and write files using the `fs` module.
- ☐ I can use `try-catch-finally` to handle errors.
- ☐ I can throw custom errors using `Error` or custom classes.
- ☐ I can serialize/deserialize data using `JSON.stringify` and `JSON.parse`.

Unit 9: Modules, Collections, and Testing

Lesson 9.1: JavaScript Modules

Lesson 9.1: JavaScript Modules

CommonJS (Node.js default)

```
// math.js --- exporting
const add = (a, b) => a + b;
const multiply = (a, b) => a * b;
const PI = Math.PI;

module.exports = { add, multiply, PI };

// Or export individually:
// module.exports.add = add;
```

```
// main.js --- importing
const { add, multiply, PI } = require("./math");

console.log(add(5, 3)); // 8
console.log(multiply(4, 6)); // 24
console.log(PI.toFixed(5)); // 3.14159
```

ES Modules (Modern — requires `"type": "module"` in package.json)

```
// math.mjs
export const add = (a, b) => a + b;
export const multiply = (a, b) => a * b;
export const PI = Math.PI;
export default function greet(name) {
  return `Hello, ${name}!`;
}
```

```
// main.mjs
import greet, { add, multiply, PI } from "./math.mjs";

console.log(greet("Alice"));
console.log(add(5, 3));
```

Lesson 9.2: Maps and Sets

`Map` — Key-Value (Better than plain objects for maps)

Accessibility Tip: Run Jest with `npx jest --verbose` to get one result line per test. Each line reads "PASS" or "FAIL" followed by the test name. Navigate the output with arrow keys after running. Verbose mode is far more screen-reader-friendly than the default dot notation.

```
const inventory = new Map();

// Adding
inventory.set("apples", 50);
inventory.set("bananas", 30);
inventory.set("oranges", 40);

// Access
console.log(inventory.get("apples")); // 50
console.log(inventory.has("bananas")); // true
console.log(inventory.size); // 3

// Delete
inventory.delete("bananas");

// Iterate
for (const [key, value] of inventory) {
  console.log(`${key}: ${value}`);
}

inventory.forEach((value, key) => {
  console.log(`${key} = ${value}`);
);

console.log([...inventory.keys()]);
console.log([...inventory.values()]);
console.log([...inventory.entries()]);
```

`Set` — *Unique Values*

```
const uniqueWords = new Set();
uniqueWords.add("apple");
uniqueWords.add("banana");
uniqueWords.add("apple"); // Duplicate --- ignored

console.log(uniqueWords.size); // 2
console.log(uniqueWords.has("banana")); // true
uniqueWords.delete("banana");

// Convert array to set (remove duplicates)
const nums = [1, 2, 2, 3, 3, 3, 4];
const unique = [...new Set(nums)];
console.log(unique); // [1, 2, 3, 4]

// Iterate
for (const item of uniqueWords) {}
  console.log(item);
}
```

Lesson 9.3: Word Frequency Analysis

```
const fs = require("fs");

function countWords(text) {
  const words = text
    .toLowerCase()
    .replace(/[^a-z\s]/g, "")
    .trim()
    .split(/\s+/)
    .filter((word) => word.length > 0);

  const frequency = new Map();
  for (const word of words) {
    frequency.set(word, (frequency.get(word) || 0) + 1);
  }

  return frequency;
}

const text = "the quick brown fox jumps over the lazy dog the quick brown fox";
const freq = countWords(text);

// Sort by frequency (most common first)
const sorted = [...freq.entries()].sort((a, b) => b[1] - a[1]);

console.log("Word Frequencies:");
for (const [word, count] of sorted) {
  console.log(`'${word}': ${count}`);
}

// Top 5 words
console.log("\nTop 5 words:");
sorted.slice(0, 5).forEach(([word, count]) => {
  console.log(`  ${word}: ${count}`);
});
```

Lesson 9.4: Working with CSV Files

```
const fs = require("fs");

function parseCSV(content) {
  const lines = content.trim().split("\n");
  const headers = lines[0].split(",").map((h) => h.trim());

  return lines.slice(1).map((line) => {
    const values = line.split(",").map((v) => v.trim());
    const record = {};
    headers.forEach((header, index) => {}
      record[header] = values[index];
    });
    return record;
  );
}
```

```
function generateCSV(records) {
  if (records.length === 0) return "";
  const headers = Object.keys(records[0]);
  const headerLine = headers.join(",");
  const dataLines = records.map((record) =>
    headers.map((h) => record[h]).join(","),
  );
  return [headerLine, ...dataLines].join("\n");
}

// Read and parse CSV
try {
  const content = fs.readFileSync("students.csv", "utf8");
  const students = parseCSV(content);

  console.log(`Loaded ${students.length} students:`);
  students.forEach((student) => {
    console.log(
      `  Name: ${student.name}, Age: ${student.age}, Grade: ${student.grade}`,
    );
  )};
 }catch (err) {
  if (err.code === "ENOENT") {
    // Create a sample CSV for demonstration
    const sampleData = `name,age,grade
Alice,15,A
Bob,14,B
Charlie,16,A`;
    fs.writeFileSync("students.csv", sampleData);
    console.log("Created sample students.csv");
  }
}
```

Lesson 9.5: Testing with Jest

Setting Up Jest

```
npm init -y
npm install --save-dev jest
```

Add to `package.json`:

```
{
  "scripts": {
    "test": "jest"
  }
}
```

Writing Tests

```
// math.js
const add = (a, b) => a + b;
const getGrade = (score) => {
  if (score >= 90) return "A";
  if (score >= 80) return "B";
  if (score >= 70) return "C";
  return "F";
};
const circleArea = (radius) => Math.PI * radius ** 2;

module.exports = { add}, getGrade, circleArea };
```

```
// math.test.js
const { add, getGrade, circleArea } = require("./math");

describe("add function", () => {
  test("adds two positive numbers", () => {
    expect(add(5, 3)).toBe(8);
  });

  test("adds negative numbers", () => {
    expect(add(-5, -3)).toBe(-8);
  );

  test("adds zero", () => {}
    expect(add(0, 7)).toBe(7);
  });
);

describe("getGrade function", () => {
  test("returns A for score >= 90", () => {}
    expect(getGrade(95)).toBe("A");
    expect(getGrade(90)).toBe("A");
  });

  test("returns B for score 80-89", () => {
    expect(getGrade(85)).toBe("B");
  });

  test("returns F for score < 60", () => {
    expect(getGrade(55)).toBe("F");
  });
);

describe("circleArea function", () => {
  test("calculates area of circle", () => {}
    expect(circleArea(5)).toBeCloseTo(78.54, 1);
  });
);
```

Run tests:

```
npm test
```

Common Jest Matchers

Matcher	Description
`toBe(value)`	Strict equality (===)
`toEqual(value)`	Deep equality (for objects/arrays)
`toBeCloseTo(num, decimals)`	Floating-point comparison
`toBeTruthy()`	Check if truthy
`toBeFalsy()`	Check if falsy
`toBeNull()`	Check if null
`toBeUndefined()`	Check if undefined
`toContain(item)`	Array/string contains item
`toHaveLength(n)`	Check length
`toThrow(error)`	Check if function throws

Skills Checklist

☐ I can create and import modules using CommonJS.
☐ I can use `Map` and `Set` for collections.
☐ I can write and run Jest tests.
☐ I can use common Jest matchers.

Unit 10: Project-Based Learning

Lesson 10.1: Text-Based Adventure Game

```
class Room {
  constructor(name, description) {
    this.name = name;
    this.description = description;
    this.exits = {};
    this.items = [];
  }

  addExit(direction, room) {}
    this.exits[direction.toLowerCase()] = room;
  }

  addItem(item) {
    this.items.push(item);
  }

  describe() {
```

```
    let desc = `--- ${this.name} ---\n${this.description}\n`;
    if (this.items.length > 0) {
      desc += `You see: ${this.items.join(", ")}\n`;
    }
    const exitList = Object.keys(this.exits).join(", ");
    desc += `Exits: ${exitList}`;
    return desc;
  }
}

class Player {
  constructor(name, startRoom) {
    this.name = name;
    this.currentRoom = startRoom;
    this.inventory = [];
    this.health = 100;
  }

  move(direction) {
    const nextRoom = this.currentRoom.exits[direction.toLowerCase()];
    if (nextRoom) {
      this.currentRoom = nextRoom;
      return true;
    }
    return false;
  }

  takeItem(itemName) {
    const index = this.currentRoom.items.findIndex(
      (item) => item.toLowerCase() === itemName.toLowerCase(),
    );
    if (index !== -1) {
      const item = this.currentRoom.items.splice(index, 1)[0];
      this.inventory.push(item);
      return true;
    }
    return false;
  }

  getStatus() {
    const inv = this.inventory.length > 0 ? this.inventory.join(", ") : "empty";
    return `${this.name} | Health: ${this.health} | Inventory: ${inv}`;
  }
}

async function playGame() {
  const { askQuestion, close } = require("./input");

  // Build world
  const cave = new Room("Dark Cave", "You are in a dimly lit cave.");
  const tunnel = new Room(
    "Stone Tunnel",
    "A narrow tunnel runs north and south.",
```

```
);
const clearing = new Room(
  "Sunny Clearing",
  "A bright clearing with tall trees.",
);

cave.addExit("north", tunnel);
cave.addItem("torch");
tunnel.addExit("south", cave);
tunnel.addExit("north", clearing);
clearing.addExit("south", tunnel);
clearing.addItem("magic key");

const player = new Player("Hero", cave);

console.log("=== Text Adventure ===");
console.log("Commands: go [direction], take [item], inventory, look, quit\n");

while (true) {
  console.log(player.currentRoom.describe());
  console.log(player.getStatus());

  const input = (await askQuestion("\n> ")).trim().toLowerCase();
  const [command, ...argParts] = input.split(/\s+/);
  const argument = argParts.join(" ");

  console.log("");

  switch (command) {
    case "go":
      if (!argument) {
        console.log("Go where?");
      } else if (!player.move(argument)) {
        console.log("You can't go that way.");
      }
      break;
    case "take":
      if (player.takeItem(argument)) {
        console.log(`You picked up the ${argument}.`);
       else {
        console.log(`There's no ${argument} here.`);
      }
      break;
    case "inventory":
      console.log(`Inventory: ${player.inventory.join(", ") || "empty"}`);
      break;
    case "look":
      // Description already shown at top
      break;
    case "quit":
      console.log("Thanks for playing!");
      close();
      return;
```

```
      default:
        console.log(`Unknown command: '${command}'`);
    }
  }
}

playGame();
```

Lesson 10.2: Command-Line To-Do Manager

```
const fs = require("fs");
const { askQuestion}, close } = require("./input");

const FILE_PATH = "todos.json";

function loadTodos() {
  try {
    const data = fs.readFileSync(FILE_PATH, "utf8");
    return JSON.parse(data);
  } catch (err) {
    if (err.code === "ENOENT") {
      return []; // File doesn't exist yet
    }
    if (err instanceof SyntaxError) {
      console.log("Warning: Corrupted todos file. Starting fresh.");
      return [];
    }
    throw err;
  }
}

function saveTodos(todos) {
  fs.writeFileSync(FILE_PATH, JSON.stringify(todos, null, 2));
}

function addTask(todos, description) {
  todos.push({
    description,
    completed: false,
    createdAt: new Date().toISOString(),
  });
  console.log(`Task added: "${description}"`);
}

function viewTasks(todos) {
  if (todos.length === 0) {
    console.log("No tasks in your list.");
    return;
  }
  console.log("--- Your To-Do List ---");
  todos.forEach((task, index) => {
    const status = task.completed ? "[X]" : "[ ]";
    console.log(`${index + 1}. ${status} ${task.description}`);
```

```
  );
  console.log("-----------------------");
}

function markComplete(todos, taskNumber) {
  const index = parseInt(taskNumber) - 1;
  if (index >= 0 && index < todos.length) {
    todos[index].completed = true;
    console.log(`Completed: "${todos[index].description}"`);
   }else {
    console.log("Invalid task number.");
  }
}

function deleteTask(todos, taskNumber) {
  const index = parseInt(taskNumber) - 1;
  if (index >= 0 && index < todos.length) {
    const removed = todos.splice(index, 1)[0];
    console.log(`Deleted: "${removed.description}"`);
   else {}
    console.log("Invalid task number.");
  }
}

async function main() {
  let todos = loadTodos();
  console.log("=== To-Do Manager ===");

  while (true) {
    console.log(
      "\n1. Add Task  2. View Tasks  3. Mark Complete  4. Delete  5. Quit",
    );
    const choice = (await askQuestion("Choice: ")).trim();

    switch (choice) {
      case "1":
        const desc = await askQuestion("Task description: ");
        addTask(todos, desc);
        saveTodos(todos);
        break;
      case "2":
        viewTasks(todos);
        break;
      case "3":
        viewTasks(todos);
        const completeNum = await askQuestion("Task number: ");
        markComplete(todos, completeNum);
        saveTodos(todos);
        break;
      case "4":
        viewTasks(todos);
        const deleteNum = await askQuestion("Task number to delete: ");
        deleteTask(todos, deleteNum);
```

```
        saveTodos(todos);
        break;
      case "5":
        console.log("Goodbye!");
        close();
        return;
      default:
        console.log("Invalid choice.");
    }
  }
}

main();
```

Unit 11: Capstone Project — Personal Data Manager in JavaScript

Overview Build a fully functional, accessible command-line personal data manager using Node.js.

Project Options

- **Contact Manager:** name, phone, email, notes
- **Book Tracker:** title, author, genre, read status
- **Recipe Collection:** name, ingredients, instructions, cuisine type
- **Budget Tracker:** category, amount, date, description

Required Features

1. **Add item** — prompt for all attributes and save.
2. **View all items** — display clearly with numbers.
3. **Search/filter** — find items by keyword.
4. **Edit item** — select and modify specific fields.
5. **Delete item** — remove by number.
6. **Persistence** — save/load to JSON file.

Project Structure

```
my-data-manager/
|--- src/
|   |--- models/
|   |   +--- contact.js      (data model/class)
|   |--- storage/
|   |   +--- dataManager.js  (JSON file I/O)
|   |--- ui/
|   |   +--- consoleUI.js    (menu and display)
|   +--- utils/
|       +--- input.js        (reusable input helper)
```

```
|--- data/
|   +--- contacts.json       (data storage)
|--- tests/
|   +--- contact.test.js     (Jest tests)
|--- package.json
|--- README.md
+--- index.js                (entry point)
```

Accessibility Guidelines

1. Every prompt must clearly describe what input is expected.
2. Print a confirmation message after every action.
3. Number all listed items for easy selection.
4. Provide clear, specific error messages.
5. Include a `help` command listing all available options.
6. Use consistent separators (`---`) to group output.

Assessment Criteria

Functionality (60%)

☐ All CRUD operations (Create, Read, Update, Delete) work correctly.
☐ Data persists to JSON between program runs.
☐ Search works by at least one field.
☐ Program handles invalid input without crashing.

Code Quality (20%)

☐ Functions are small with single responsibilities.
☐ Consistent use of `const`/`let`, arrow functions, template literals.
☐ Meaningful names for all variables and functions.
☐ Code is organized into modules.
☐ Includes JSDoc comments for key functions.

Accessibility & UX (15%)

☐ All prompts are descriptive and clear.
☐ Confirmation messages after every action.
☐ Consistent, predictable output format.
☐ Help system available.

Documentation & Tests (5%)

☐ `README.md` with setup and usage instructions.
☐ At least 5 Jest unit tests for key functions.

Deliverables

1. All source code files.
2. `data/` folder with example JSON data.
3. `README.md` including:

   - Installation steps: `npm install`
   - Run instructions: `node index.js`
   - Available commands list.
   - Screen reader usage tips.
   - Reflection on challenges faced.

Part III

JAVA Curriculum

Introduction to Computer Science with Java

For Visually Impaired Students

Course Overview This curriculum introduces computer science concepts using Java, adapted for students who rely on screen readers. All exercises are text-based and terminal-focused. No graphical output is required.

Target Audience: Students with no prior programming experience using screen readers (NVDA, JAWS, VoiceOver, or Windows Narrator).

Tools Required

- **Java Development Kit (JDK) 17+** — `https://adoptium.net/`
- **Visual Studio Code** with the **Extension Pack for Java** by Microsoft
- A screen reader (NVDA, JAWS, or VoiceOver)

Accessibility Notes

- All program output is text-based (console/terminal).
- VS Code's Screen Reader Optimized Mode works well with Java.
- Configure your screen reader to announce indentation changes — Java uses braces `{}` to define blocks, but consistent indentation is still essential for readability.
- Use `Ctrl + Shift + P` -> "Toggle Screen Reader Optimized" in VS Code.
- Use `F8` / `Shift+F8` to navigate between errors.

Units

Unit	Title
Unit 0	Setting Up Your Java Development Environment
Unit 1	Foundations of Computing and Getting Started with Java

Unit	Title
Unit 2	Writing Simple Programs and Numeric Computations
Unit 3	Control Flow — Decisions and Loops
Unit 4	Data Structures — Arrays and ArrayLists
Unit 5	User Input and String Processing
Unit 6	Methods (Functions)
Unit 7	Classes and Object-Oriented Programming
Unit 8	Files and Exception Handling
Unit 9	Collections and Basic Data Analysis
Unit 10	Project-Based Learning
Unit 11	Capstone Project

References

- Eck, D.J. (2022). *Introduction to Programming Using Java* (9th ed.). Free PDF available at https://math.hws.edu/javanotes/
- Horstmann, C.S. (2018). *Big Java: Early Objects* (6th ed.). Wiley.

Teacher's Guide: Introduction to Computer Science with Java

For Students with Visual Impairments — Complete Instructor Edition

---

Table of Contents

1. Philosophy and Why Java
2. Java-Specific Accessibility Challenges
3. Classroom Setup and Configuration
4. Spoken Java Conventions
5. Unit 0–1: Setup and Foundations
6. Unit 2: I/O and the Math Class
7. Unit 3: Control Flow
8. Unit 4: Arrays and ArrayLists
9. Unit 5: Strings and String Processing
10. Unit 6: Methods
11. Unit 7: Classes and OOP
12. Unit 8: Files and Exception Handling
13. Unit 9: Collections, Analysis, and Testing
14. Units 10–11: Projects and Capstone
15. Bridging from Python to Java
16. Assessment Strategies
17. Supporting Diverse Needs
18. Alternative and Supplementary Resources
19. Quick Reference Sheets

---

1. Philosophy and Why Java

The Case for Teaching Java to Blind Students Java is not the easiest language for beginners, and its verbosity creates overhead that requires specific pedagogical attention. Despite this, Java has genuine advantages for visually impaired learners that are often overlooked:

Compile-time error detection. Java's compiler catches type errors, name errors, and structural mistakes before any code runs. A blind student does not need to search through runtime output for subtle type mismatches — the compiler announces them precisely and immediately with file name, line number, and error description.

Strong, static typing is auditorily informative. When IntelliSense in VS Code suggests methods, it includes the return type: "`double getBalance()` — returns a double." [27] This type information helps a screen reader user understand what a method does without having to navigate to its definition.

Java's compiler output follows a consistent, parseable format:

Accessibility Tip: Java compiler errors follow the format `Filename.java:line: error: message`. Navigate to the terminal after compilation, read the first error (line number and message), use `Ctrl+G` in VS Code to jump to that line, fix it, then recompile. Address one error at a time—fixing the first often resolves several others.

Java's compiler output follows a consistent, parseable format: `filename.java:linenum: error: message`. This predictability makes it far easier to navigate with a screen reader than Python's multi-line tracebacks or JavaScript's runtime surprises. Teach students the compiler output format explicitly

Enforced code structure. Every Java program requires a class; every executable program requires a `main` method with a specific signature. This structure is more overhead to type but less ambiguity about how programs are organized. For a student who cannot visually scan a file for structure, predictable structural conventions reduce cognitive load.

Exception specification. Java's checked exceptions force programmers to declare which exceptions a method can throw. This means a screen reader user reading a method signature gets information about failure modes that Python provides only through documentation.

Enterprise relevance. Java remains a dominant language in enterprise software, Android development, and many backend systems. A student who learns Java learns patterns that transfer directly to professional employment.

---

2. Java-Specific Accessibility Challenges

The Verbosity Problem Java requires significantly more text than Python to accomplish the same task:

```java
// Java: Hello, World
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

```python
# Python: Hello, World
print("Hello, World!")
```

For a sighted student, extra text is a visual annoyance. For a blind student, it is additional audio before they reach the meaningful code. **Mitigation strategies:**

1. **Code templates.** Provide a starter file the student copies for every new program. They modify only the method body for early lessons.[28] This reduces the typing burden and prevents structural errors while the student is learning concepts.

2. **JShell (Java REPL).** Java includes an interactive shell since Java 9: run `jshell` in the terminal. JShell strips away class and method boilerplate. For exploration and experimentation, JShell is far more accessible than creating files:

```
jshell> int x = 5;
x ==> 5
```

[28] A single `Starter.java` template that the student renames and modifies is far more accessible than typing the class/main boilerplate from scratch. Create the template on day one, commit it to git, and have students `cp Starter.java NewProgram.java` for every exercise.

```
jshell> System.out.println(x * 2);
10
```

3. **Progressive disclosure.** In early lessons, treat `public static void main(String[] args) {` as a formula, not something to understand. something to understand. Students learn to recognize it by ear. Explain each word only when the concept it relates to (access modifiers, static methods, arrays) is being taught.

"Formula before explanation" is a proven strategy in VI programming education. The student hears the boilerplate, learns to type it, builds muscle memory, and only later learns what each word means. Attempting to explain `public static void main` before the student has written a working program wastes instructional time that should go to concepts.

Integer Division Surprises Java's `/` operator performs integer division when both operands are integers:

- `15 / 4` = `3` (not `3.75`)
- `15.0 / 4` = `3.75`
- `(double) 15 / 4` = `3.75`

This is silent — there is no error or warning. A blind student may not catch a calculation that silently discards its decimal portion. catch a calculation that silently discards its decimal portion. Teach the rule explicitly: **if you want a decimal result, at least one operand must be a `double`.**

Java's integer division is in the same category as JavaScript's type coercion:

Accessibility Tip: When dividing in Java, if you want a decimal result, cast at least one operand: `(double) numerator / denominator`. Integer division silently discards the decimal portion with no warning. Test every division expression in JShell first to verify it produces the expected type of result.

Java's integer division is in the same category as JavaScript's type coercion: a silent wrong-answer failure with no error. Establish the rule as a mantra: "If you want a decimal, cast one operand to `double`." Require students to say it aloud when writing any division expression in early units.

The String Equality Trap `==` on Strings compares memory addresses, not content. Two strings with identical characters in different memory locations are `==` false but `.equals()` true. This never appears in Python; blind students coming from Python are not conditioned to suspect `==` as a source of String bugs.

Teaching approach: Show the broken behavior explicitly. Use `new String("hello") == new String("hello")` which is guaranteed to be `false` even though the strings are identical. Then show `.equals()` which is `true`. Make `.equals()` for Strings a non-negotiable rule from day one.

The `==` vs `.equals()` distinction is Java's most common beginner trap.

Accessibility Tip: In Java, always use `str1.equals(str2)` to compare strings, never `str1 == str2`. The `==` operator compares memory addresses, not content—two strings with identical characters may be `==` false but `.equals()` true. This bug is silent and produces wrong output with no error message.

The `==` vs `.equals()` distinction is Java's most common beginner trap. Python students are especially vulnerable because Python's `==` works correctly on strings. Frame it as: "In Python, `==` compares values. In Java, `==` compares identity—memory address. For Strings, always use `.equals()`." Then never allow `==` on a String in student code.

Null Pointer Exceptions `NullPointerException` is the most common Java runtime error. It occurs when you call a method or access a field on a variable that holds `null` (no object reference). Unlike Python's `AttributeError` which gives useful context, Java's NPE message is often terse. Teaching students to recognize and debug NPEs is essential. [29] [30]

[29] A `NullPointerException` with no message is among the least accessible errors in Java.

Accessibility Tip: Before calling any method on an object reference, check that it is not null: `if (obj != null) { obj.method(); }`. A `NullPointerException` crashes the program immediately and is easy to miss in terminal output if you have navigated away. Null checks on any variable received from user input, file reads, or method returns

---

3. Classroom Setup and Configuration

IDE Configuration **VS Code with Extension Pack for Java:** This is the recommended setup. The Extension Pack includes Language Support for Java (Red Hat), Debugger for Java, Java Test Runner, Maven for Java, and Project Manager for Java.

Alternative IDEs to consider:

IDE	Accessibility	Notes
VS Code + Extension Pack	High	Recommended. Full accessibility via keyboard.
IntelliJ IDEA Community	Medium–High	Powerful but complex. More keyboard shortcuts to learn. JetBrains has an accessibility guide.
Eclipse	Medium	Older UI; can be navigated but more complex accessibility model.
BlueJ	Low–Medium	Designed for teaching; visual design features may impede accessibility.
DrJava	Low–Medium	Simple; not actively maintained.
jGRASP	Low	Visualization-focused; primarily visual.
Greenfoot	Low	Visual/game-focused; not accessible for screen reader use.

VS Code Java Specific Settings:

```
{
  "java.completion.enabled": true,
  "java.format.enabled": true,
  "editor.formatOnSave": true,
  "java.saveActions.organizeImports": true,
  "java.completion.importOrder": ["java", "javax", "com", ""],
  "editor.suggestOnTriggerCharacters": true
}
```

announce available methods. This allows audio exploration of an object's API.

For blind students, IntelliSense triggered by typing . is a form of audio API documentation.

Accessibility Tip: Type a variable name followed by a dot (`myList.`) and wait for IntelliSense to open. Use the arrow keys to navigate the completion list and hear each method name and its return type. This is how you explore what a Java object can do without memorizing or looking up documentation.

For blind students, IntelliSense triggered by typing . is a form of audio API documentation. The student does not need to memorize method names—they can "browse" an object's interface by listening to the completion list. This is a first-class accessibility feature, not a shortcut.

JDK Installation Verification

```
java --version      # Runtime version
javac --version     # Compiler version
jshell              # Start REPL (Ctrl+D or /exit to quit)
```

All three must work before proceeding. If `jshell` is not found, the student may have installed only the JRE (runtime), not the full JDK. The JDK must be installed.

4. Spoken Java Conventions Extend the Python symbol conventions with Java-specific additions:

Symbol/Pattern	Spoken Form
`public`	"public"
`private`	"private"
`static`	"static"
`void`	"void" (returns nothing)
`new`	"new"
`this.`	"this dot"
`super.`	"super dot"
`@Override`	"at-override annotation"
`// comment`	"double-slash comment"
`/* ... */`	"block comment"
`/** ... */`	"JavaDoc comment"
`String[]`	"String array"
`ArrayList<String>`	"ArrayList of String"
`HashMap<String, Integer>`	"HashMap from String to Integer"
`throws IOException`	"throws IOException"
`System.out.println()`	"sysout" or "system-out-println"
`System.out.printf()`	"system-out-printf"
`Integer.parseInt()`	"Integer-parse-int"
`(int) x`	"cast to int"
`null`	"null"
`NullPointerException`	"null pointer exception"
`ArrayIndexOutOfBoundsException`	"array index out of bounds"

Reading a method signature:

```
public double calculateArea(double radius) throws IllegalArgumentException
```

"public double method calculateArea, takes a double called radius, may throw IllegalArgumentException"

---

5. Unit 0–1: Setup and Foundations

Lesson 0.1: Installing the JDK

Instructional Cues

- **Use Adoptium (Eclipse Temurin) rather than Oracle JDK.** The Oracle JDK requires account creation and license agreement navigation.

"Sysout" is the universal Java shorthand.

Accessibility Tip: In VS Code with the Java extension, type `sysout` and press Tab to expand it to `System.out.println()`. This live template saves keystrokes and reduces the chance of typing the verbose output statement incorrectly. Set up similar snippets for `Scanner` and `ArrayList` boilerplate.

"Sysout" is the universal Java shorthand. Most IDEs include a `sysout` live template that expands to `System.out.println()` on Tab. Teaching this shorthand reduces the cognitive overhead of Java's verbose output syntax.

license agreement navigation. [31] [32] Adoptium is fully open-source and simpler to download.

- **Verify the download before running.** Adoptium provides SHA checksums. Teaching students to verify downloads is a security habit worth establishing.
- Ensure this is checked. Many Java tools (Maven, Gradle, some IDEs) require `JAVA_HOME` to be set. [33]

JShell as the First Environment Before any file-based Java, spend time in JShell:

```
$ jshell
|  Welcome to JShell -- Version 17.0.x
|  For an introduction type: /help intro

jshell> 5 + 3
$1 ==> 8

jshell> "Hello " + "World"
$2 ==> "Hello World"

jshell> int x = 42;
x ==> 42

jshell> System.out.println("Value: " + x);
Value: 42
```

JShell commands to teach:

- `/list` — show all code entered so far
- `/edit` — open in editor
- `/vars` — show all variables
- `/methods` — show all methods defined
- `/exit` — quit JShell

Thought Experiments

1. *"Python is interpreted — each line runs immediately. Java compiles to bytecode first, then runs. What does this mean for the kind of errors you encounter, and when? If you have a SyntaxError on line 50, in Python you might not discover it until the program reaches line 50. In Java, when do you discover it?"*

2. *"The Java Virtual Machine (JVM) allows the same Java bytecode to run on Windows, macOS, and Linux. The JVM is different on each platform, but the bytecode is the same. What does this tell you about how Java achieves portability? What alternative approaches could a language designer take?"*

[31] Oracle's JDK download page requires navigating a login dialog

Accessibility Tip: Download the JDK from `adoptium.net` rather than Oracle's website. Adoptium provides direct download links with no account creation or licence acceptance dialog. Choose the LTS version (currently 21) for the most stable and long-supported option.

[33] `JAVA_HOME` is an environment variable that tells build tools (Maven, Gradle, ant) where the JDK is installed. If it is not set, the IDE that has customarily been difficult with screen readers. Verify with `echo %JAVA_HOME%` (Win) or `echo $JAVA_HOME` (Mac/Linux) immediately after installation.

[32] Oracle's JDK download page requires navigating a login dialog and licence acceptance that has historically been difficult with screen readers. Adoptium (Win) provides a straightforward download page with direct links to installers, no account required.

JShell is Java's answer to the Python REPL.

Accessibility Tip: Start every new Java concept in JShell before writing a full class. Type an expression or statement and hear the result immediately. JShell eliminates the compile-run-navigate cycle, giving immediate audio feedback equivalent to the Python REPL.

JShell is Java's answer to the Python REPL. For VI students, this is the recommended first Java environment for the same reason the Python REPL is: immediate audio feedback with no file navigation, compilation step, or run command.

3. *"JShell is a Read-Eval-Print Loop (REPL). Python has one; JavaScript has one; many languages have one. But Java didn't have one until 2016. Why might a language's designers add a REPL 20 years after the language was created? What did programmers do without it?"*

4. *"Java was originally designed for embedded systems (like televisions), then became dominant for web servers, then for Android apps. Each time it succeeded in a new domain, it required extending the language. What does this history suggest about the relationship between a language's design and the problems it eventually solves?"*

Lesson 1.2: Your First Java Program

Instructional Cues **Anatomy reading exercise — before typing any code:** Read the Hello World program aloud, symbol by symbol, using the pronunciation table:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

"public class HelloWorld open-brace newline four-spaces public static void main open-paren String array args close-paren open-brace newline eight-spaces System dot out dot println open-paren quote Hello comma World! close-quote close-paren semicolon newline four-spaces close-brace newline close-brace"

This exercise establishes pattern recognition for the boilerplate that the student will type hundreds of times. Making it auditorily familiar early reduces cognitive load later.

Three `System.out` methods:

```
System.out.println("With newline at end");     // Most common
System.out.print("No newline at end");          // For prompts
System.out.printf("Pi is %.2f%n", Math.PI);    // Formatted
```

For accessible programs, `printf` is extremely valuable — it produces consistent, predictable output formatting that reads cleanly with a screen reader.

Thought Experiments

1. *"Every Java program requires a `public class` whose name matches the file name. Why does Java enforce this one-to-one correspondence? What would break if you could put multiple public classes in one file?"*

2. *"The `main` method signature is exactly `public static void main(String[] args)`. It cannot be `public void main()` or `static void main(String[] args)`. Why is this signature fixed? What does each word contribute?"*

3. *"Java uses curly braces `{}` to delimit code blocks; Python uses indentation. From an accessibility perspective — for a blind programmer — which approach provides better information about structure when read aloud? Consider that braces can be read as 'open block' and 'close block,' while indentation changes require screen reader configuration."*

4. *"`System.out.println()` is long to type but descriptively named. Python's `print()` is short. What naming philosophy does each represent? When does conciseness become ambiguity?"*

Lesson 1.3: Variables and Primitive Data Types

Instructional Cues **Type declaration as specification:** Frame Java's type declarations positively: *"When you write `int score = 85`, you're telling Java: this variable will always hold a whole number. Java then protects you — if you accidentally try to put a name or a decimal into it, the compiler tells you immediately."*

The eight primitive types — prioritize four: In this curriculum, students need: `int`, `double`, `boolean`, `char`. The others (`byte`, `short`, `long`, `float`) exist but are rarely needed in beginner programs. Introduce the four primary types fully before acknowledging the others exist.

Type casting demonstration:

```
// Widening (automatic)
int i = 100;
double d = i;   // 100 -> 100.0, automatic

// Narrowing (explicit, may lose data)
double pi = 3.14159;
int truncated = (int) pi;  // 3.14159 -> 3, data LOST

System.out.println("Truncated: " + truncated);  // 3, not 4!
```

The truncation behavior (not rounding) is consistently surprising. Demonstrate it with `(int) 3.9` producing `3`, and ask: "Is this rounding or truncating? What is the difference? When would truncation cause a bug?"

Non-Obvious Accessibility Strategies

- **`System.out.printf()` for consistent numeric output:** `printf("Balance: $%.2f%n", balance)` always produces exactly 2 decimal places. With-

out formatting, `double` values may produce varying numbers of decimal places (`3.0`, `3.14`, `3.1415926535...`) that read differently each time. Consistent formatting makes output more predictable.

- **`Scanner.hasNext___()` methods:** Before calling `scanner.nextInt()`, check `scanner.hasNextInt()`. If the user typed a non-integer, this returns `false` and prevents the exception, allowing graceful re-prompting. For a blind student who may not immediately catch an exception in terminal output, this is defensive programming with real accessibility value.

Thought Experiments

1. *"Java's `int` holds values from about -2.1 billion to +2.1 billion. What happens if you exceed this limit (integer overflow)? The value silently wraps around to the opposite extreme. Name a real-world system where integer overflow has caused a documented failure."*

2. *"Java has `int` (primitive) and `Integer` (object). They hold the same values but behave differently. `int` cannot be null; `Integer` can. `int` cannot be used in a `List<int>` — you need `List<Integer>`. Why would a language designer make this distinction? What does it cost? What does it protect against?"*

3. *"Java's `char` holds a single Unicode character. `char c = 'A'` stores the letter A. But `char` is also a numeric type — `(int) 'A'` is `65`. What does this dual nature (character and number) tell you about how computers store text internally?"*

---

6. Unit 2: I/O and the Math Class

Lesson 2.2: Scanner for User Input

Instructional Cues **The Scanner as a wrapper — explain the layers:**

```
Keyboard
  V produces bytes
System.in (InputStream)
  V raw byte stream
new Scanner(System.in)
  V wraps with text-reading methods
scanner.nextLine(), scanner.nextInt(), etc.
```

This layered model explains why `Scanner` is needed rather than reading from `System.in` directly. It also foreshadows file reading where `new Scanner(new File(...))` replaces `new Scanner(System.in)`.

`nextInt()` / `nextLine()` pitfall — mandatory demonstration:

```
Scanner scanner = new Scanner(System.in);
System.out.print("Enter your age: ");
int age = scanner.nextInt();          // Reads "25", leaves "\n" in buffer
System.out.print("Enter your name: ");
String name = scanner.nextLine();     // Reads the leftover "\n" --- gets EMPTY string!
System.out.println("Hello " + name + ", age " + age);
// Output: "Hello , age 25" --- name is empty!
```

The fix:

```
int age = scanner.nextInt();
scanner.nextLine();          // Consume the leftover newline
String name = scanner.nextLine();   // Now reads properly
```

Run the broken version first. The student should hear the incorrect output and understand why before seeing the fix.

Thought Experiments

1. *"The `Scanner` reads from a 'stream' of input. A stream is a sequence of data that flows from a source to a destination. `System.in` is the keyboard stream. What other streams exist in a typical program? (Think about file streams, network streams, the output stream.)"*
2. *"After `scanner.nextInt()`, there's a leftover newline in the buffer. What IS a buffer? Why does the operating system buffer keyboard input rather than delivering each character the moment a key is pressed?"*
3. *"In GUI (graphical) programs, users fill out forms with text fields and click buttons. There's no `scanner.nextLine()`. What mechanism do GUI programs use to collect user input? How does event-driven GUI input differ from sequential console input?"*

---

7. Unit 3: Control Flow

Lessons 3.1–3.3

Instructional Cues **String comparison — immediate practice:**

```
String day = "Wednesday";


// Wrong --- compares memory addresses
if (day == "Wednesday") {
    System.out.println("Middle of the week");  // May or may not print
}
```

```java
// Correct --- compares content
if (day.equals("Wednesday")) {
    System.out.println("Middle of the week");  // Always correct
}

// Case-insensitive
if (day.equalsIgnoreCase("wednesday")) {
    System.out.println("This works too");
}
```

Create a practice file with 10 comparisons, half using `==` and half using `.equals()`. Ask the student to predict which ones will work correctly before running.

Switch expressions (Java 14+):

Modern Java supports switch expressions that return a value:

```java
String dayType = switch (day) {
    case "Monday", "Tuesday", "Wednesday", "Thursday", "Friday" -> "Weekday";
    case "Saturday", "Sunday" -> "Weekend";
    default -> "Unknown";
};
System.out.println(day + " is a " + dayType);
```

This eliminates the fallthrough problem and is more concise. Teach both traditional `switch` (for reading legacy code) and switch expressions (for writing new code).

Thought Experiments

1. *"Java uses `&&` for AND, `||` for OR, `!` for NOT. Python uses the words `and`, `or`, `not`. Which approach is more readable when heard aloud by a screen reader? Is there a meaningful accessibility argument for one over the other?"*

2. *"The enhanced for loop `for (String item : list)` is simpler to write than the traditional `for (int i = 0; i < list.size(); i++)`. But sometimes you need the index. Name three operations you can do with the traditional loop that you cannot do (directly) with the enhanced loop."*

3. *"Java's `switch` statement traditionally falls through from one case to the next unless you write `break`. This 'feature' has caused countless bugs. Modern Java (14+) has switch expressions that don't fall through. What does this change tell you about how programming language design evolves in response to real-world usage patterns?"*

4. *"Short-circuit evaluation: `if (obj != null && obj.getValue() > 0)` — if `obj` is null, the second condition is never evaluated. This prevents a NullPointerException. How does this behavior make certain coding patterns possible that would otherwise crash?"*

---

8. Unit 4: Arrays and ArrayLists

Lesson 4.1: Arrays

Instructional Cues **The fixed-size limitation — demonstrate, don't just describe:**

```
String[] fruits = new String[3];
fruits[0] = "apple";
fruits[1] = "banana";
fruits[2] = "cherry";
fruits[3] = "date";   // ArrayIndexOutOfBoundsException!
```

Let the student hear the exception and analyze the message: `Index 3 out of bounds for length 3`. Ask: "The message gives you two numbers. What are they? What does each tell you?"

`Arrays.toString()` as mandatory print method:

The first time `print(fruits)` produces `[Ljava.lang.String;@7a5d012b`, stop and explain: "This is the memory address of the array, not its contents. Java arrays don't have a built-in 'print nicely' behavior. Always use `Arrays.toString(fruits)` or loop through manually."

The `for` vs `for-each` tradeoff — make it concrete:

```
// for-each: simple, clear, but no index
for (String fruit : fruits) {
    System.out.println(fruit);
}

// traditional: more complex, but gives index
for (int i = 0; i < fruits.length; i++) {
    System.out.println(i + ": " + fruits[i]);
}

// The numbered version is more accessible!
```

For blind students, numbered output (showing both index and value) is generally more useful than unnumbered output. Teach the traditional `for` loop for array printing specifically because it enables indexed output.

Thought Experiments

1. *"Arrays are stored in contiguous memory — all elements in a row, one after another. This is why accessing by index is fast: the address of element `i` is just the start address plus `i × element size`. Why does this contiguous storage make inserting into the middle of an array slow? What must happen to all the elements after the insertion point?"*

2. *"If you create `String[] arr = new String[5]`, what are the initial values of `arr[0]` through `arr[4]`? Now create `int[] nums = new int[5]` —*

what are the initial values? Why are the defaults different between String arrays and int arrays?"

3. *"Two-dimensional arrays in Java are 'arrays of arrays.' This means each row can have a different length — a 'jagged array.' When would a jagged array be more appropriate than a rectangular 2D array? Give a real-world data example."*

Lesson 4.2: ArrayLists

Instructional Cues **Autoboxing — explain the invisibility:**

```java
ArrayList<Integer> numbers = new ArrayList<>();
numbers.add(42);        // 42 (int) is auto-boxed to Integer(42)
int first = numbers.get(0);  // Integer(42) is auto-unboxed to 42
```

This happens invisibly but has a performance cost. For the course, tell students: "Use `ArrayList<Integer>` when you need a flexible list of integers. The boxing/unboxing is automatic but slightly less efficient than an array of `int`."

Removal by index vs. by value:

```java
ArrayList<String> list = new ArrayList<>(Arrays.asList("a", "b", "c", "b", "d"));
list.remove(1);            // Removes by INDEX --- removes "b" at position 1
list.remove("b");          // Removes by VALUE --- removes first "b" found
list.removeAll(Arrays.asList("b", "c"));  // Removes all matching values
```

Ask the student: "If I call `list.remove(0)` on a list of integers `[10, 20, 30]`, does this remove the element at index 0 (which is `10`) or remove the value `0` (which doesn't exist)? How does Java decide?" (Answer: removes index 0, because `0` is an `int`, not an `Integer`.)

Thought Experiments

1. *"An `ArrayList` doubles its internal capacity when it runs out of space. If you start with capacity 16 and keep adding elements, when does reallocation occur? Trace the capacity: 16 -> 32 -> 64 -> 128… What is the total cost (in element moves) of adding 1000 elements to an initially empty ArrayList this way? Is this efficient?"*

2. *"Java generics `ArrayList<String>` enforce type safety at compile time. Python lists can hold any types mixed together. When is a typed list (Java style) better? When is a mixed list (Python style) more flexible? Which approach produces more reliable programs?"*

---

9. Unit 5: Strings and String Processing

Instructional Cues **String immutability — the Java version:**

In Java, Strings are immutable for the same reason as Python. But Java's `String` object is also **interned** — the JVM may reuse the same `String` object for two variables holding the same literal value. This is why == sometimes works for String comparison (when both refer to the same interned object) but is not reliable.

`StringBuilder` motivation:

```java
// Inefficient --- creates 100,000 String objects
String result = "";
for (int i = 0; i < 100000; i++) {
    result += "x";
}

// Efficient --- modifies a single buffer
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 100000; i++) {
    sb.append("x");
}
String result = sb.toString();
```

The + in a loop creates a new String object on every iteration, then discards the old one. For small loops this is fine; for large loops it's a significant performance problem. `StringBuilder` maintains a mutable buffer.

Thought Experiments

1. *"String concatenation with + is convenient but expensive in loops. In Python, the equivalent is also discouraged — use `''.join(list)` instead. What does this common advice across two languages tell you about how immutable strings interact with loops?"*

2. *"Java's `String.format()` and `printf()` use format specifiers like `%s`, `%d`, `%f`. Python uses f-strings with `{}`. What are the accessibility tradeoffs of each approach? Which reads more naturally when heard aloud? Which is easier to type?"*

3. *"Regular expressions are a tiny language inside a programming language. `\\d+` matches one or more digits. `[a-zA-Z]+` matches one or more letters. Why do programmers use regular expressions despite their difficult-to-read syntax? What would the alternative be for complex pattern matching?"*

---

10. Unit 6: Methods

Instructional Cues **Method overloading — auditory navigation:**

```
public static int add(int a, int b) { return a + b; }
public static double add(double a, double b) { return a + b; }
public static int add(int a, int b, int c) { return a + b + c; }
```

When IntelliSense shows overloads in VS Code, pressing `Shift+Alt+F2` (Accessible View) lists them all. Teach students to use this to discover available overloads.

`void` vs. return type — make the distinction tangible:

```
// void: for side effects only
public static void printGreeting(String name) {
    System.out.println("Hello, " + name + "!");
}

// Return type: for computation
public static double calculateTip(double billAmount, double rate) {
    return billAmount * rate;
}

// Using them:
printGreeting("Alice");                    // Standalone call
double tip = calculateTip(50.00, 0.18);    // Must capture return value
System.out.printf("Tip: $%.2f%n", tip);
```

"`void` methods exist to DO something. Return-type methods exist to CALCULATE something and give you the result. Ask: 'Do I need to use the output later in my code?' If yes, it needs a return type."

Thought Experiments

1. *"Java methods can throw exceptions: `public double sqrt(double n) throws IllegalArgumentException`. This signature tells callers that the method might fail in a specific way. Python has no equivalent syntax. What is the programmer's obligation when they see `throws` in a method signature?"*

2. *"Static methods belong to the class, not to any instance: `Math.sqrt(25)` — you don't need a `Math` object. Instance methods belong to an object: `myString.length()`. What is the conceptual difference? When should a method be static vs. instance?"*

3. *"Method parameters in Java are 'pass by value' for primitives and 'pass by reference' for objects. For primitives: the method gets a copy, and modifying the parameter doesn't affect the original. For objects: the method gets the reference, and modifications to the object's fields DO affect the original. Design a test to confirm this behavior for both cases."*

4. _"Recursion: a method that calls itself. `int factorial(int n) { return n == 0 ? 1 : n _ factorial(n-1); }`. Every recursive solution has an iterative equivalent. When is the recursive version clearer? What is the risk of deep recursion (many nested calls)?"*

Extended Instructional Cues **The mandatory `__str__` equivalent in Java — `toString()`:**

Every student-defined class must override `toString()`. Without it, `System.out.println(myObject)` produces an unreadable string. The `@Override` annotation makes this explicit:

```
@Override
public String toString() {
    return String.format("Dog[name=%s, age=%d]", name, age);
}
```

Require this in every class from Unit 7 onward. Explain: "When your screen reader reads `System.out.println(myDog)`, it reads whatever `toString()` returns. If you don't define `toString()`, you'll hear a memory address. That's useless."

JavaDoc comments as accessible documentation:

Teach this format for every method:

```
/**
 * Calculates the area of this circle.
 *
 * @return the area as a double, in square units
 * @throws IllegalArgumentException if radius is negative
 */
public double getArea() {
    if (radius < 0) throw new IllegalArgumentException("Radius cannot be negative");
    return Math.PI * radius * radius;
}
```

When IntelliSense shows this method in VS Code, pressing `Shift+Alt+F2` reads the JavaDoc aloud. This makes the student's own code navigable via audio — a profound accessibility benefit.

The VS Code symbol navigator: `Ctrl+Shift+O` opens a dropdown listing all methods in the current file. Type a few letters to filter. This allows jumping to any method by name — much more efficient than arrow-key navigation through a long file.

Access modifiers — enforce from day one:

Modifier	Rule	Reason
`private` on fields	Always	Encapsulation; prevents direct manipulation
`public` on getters/setters	When needed	Controlled access
`private` on helper methods	Yes	Internal implementation detail

Modifier	Rule	Reason
`public` on methods the user calls	Yes	Interface to the class

Establish this as a non-negotiable habit, not a preference. *"Private fields, public interface. Every time."*

Thought Experiments

1. *"Encapsulation means the internal state of an object is private. Only its public methods can access or modify it. Why would you want to prevent direct access to `account.balance`? What bugs could occur if balance were public?"*
2. *"Inheritance creates a hierarchy. A `SavingsAccount` extends `BankAccount`. Every `SavingsAccount` is-a `BankAccount`, but not every `BankAccount` is-a `SavingsAccount`. What are the type-safety implications of this? If you have a `BankAccount` variable, can it hold a `SavingsAccount` object?"*
3. *"Java interfaces define contracts — a list of methods that any implementing class must provide. A class can implement multiple interfaces but can only extend one class. Why this asymmetry? What problem does single inheritance solve? What does it prevent?"*
4. *"Design Pattern: the Factory Method creates objects without specifying the exact class. Instead of `new Dog()`, you'd call `AnimalFactory.create("dog")`. What is the advantage of this indirection? When would it be useful?"*
5. *"Object-oriented programming was supposed to model the real world. A `Dog` is-a `Animal`. But real-world relationships are complex — a `FlyingFish` is both a `Fish` and can fly like a `Bird`. How does Java's single-inheritance model handle this situation? What alternatives exist?"*
6. *"Polymorphism means one interface, many implementations. `Animal[] animals = {new Dog(), new Cat(), new Bird()}; for (Animal a : animals) { a.speak(); }` — each animal speaks differently. Why is this more flexible than checking `instanceof` and calling different methods explicitly?"*

Alternative Resources

- **"Head First Design Patterns" by Freeman, Robson et al.:** Visual book, but audio-describable concepts. Covers fundamental OOP design patterns. For advanced students.
- **"Effective Java" by Joshua Bloch:** The definitive book on Java best practices. Dense but invaluable for understanding WHY Java is designed the way it is.

- **Oracle Java Tutorials (docs.oracle.com/javase/tutorial/):** Comprehensive, plain HTML, fully accessible. Covers OOP concepts with extensive examples.

---

12. Unit 8: Files and Exception Handling

Instructional Cues **Checked vs. unchecked exceptions — the core distinction:**

Type	Examples	Must Handle?	Why?
Checked	`IOException`, `FileNotFoundException`, `SQLException`	Yes — compiler enforces	These are recoverable; programmer should handle them
Unchecked (RuntimeException)	`NullPointerException`, `ArrayIndexOutOfBoundsException`, `IllegalArgumentException`	No — optional	Usually indicate programming errors; may not be recoverable

"Checked exceptions are like warning labels: 'This operation can fail. You must decide what to do if it does.' Unchecked exceptions are usually bugs — the program did something wrong."

Try-with-resources — teach first, manual close second:

```
// Modern: try-with-resources (always teach this first)
try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.out.println("Error reading file: " + e.getMessage());
}
// reader is automatically closed here, even if an exception occurred

// Legacy: manual close (show this so students understand what try-with-resources does)
BufferedReader reader = null;
try {
    reader = new BufferedReader(new FileReader("data.txt"));
    // ... use reader
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    if (reader != null) {
        try { reader.close(); } catch (IOException e) { /* ignore */ }
```

```
    }
}
```

Show the legacy version to explain why try-with-resources exists and what it simplifies.

Thought Experiments

1. *"Checked exceptions force you to handle failures at compile time. Critics argue this creates 'exception chaining hell' — every method signature becomes littered with `throws` clauses. Defenders argue it makes failure modes explicit and programs more reliable. Which side do you find more persuasive? Why?"*

2. *"Java's `Path` and `Files` classes (java.nio.file) are a modern replacement for the older `File` class. The older `File` class's `delete()` method returns a `boolean` (true/false) instead of throwing an exception. What is the danger of reporting failure via a return value? What might a programmer accidentally do wrong?"*

3. *"A `NullPointerException` is called 'The Billion Dollar Mistake' by its inventor Tony Hoare. Explain why null references cause so many bugs. Newer languages (Kotlin, Swift) have built-in null safety. What would Java code look like if null didn't exist?"*

---

13. Unit 9: Collections, Analysis, and Testing

JUnit Testing Deep Dive

Instructional Cues **Test output is inherently accessible:** JUnit produces plain text output: . for each passing test, `F` for failures, with descriptive failure messages. This is one of the most screen-reader-friendly outputs in the entire course. Let students experience testing by running a test suite and listening to the results before writing their own tests.

Test naming convention — sentence style:

```
@Test
public void calculateArea_withPositiveRadius_returnsCorrectValue() { }

@Test
public void calculateArea_withZeroRadius_returnsZero() { }

@Test
public void calculateArea_withNegativeRadius_throwsException() { }
```

The naming convention `methodName_condition_expectedBehavior` makes test names read as complete sentences when the test runner announces

them. This is both good practice and an accessibility benefit — each test announcement tells you exactly what was tested.

`assertThrows` — test that exceptions work correctly:

```
@Test
public void withdraw_withInsufficientFunds_throwsException() {
    BankAccount account = new BankAccount(100.0);
    assertThrows(InsufficientFundsException.class, () -> account.withdraw(200.0));
}
```

Testing for exceptions is as important as testing for correct outputs. Many beginners only test the "happy path."

Thought Experiments

1. *"Test-driven development (TDD) says: write the test first, then write the implementation. How does writing a test first change the way you think about a method? What is forced to be clear before you write a line of implementation?"*

2. *"Mocking means replacing real dependencies (like a database or file system) with fake versions for testing. Why would you want to test a method that reads from a database without using a real database? What complexity does mocking introduce?"*

3. *"A project has 100% test coverage — tests execute every line. Is the software correct? Name a bug that 100% coverage would not catch. (Hint: think about a bug that only appears with certain combinations of inputs.)"*

---

14. Units 10–11: Projects and Capstone

Java Project Architecture For the capstone, encourage a clean package structure:

```
src/
|--- com/
|   +--- student/
|       +--- datamanager/
|           |--- model/
|           |   +--- Contact.java
|           |--- storage/
|           |   |--- DataManager.java
|           |   +--- JsonStorage.java
|           |--- ui/
|           |   +--- ConsoleUI.java
|           +--- Main.java
```

```
test/
+--- com/
    +--- student/
        +--- datamanager/
            |--- model/
            |   +--- ContactTest.java
            +--- storage/
                +--- DataManagerTest.java
```

Navigating packages in VS Code: `Ctrl+Shift+E` for Explorer (shows package tree), `Ctrl+P` to open any file by name, `Ctrl+Shift+O` for symbols within the current file.

IDE-Generated Code Accessibility VS Code and IntelliJ can generate getters, setters, constructors, and toString. Teach these as time-savers, but require students to read and understand all generated code:

- `Ctrl+Shift+P` -> "Java: Generate Getters and Setters"
- `Ctrl+Shift+P` -> "Java: Override/Implement Methods"
- `Ctrl+Shift+P` -> "Java: Generate toString"

Generated code is correct but often not styled for accessibility (missing JavaDoc, generic names). Require students to add JavaDoc and verify `toString()` output is meaningful when heard.

Thought Experiments

1. *"Java in enterprise software is often criticized for having too much 'boilerplate' — code that exists for structural reasons rather than to express logic. How does this boilerplate affect the experience of maintaining a large Java codebase? What tools and patterns does the Java ecosystem provide to reduce it?"*

2. *"Your capstone application stores data in a file. If your team adds a new field to the data class, existing saved files no longer match the new format. This is a 'schema migration' problem. How would you design your storage layer to handle this gracefully?"*

---

15. Bridging from Python to Java For courses that teach Python before Java, use these comparison thought experiments to bridge the languages:

Syntax bridge table:

Python	Java	Notes
`print("Hello")`	`System.out.println("Hello")`	
`x = 5`	`int x = 5;`	Java requires type declaration and semicolon
`if condition:`	`if (condition) {`	Parens around condition; brace instead of colon
`elif`	`else if`	Two words in Java
`for item in list:`	`for (Type item : list) {`	Java requires type
`def func(x):`	`public static void func(int x) {`	Return type and access modifier required
`return x`	`return x;`	Semicolon
`# comment`	`// comment`	
`None`	`null`	
`True/False`	`true/false`	Lowercase in Java
`len(list)`	`list.size()`	Method call, not function
`list.append(x)`	`list.add(x)`	
`dict["key"]`	`map.get("key")`	
`try/except`	`try/catch`	
`import math`	`import java.lang.Math;` (or use `Math.` directly)	

Thought experiments for bridging:

1. *"In Python you write `x = 5` and Python figures out the type. In Java you write `int x = 5` and declare the type explicitly. Which approach do you prefer after using both? Which one would you choose for a large team project? For a quick experiment?"*

2. *"Python lists can hold any mixture of types: `[1, 'hello', True, 3.14]`. Java's `ArrayList<String>` can only hold Strings. What bugs does Python's flexibility enable? What bugs does it prevent compared to Java?"*

3. *"Python functions are first-class objects — you can store them in variables, pass them as arguments. In Java (before lambdas), this was much more complex. Java added lambdas in 2014. What does it mean for a language to add a major feature 19 years after it was created?"*

---

16. Assessment Strategies

Alternative Assessment Forms **Code review presentations:** The student explains their code out loud, as if presenting to a team. The teacher

asks questions about design decisions. This tests understanding more authentically than multiple-choice questions.

Bug-hunting assessments: Provide a working program with 5–8 intentional bugs of different types. The student must find and fix them using only their screen reader and development tools. This directly tests real-world debugging skill.

Design walkthroughs: Give the student a problem and ask them to design the solution — class structure, methods, data structures — without writing any code. Assess the quality of the design, not the typing speed.

Pair programming sessions: Two students work on one computer. One dictates code; the other types. Roles swap every 10–15 minutes. This surfaces communication and reasoning skills that solo assessments miss.

---

17. Supporting Diverse Needs

For Braille Display Users Java's verbose syntax can be particularly burdensome on a braille display with limited width. Strategies:

- Keep lines under 79 characters (PEP 8 applies to style; the Java equivalent is generally under 100).
- Use shorter variable names for very long class hierarchies where the full name is unwieldy.
- Encourage use of IntelliJ IDEA, which has better braille display integration than VS Code in some configurations.

For Students New to Screen Readers If a student is still developing screen reader proficiency:

- Begin with JShell instead of file-based programming. Single-line interactions provide immediate feedback.
- Use pair work more extensively until independent navigation is fluent.
- Reduce the number of new concepts per session until screen reader fluency stabilizes.

---

18. Alternative and Supplementary Resources

Textbooks and Reading

Resource	Level	Format	Accessibility
Introduction to Programming Using Java (Eck, free online)	Beginner–Intermediate	Free HTML/PDF	HTML version excellent; comprehensive
Big Java: Early Objects (Horstmann)	Beginner	Print/ebook	Standard university text; clear explanations
Effective Java, 3rd Ed. (Bloch)	Intermediate–Advanced	Print/ebook	Essential for understanding Java's design philosophy
Head First Java (Sierra, Bates)	Beginner	Print	Very visual; but text portions accessible; engaging writing style
Java: The Complete Reference (Schildt)	Reference	Print/ebook	Encyclopedic; good for looking up specific features
Oracle Java Tutorials (docs.oracle.com)	Beginner–Advanced	Free HTML	Excellent HTML accessibility; definitive
Java Documentation (javadoc)	Reference	Free HTML	Standard library docs; fully screen-reader-accessible

Online Courses and Platforms

Platform	Course	Accessibility	Notes
edX / MITx 6.092	Introduction to Programming in Java	High	MIT course; video transcripts available
Coursera	Java Programming (Duke)	Medium	Free audit available
Codecademy	Learn Java	Partial	Some UI inaccessible; editor readable
MOOC.fi (University of Helsinki)	Java Programming MOOC	High	Text-based exercises; excellent accessibility
CodingBat Java	Java practice problems	High	Plain HTML; no complex UI
Exercism Java Track	Structured exercises with mentoring	High	Terminal-based; excellent
LeetCode	Algorithm problems	Medium	Code editor accessible; UI moderate

MOOC.fi is especially recommended for blind students — the University of Helsinki's Java course has no-frills HTML exercises and very clear progression. It is one of the most accessible Java learning platforms available.

Video and Audio Resources

Resource	Description	Accessibility
CS50 AP (Harvard)	AP Computer Science A course using Java	Captions and transcripts
Derek Banas Java Tutorials (YouTube)	Concise concept-by-concept tutorials	Captions available
Cave of Programming Java (YouTube/website)	Beginner Java series	Captions; text summaries
thenewboston Java (YouTube)	Large beginner series	Captions available
Coding with John (YouTube)	Clear explanations of Java concepts	Good pacing for beginners

---

19. Quick Reference Sheets

Java Compilation and Running

```
# Compile a single file
javac HelloWorld.java    # Creates HelloWorld.class

# Run compiled program
java HelloWorld          # No .class extension!

# Compile with output directory
javac -d bin src/HelloWorld.java

# Run JShell
jshell                   # Interactive REPL
/exit                    # Exit JShell

# Run tests (if using Maven)
mvn test

# Run tests (if using Gradle)
./gradlew test
```

Error Type Quick Reference

Error Type	Common Cause	First Check
`cannot find symbol`	Misspelled name or missing import	Spelling; check imports
`incompatible types`	Assigning wrong type or missing cast	Type of both sides of assignment
`';' expected`	Missing semicolon	End of the line indicated

Error Type	Common Cause	First Check
`reached end of file while parsing`	Missing `}`	Count open braces vs close braces
`non-static method cannot be referenced from a static context`	Calling instance method from `main` without object	Need to create an object first
`NullPointerException`	Calling method on null reference	Which variable is null
`ArrayIndexOutOfBoundsException`	Index beyond array length	Print length and the index used
`ClassCastException`	Invalid cast	Check `instanceof` before casting
`StackOverflowError`	Infinite recursion	Verify base case
`NumberFormatException`	`parseInt()` on non-numeric string	Check input string before parsing

Scanner Method Reference

```
Scanner scanner = new Scanner(System.in);

String line   = scanner.nextLine();     // Entire line as String
String word   = scanner.next();         // Single token (whitespace-delimited)
int n         = scanner.nextInt();      // Integer
double d      = scanner.nextDouble();   // Double
boolean b     = scanner.nextBoolean();  // Boolean (true/false)

// Safe versions with type checking
if (scanner.hasNextInt()) {
    int n = scanner.nextInt();
}

// After nextInt(), consume leftover newline:
scanner.nextLine();  // ALWAYS do this before reading a String

// From file
Scanner fileScanner = new Scanner(new File("data.txt"));
// Same methods work; wrap in try-catch for FileNotFoundException

scanner.close();  // Always close when done
```

Accessible Output Template

```
// Numbered list output
System.out.println("--- Contacts ---");
for (int i = 0; i < contacts.size(); i++) {
    System.out.printf("%d. %s%n", i + 1, contacts.get(i));
}
System.out.printf("Total: %d contacts%n", contacts.size());

// Menu
```

```
System.out.println("\n--- Main Menu ---");
System.out.println("1. Add contact");
System.out.println("2. View all contacts");
System.out.println("3. Search contacts");
System.out.println("4. Delete contact");
System.out.println("5. Save and quit");
System.out.println("------------------");
System.out.print("Enter option (1-5): ");

// Confirmation messages
System.out.printf("Contact '%s' added successfully.%n", name);
System.out.printf("Found %d matching records.%n", count);
System.out.println("Data saved to contacts.json.");
System.out.println("Error: Name field cannot be empty. Please try again.");
```

Unit 0: Setting Up Your Java Development Environment

Overview This unit walks through installing Java and configuring VS Code for accessible Java development.

Lesson 0.1: Installing the Java Development Kit (JDK)

What Is the JDK? The **JDK (Java Development Kit)** includes everything you need to write, compile, and run Java programs:

- **JDK** — Tools for developers (compiler, debugger).
- **JVM (Java Virtual Machine)** — Runs compiled Java programs.
- **JRE (Java Runtime Environment)** — Libraries and runtime support.

Installing the JDK **Recommended:** Download Eclipse Temurin (OpenJDK) from `https://adoptium.net/`.

Choose **JDK 17 LTS** (Long Term Support) for stability.

Windows

1. Download the `.msi` installer.
2. **CRUCIAL STEP:** In the installer, ensure "Set JAVA_HOME variable" and "Add to PATH" are checked.
3. Activate "Install" and then "Finish".

macOS

1. Download the `.pkg` installer.
2. Follow the standard macOS installation steps.
3. The JDK is automatically added to your PATH.

Linux (Debian/Ubuntu)

```
sudo apt update
sudo apt install openjdk-17-jdk
```

Verify Installation Open a terminal and run:

```
java --version
javac --version
```

You should hear the version number (e.g., "openjdk 17.0.x").

Skills Checklist

☐ I have installed JDK 17 or later.
☐ Running `java --version` in the terminal shows the version number.
☐ Running `javac --version` in the terminal shows the compiler version.

Lesson 0.2: Installing VS Code and the Java Extension Pack

1. Install VS Code from https://code.visualstudio.com/.
2. Open VS Code and press `Ctrl + Shift + X` to open Extensions.
3. Search for **"Extension Pack for Java"** by Microsoft. Install it. This installs: Language Support for Java, Debugger for Java, Java Test Runner, and Maven support.

Configure Screen Reader Mode

- Press `Ctrl + Shift + P` -> type "Toggle Screen Reader Optimized" -> press Enter.

VS Code Keyboard Shortcuts for Java

Action	Shortcut
Run Java file	`Ctrl + F5`
Debug Java file	`F5`
Command Palette	`Ctrl + Shift + P`
Problems View	`Ctrl + Shift + M`
Next Error	`F8`
Go to Definition	`F12`
Format Code	`Shift + Alt + F`

Skills Checklist

☐ VS Code is installed with the Extension Pack for Java.
☐ I can toggle Screen Reader Optimized Mode.
☐ I can navigate errors using `F8`.

Lesson 0.3: Your First Java Project

Creating a Project

1. Press `Ctrl + Shift + P` -> "Java: Create Java Project".
2. Select "No build tools".
3. Choose a folder and name the project `HelloWorld`.
4. VS Code will create a `src` folder with a sample file.

Writing Hello World

Writing Hello World Create a file `HelloWorld.java` in the `src` folder:

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Accessibility Tip: In VS Code, press `Ctrl+F5` to compile and run a Java file. Output appears in the Terminal. Use `Ctrl+`` to focus the terminal, then arrow keys to read output. If you see a compiler error, note the line number in the message and press `Ctrl+G` in the editor to jump there.

Running the Program

- Press `Ctrl + F5` to run without debugging.
- Or open the terminal ('Ctrl + ") and type:

```
javac HelloWorld.java
java HelloWorld
```

Key Differences from Python

Python	Java
No required structure	Every program needs a `class`
No semicolons	Statements end with `;`
Indentation defines blocks	Curly braces `{}` define blocks
`print()`	`System.out.println()`
Interpreted	Compiled (`.java` -> `.class` -> run)
Dynamic typing	Static typing (declare variable types)

Skills Checklist

☐ I can create a new Java project in VS Code.
☐ I can write and run a Hello World program.
☐ I understand the basic structure of a Java program.

Unit 1: Foundations of Computing and Getting Started with Java

Overview This unit introduces computer science concepts and the fundamentals of Java programming.

Objectives:

- Understand hardware vs. software.
- Write and run basic Java programs.
- Use variables and data types.

Lesson 1.1: Welcome to Computer Science (See Python Unit 1.1 for conceptual content — hardware, software, algorithms, and problem-solving are language-agnostic.)

Java-Specific Notes Java was created by James Gosling at Sun Microsystems in 1995. Key principles:

- **"Write Once, Run Anywhere"** — Java programs compile to bytecode that runs on any JVM.
- **Strongly typed** — Every variable must have a declared type.
- **Object-Oriented** — Everything lives inside classes.

Lesson 1.2: Your First Java Program

Anatomy of a Java Program

```java
// This is a comment
public class HelloWorld {                    // Class declaration
    public static void main(String[] args) { // Main method --- program entry point
        System.out.println("Hello, World!"); // Print a line to the console
    }
}
```

Key rules:

- The file name **must match** the class name: `HelloWorld.java`.
- Every Java program starts execution from `public static void main(String[] args)`.
- Statements end with a semicolon `;`.
- Blocks of code are surrounded by curly braces `{}`.

Output Methods

```
System.out.println("This prints with a newline at the end.");
System.out.print("This prints without a newline.");
System.out.printf("My name is %s and I am %d years old.%n", "Alice", 15);
```

Examples **Example 1.2.1: Simple Greeting**

```
public class SimpleGreeting {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
        System.out.println("Welcome to Java programming!");
    }
}
```

Example 1.2.2: Using printf for Formatted Output

```
public class FormattedOutput {
    public static void main(String[] args) {
        System.out.printf("Pi is approximately %.2f%n", 3.14159);
        System.out.printf("%-15s %5d%n", "Apples:", 42);
    }
}
```

Skills Checklist

☐ I can write a basic Java program with a `main` method.
☐ I can use `System.out.println()` to display output.
☐ I understand that Java files must match their class name.

Lesson 1.3: Variables and Primitive Data Types

Primitive Data Types

Type	Description	Size	Example
`int`	Whole numbers	32-bit	`int age = 25;`
`long`	Large whole numbers	64-bit	`long population = 8000000000L;`
`double`	Decimal numbers	64-bit	`double pi = 3.14159;`
`float`	Decimal (less precise)	32-bit	`float price = 9.99f;`
`boolean`	True or False	1-bit	`boolean isRaining = false;`
`char`	Single character	16-bit	`char grade = 'A';`

String (Reference Type)

```
String name = "Alice";
String message = "Hello, " + name + "!";  // Concatenation
```

Variable Declaration and Assignment

```
// Declare and assign in one line
int score = 100;
double price = 4.99;
boolean isActive = true;
String studentName = "Charlie";

// Declare first, assign later
int total;
total = 50 + 25;
```

Constants

```
final double PI = 3.14159265358979;
final int MAX_SIZE = 100;
```

Type Conversion

```
// Widening (automatic --- safe)
int intValue = 42;
double doubleValue = intValue;   // 42 -> 42.0

// Narrowing (explicit cast --- may lose data)
double d = 9.99;
int i = (int) d;                 // 9.99 -> 9

// String to number
int parsed = Integer.parseInt("123");
double parsedD = Double.parseDouble("3.14");

// Number to String
String s = String.valueOf(42);
String s2 = Integer.toString(42);
```

Arithmetic Operators

Operator	Description	Example
`+`	Addition	`5 + 3` = 8
`-`	Subtraction	`10 - 4` = 6
`*`	Multiplication	`7 * 6` = 42
`/`	Division	`15 / 4` = `3` (integer division!)
`%`	Modulo	`10 % 3` = 1
`++`	Increment	`count++`
`--`	Decrement	`count--`

Important: Integer division in Java truncates: `15 / 4` = `3`, not `3.75`. To get a decimal result, use `double`: `15.0 / 4` = `3.75`.

Examples **Example 1.3.1: Variables and Arithmetic**

```java
public class VariablesDemo {
    public static void main(String[] args) {
        int appleCount = 5;
        int orangeCount = 3;
        int totalFruit = appleCount + orangeCount;
        System.out.println("You have " + totalFruit + " fruits in total.");

        double pricePerItem = 2.50;
        int quantity = 4;
        double totalCost = pricePerItem * quantity;
        System.out.println("The total cost is: " + totalCost);

        // Integer division
        int a = 10, b = 3;
        System.out.println("10 / 3 = " + (a / b));       // 3
        System.out.println("10.0 / 3 = " + (10.0 / b));  // 3.3333...
        System.out.println("10 % 3 = " + (a % b));        // 1
    }
}
```

Activities **Activity 1.3.1: Personal Information**

```java
public class PersonalInfo {
    public static void main(String[] args) {
        String myName = "Jamie";
        int favoriteNumber = 7;
        double heightInMeters = 1.75;
        boolean likesPizza = true;

        System.out.println("My name is: " + myName);
        System.out.println("Favorite number: " + favoriteNumber);
        System.out.printf("Height: %.2f meters%n", heightInMeters);
        System.out.println("Likes pizza: " + likesPizza);
    }
}
```

Activity 1.3.2: Simple Math

```java
public class SimpleMath {
    public static void main(String[] args) {
        int num1 = 15;
        int num2 = 7;

        System.out.println("Sum: " + (num1 + num2));
        System.out.println("Difference: " + (num1 - num2));
        System.out.println("Product: " + (num1 * num2));
        System.out.printf("Quotient: %.2f%n", (double) num1 / num2);
        System.out.println("Remainder: " + (num1 % num2));
    }
}
```

Skills Checklist

- ☐ I can declare variables with the correct Java types.
- ☐ I can perform basic arithmetic operations in Java.
- ☐ I understand the difference between integer and floating-point division.
- ☐ I can convert between types using casting and parse methods.
- ☐ I can use `final` to declare constants.

Unit 2: Writing Simple Programs and Input/Output

Overview This unit covers getting user input, type conversions, and working with the Math library in Java.

Lesson 2.1: The Software Development Process

Lesson 2.1: The Software Development Process *(See Python Unit 2.1 — the same design principles apply: analyze, design, implement, test.)*

Lesson 2.2: Getting User Input with Scanner

Accessibility Tip: Create your `Scanner` once at the top of `main`: `Scanner scanner = new Scanner(System.in);`. Always print a prompt with `System.out.print()` (not `println`) immediately before each `scanner.nextLine()` call so the screen reader announces what you should type.

The `Scanner` Class Java uses the `Scanner` class to read input from the user:

```java
import java.util.Scanner;

public class UserInput {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("What is your name? ");
        String name = scanner.nextLine();

        System.out.print("How old are you? ");
        int age = scanner.nextInt();

        System.out.println("Hello, " + name + "!");
        System.out.println("Next year you will be " + (age + 1) + " years old.");

        scanner.close();
    }
}
```

Scanner Methods

Method	Reads	Example
`nextLine()`	Entire line as String	`String name = scanner.nextLine();`
`next()`	Single word (up to whitespace)	`String word = scanner.next();`
`nextInt()`	Integer	`int n = scanner.nextInt();`
`nextDouble()`	Double	`double d = scanner.nextDouble();`
`nextBoolean()`	Boolean	`boolean b = scanner.nextBoolean();`

Common Pitfall: `nextInt()` + `nextLine()` After calling `nextInt()`, there is a leftover newline character. Call `scanner.nextLine()` to consume it before reading another string:

```
int age = scanner.nextInt();
scanner.nextLine();            // Consume leftover newline
String name = scanner.nextLine();  // Now reads correctly
```

Examples **Example 2.2.1: Temperature Converter**

```
import java.util.Scanner;

public class TempConverter {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter temperature in Fahrenheit: ");
        double fahrenheit = scanner.nextDouble();

        double celsius = (fahrenheit - 32) * 5.0 / 9.0;

        System.out.printf("%.1f°F is %.1f°C%n", fahrenheit, celsius);
        scanner.close();
    }
}
```

Example 2.2.2: Circle Calculator

```
import java.util.Scanner;

public class CircleCalculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the radius: ");
        double radius = scanner.nextDouble();

        double area = Math.PI * radius * radius;
        double circumference = 2 * Math.PI * radius;

        System.out.printf("Area: %.2f%n", area);
        System.out.printf("Circumference: %.2f%n", circumference);
        scanner.close();
    }
}
```

Skills Checklist

- ☐ I can import and create a `Scanner` object.
- ☐ I can read different types of input from the user.
- ☐ I can handle the `nextInt()`/`nextLine()` newline issue.

Lesson 2.3: The Math Class

```java
// Math class provides static methods --- no import needed
double squareRoot = Math.sqrt(25.0);     // 5.0
double power = Math.pow(2, 10);           // 1024.0
double absolute = Math.abs(-7.5);         // 7.5
int ceiling = (int) Math.ceil(4.2);       // 5
int floor = (int) Math.floor(4.8);       // 4
double rounded = Math.round(4.6);         // 5
double maxVal = Math.max(10, 20);         // 20
double minVal = Math.min(10, 20);         // 10
double piValue = Math.PI;                 // 3.14159...
double eValue = Math.E;                   // 2.71828...
double sinVal = Math.sin(Math.PI / 2);   // 1.0
double logVal = Math.log(Math.E);        // 1.0  (natural log)
double log10 = Math.log10(100);          // 2.0
```

Skills Checklist

☐ I can use common `Math` class methods.
☐ I can use `Math.PI` for circle calculations.

Unit 3: Control Flow — Decisions and Loops

Overview This unit introduces conditional statements and loops in Java.

Objectives:

- Use `if`, `else if`, and `else` for decision-making.
- Use `for`, `while`, and `do-while` loops.
- Use `break` and `continue`.

Lesson 3.1: Boolean Expressions and `if` Statements

Comparison Operators

Operator	Meaning	Example
`==`	Equal to	`age == 18`
`!=`	Not equal to	`score != 0`
`<`	Less than	`temp < 0`
`>`	Greater than	`score > 90`
`<=`	Less than or equal	`age <= 17`
`>=`	Greater than or equal	`score >= 60`

Logical Operators

Operator	Meaning	Example
`&&`	AND (both true)	`age >= 18 && hasId == true`
`||`	OR (at least one true)	`day.equals("Sat") || day.equals("Sun")`
`!`	NOT (negates)	`!isRaining`

`if-else if-else`

```
int score = 85;

if (score >= 90) {
    System.out.println("Grade: A");
} else if (score >= 80) {
    System.out.println("Grade: B");
} else if (score >= 70) {
    System.out.println("Grade: C");
} else {
    System.out.println("Grade: F");
}
```

String Comparison **Important:** Never use `==` to compare Strings. Use `.equals()`:

```
String color = "blue";
if (color.equals("blue")) {
    System.out.println("It's blue!");
}
if (color.equalsIgnoreCase("BLUE")) {
    System.out.println("Case-insensitive match!");
}
```

`switch` Statement

```
int day = 3;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    default:
        System.out.println("Other day");
}
```

Examples **Example 3.1.1: Age Check**

```java
import java.util.Scanner;

public class AgeCheck {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter your age: ");
        int age = scanner.nextInt();

        if (age >= 18) {
            System.out.println("You are an adult.");
        } else {
            System.out.println("You are a minor.");
        }
        scanner.close();
    }
}
```

Example 3.1.2: Grade Calculator

```java
import java.util.Scanner;

public class GradeCalculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter your score (0-100): ");
        int score = scanner.nextInt();

        String grade;
        if (score >= 90) {
            grade = "A";
        } else if (score >= 80) {
            grade = "B";
        } else if (score >= 70) {
            grade = "C";
        } else if (score >= 60) {
            grade = "D";
        } else {
            grade = "F";
        }

        System.out.println("Your grade: " + grade);
        scanner.close();
    }
}
```

Skills Checklist

☐ I can write `if`, `else if`, and `else` statements in Java.
☐ I can use `&&`, `||`, and `!` to combine conditions.
☐ I use `.equals()` (not `==`) to compare Strings.
☐ I can write a `switch` statement.

Lesson 3.2: `for` Loops

```java
// Basic for loop
for (int i = 0; i < 5; i++) {
    System.out.println("i = " + i);
}

// Counting down
for (int i = 10; i > 0; i--) {
    System.out.println(i);
}
System.out.println("Blast off!");

// Stepping by 2
for (int i = 0; i <= 10; i += 2) {
    System.out.println(i);
}
```

Enhanced `for` Loop (for-each)

```java
String[] fruits = {"apple", "banana", "cherry"};
for (String fruit : fruits) {
    System.out.println("I like " + fruit + ".");
}

int[] numbers = {1, 2, 3, 4, 5};
int sum = 0;
for (int num : numbers) {
    sum += num;
}
System.out.println("Sum: " + sum);
```

Examples **Example 3.2.1: Multiplication Table**

```java
public class MultiplicationTable {
    public static void main(String[] args) {
        int n = 5;
        System.out.println("Multiplication table for " + n + ":");
        for (int i = 1; i <= 10; i++) {
            System.out.println(n + " x " + i + " = " + (n * i));
        }
    }
}
```

Skills Checklist

☐ I can write a basic `for` loop with initialization, condition, and update.
☐ I can use an enhanced `for` loop to iterate over an array.

Lesson 3.3: `while` and `do-while` Loops

`while` Loop

Accessibility Tip: In Java, every block is defined by braces `{}`—not indentation. Indent consistently (4 spaces) so that when your screen reader announces indent level changes, they correspond to block entry and exit. Add a comment at closing braces of long blocks: `} // end while` to help you navigate.

`while` Loop

```
int count = 1;
while (count <= 5) {
    System.out.println("Count: " + count);
    count++;
}
```

`do-while` Loop The `do-while` loop runs at least once before checking the condition:

```
Scanner scanner = new Scanner(System.in);
String input;
do {
    System.out.print("Enter a positive number: ");
    int number = scanner.nextInt();
    System.out.println("You entered: " + number);
    System.out.print("Continue? (yes/no): ");
    input = scanner.next();
} while (input.equals("yes"));
```

`break` and `continue`

```
// break --- exit the loop entirely
for (int i = 0; i < 10; i++) {
    if (i == 5) break;
    System.out.println(i);   // Prints 0, 1, 2, 3, 4
}

// continue --- skip the current iteration
for (int i = 0; i < 10; i++) {
    if (i % 2 == 0) continue;
    System.out.println(i);   // Prints 1, 3, 5, 7, 9
}
```

Skills Checklist

- ☐ I can write a `while` loop.
- ☐ I can write a `do-while` loop.
- ☐ I can use `break` to exit a loop early.
- ☐ I can use `continue` to skip an iteration.

Unit 4: Data Structures — Arrays and ArrayLists

Overview This unit introduces Java's built-in array type and the flexible `ArrayList` class.

Lesson 4.1: Arrays

Declaring and Initializing Arrays

```java
// Declare and allocate
int[] numbers = new int[5];            // [0, 0, 0, 0, 0]

// Declare and initialize with values
String[] colors = {"red", "blue", "green", "yellow"};

// Access elements (zero-based index)
System.out.println(colors[0]);         // "red"
System.out.println(colors[colors.length - 1]);  // "yellow"

// Modify an element
colors[1] = "purple";

// Loop through array
for (int i = 0; i < numbers.length; i++) {
    numbers[i] = i * 2;
    System.out.println("numbers[" + i + "] = " + numbers[i]);
}
```

Common Array Operations

```java
import java.util.Arrays;

int[] data = {5, 2, 9, 1, 7, 3};

// Sort
Arrays.sort(data);
System.out.println(Arrays.toString(data));   // [1, 2, 3, 5, 7, 9]

// Search (array must be sorted first)
int index = Arrays.binarySearch(data, 7);
System.out.println("Found 7 at index: " + index);

// Copy
int[] copy = Arrays.copyOf(data, data.length);
int[] partial = Arrays.copyOfRange(data, 1, 4);  // [2, 3, 5]
```

Skills Checklist

- ☐ I can declare and initialize an array in Java.
- ☐ I can access and modify array elements by index.
- ☐ I can iterate over an array using a `for` loop.
- ☐ I can use `Arrays.sort()` and `Arrays.toString()`.

Accessibility Tip: Prefer `ArrayList` over arrays. To inspect contents accessibly, iterate with index and print one item per line: `for (int i = 0; i < list.size(); i++) System.out.println((i+1) + ". " + list.get(i));`. Numbered output is easy to navigate line-by-line with a screen reader.

Lesson 4.2: ArrayLists `ArrayList` is a resizable array that can grow or shrink dynamically.

```java
import java.util.ArrayList;

ArrayList<String> shoppingList = new ArrayList<>();

// Add items
shoppingList.add("milk");
shoppingList.add("bread");
shoppingList.add("eggs");
shoppingList.add(1, "butter");   // Insert at index 1

// Access items
System.out.println(shoppingList.get(0));          // "milk"
System.out.println(shoppingList.size());           // 4

// Remove items
shoppingList.remove("bread");                     // Remove by value
shoppingList.remove(0);                           // Remove by index

// Check membership
boolean hasMilk = shoppingList.contains("milk");

// Loop through
for (String item : shoppingList) {
    System.out.println("- " + item);
}
```

ArrayList vs. Array

Feature	`Array`	`ArrayList`
Size	Fixed	Dynamic
Type	Primitive or Object	Objects only
Syntax	`int[]`	`ArrayList<Integer>`
`.length` / `.size()`	`.length`	`.size()`
Performance	Slightly faster	More flexible

Skills Checklist

- ☐ I can create an `ArrayList` with a generic type.
- ☐ I can add, access, remove, and search items in an `ArrayList`.
- ☐ I can explain when to use an array vs. an `ArrayList`.

Lesson 4.3: Multidimensional Arrays

```java
// 2D array (matrix)
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// Access element at row 1, column 2
System.out.println(matrix[1][2]);   // 6

// Loop through 2D array
for (int row = 0; row < matrix.length; row++) {
    for (int col = 0; col < matrix[row].length; col++) {
        System.out.print(matrix[row][col] + " ");
    }
    System.out.println();
}
```

Skills Checklist

☐ I can declare and initialize a 2D array.
☐ I can access elements using row and column indices.
☐ I can iterate through a 2D array using nested loops.

Unit 5: Strings and String Processing

Overview Java's `String` class is rich with methods for text manipulation.

Lesson 5.1: Working with Strings

String Basics

```
String name = "Alice Smith";

// Length
System.out.println(name.length());           // 11

// Accessing characters
System.out.println(name.charAt(0));          // 'A'
System.out.println(name.charAt(name.length() - 1));  // 'h'

// Substrings
System.out.println(name.substring(6));       // "Smith"
System.out.println(name.substring(0, 5));    // "Alice"

// Case conversion
System.out.println(name.toUpperCase());      // "ALICE SMITH"
System.out.println(name.toLowerCase());      // "alice smith"

// Trimming whitespace
String padded = "  hello  ";
System.out.println(padded.trim());           // "hello"
System.out.println(padded.strip());          // "hello" (better Unicode support)

// Searching
System.out.println(name.indexOf("Smith"));   // 6
System.out.println(name.contains("Alice"));  // true
System.out.println(name.startsWith("Ali"));  // true
System.out.println(name.endsWith("ith"));    // true

// Replacing
String result = name.replace("Alice", "Bob");   // "Bob Smith"
String clean = "a,b,c".replace(",", " ");       // "a b c"

// Splitting
String sentence = "the quick brown fox";
String[] words = sentence.split(" ");
for (String word : words) {
    System.out.println(word);
}

// Checking equality
String s1 = "hello";
String s2 = "hello";
```

```
System.out.println(s1.equals(s2));           // true (correct)
System.out.println(s1.equalsIgnoreCase("HELLO")); // true

// Converting other types to String
String fromInt = String.valueOf(42);
String fromDouble = String.valueOf(3.14);
String formatted = String.format("Name: %-10s Age: %3d", "Alice", 15);
```

`StringBuilder` for Efficient Concatenation Use `StringBuilder` when building strings in a loop (more efficient than `+`):

```
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 5; i++) {
    sb.append("Item ").append(i).append("\n");
}
System.out.println(sb.toString());
```

Examples **Example 5.1.1: String Analysis**

```
import java.util.Scanner;

public class StringAnalysis {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String sentence = scanner.nextLine();

        String[] words = sentence.trim().split("\\s+");

        System.out.println("Original: " + sentence);
        System.out.println("Character count: " + sentence.length());
        System.out.println("Word count: " + words.length);
        System.out.println("Uppercase: " + sentence.toUpperCase());
        System.out.println("Lowercase: " + sentence.toLowerCase());

        scanner.close();
    }
}
```

Example 5.1.2: Word Frequency Counter

```
import java.util.HashMap;
import java.util.Map;

public class WordFrequency {
    public static void main(String[] args) {
        String text = "the quick brown fox jumps over the lazy dog the quick brown fox";
        String[] words = text.toLowerCase().split("\\s+");

        Map<String, Integer> frequency = new HashMap<>();
        for (String word : words) {
            frequency.put(word, frequency.getOrDefault(word, 0) + 1);
        }
```

```
        System.out.println("Word Frequencies:");
        for (Map.Entry<String, Integer> entry : frequency.entrySet()) {
            System.out.println("'" + entry.getKey() + "': " + entry.getValue());
        }
    }
}
```

Skills Checklist

- ☐ I can use common `String` methods in Java.
- ☐ I can compare strings using `.equals()`.
- ☐ I can split a string into an array of words.
- ☐ I can use `StringBuilder` for efficient string building.

Unit 6: Methods (Functions)

Overview Java organizes reusable code into **methods** (the Java equivalent of Python functions).

Lesson 6.1: Defining and Calling Methods

Lesson 6.1: Defining and Calling Methods

Method Syntax

```
accessModifier returnType methodName(paramType paramName, ...) {
    // method body
    return value;  // if returnType is not void
}
```

Examples **Example 6.1.1: Simple Greeting Method**

```
public class GreetingDemo {

    public static void greetUser() {
        System.out.println("Hello, Java programmer!");
    }

    public static void greetByName(String name) {
        System.out.println("Hello, " + name + "!");
    }

    public static void main(String[] args) {
        greetUser();
        greetByName("Alice");
        greetByName("Bob");
```

Accessibility Tip: Write a Javadoc comment above every method: `/** Returns the area of a circle. @param r radius @return area */`. VS Code's IntelliSense reads this comment aloud when you type the method name. This is how you document your methods for audio navigation.

```
    }
}
```

Example 6.1.2: Return Values

```
public class Calculator {

    public static int addNumbers(int a, int b) {
        return a + b;
    }

    public static double calculateCircleArea(double radius) {
        return Math.PI * radius * radius;
    }

    public static String getGrade(int score) {
        if (score >= 90) return "A";
        else if (score >= 80) return "B";
        else if (score >= 70) return "C";
        else if (score >= 60) return "D";
        else return "F";
    }

    public static void main(String[] args) {
        int sum = addNumbers(5, 3);
        System.out.println("Sum: " + sum);

        double area = calculateCircleArea(5.0);
        System.out.printf("Circle area: %.2f%n", area);

        System.out.println("Grade for 85: " + getGrade(85));
    }
}
```

Method Overloading Java allows multiple methods with the same name but different parameters:

```
public static int add(int a, int b) { return a + b; }
public static double add(double a, double b) { return a + b; }
public static int add(int a, int b, int c) { return a + b + c; }
```

Passing Arrays to Methods

```
public static int sum(int[] numbers) {
    int total = 0;
    for (int num : numbers) {
        total += num;
    }
    return total;
}

// Calling:
```

```
int[] data = {1, 2, 3, 4, 5};
System.out.println("Sum: " + sum(data));
```

Varargs (Variable Arguments)

```
public static int sumAll(int... numbers) {
    int total = 0;
    for (int num : numbers) {
        total += num;
    }
    return total;
}

// Calling:
System.out.println(sumAll(1, 2, 3));          // 6
System.out.println(sumAll(10, 20, 30, 40));    // 100
```

Skills Checklist

- ☐ I can define a method with a return type and parameters.
- ☐ I can call a method and use its return value.
- ☐ I understand the difference between `void` and non-`void` methods.
- ☐ I can overload methods with different parameter lists.
- ☐ I can pass arrays to methods.

Unit 7: Classes and Object-Oriented Programming

Overview Java is fundamentally object-oriented. This unit covers defining and using classes.

Lesson 7.1: Why Learn About Classes? (See Python Unit 7.1 for conceptual content — OOP concepts are language-agnostic.)

Java is a **class-based** language — every Java program is a class. Understanding OOP is essential for Java.

Accessibility Tip: Override `toString()` on every class you create so that `System.out.println(myObject)` produces a readable description rather than a memory address like `Dog@7852e922`. Return a clear string: `return "Dog[name=" + name + ", age=" + age + "]";`

Lesson 7.2: Creating and Using a Class

Class Syntax

```
public class Dog {
    // Instance variables (fields)
    private String name;
    private int age;

    // Constructor
    public Dog(String name, int age) {
```

```
        this.name = name;
        this.age = age;
    }

    // Methods
    public void sit() {
        System.out.println(name + " is now sitting.");
    }

    public void rollOver() {
        System.out.println(name + " rolled over!");
    }

    public void getOlder(int years) {
        this.age += years;
        System.out.println(name + " is now " + age + " years old.");
    }

    // Getters and Setters
    public String getName() { return name; }
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }

    @Override
    public String toString() {
        return "Dog{name='" + name + "', age=" + age + "}";
    }
}
```

Using the Class

```
public class DogDemo {
    public static void main(String[] args) {
        Dog myDog = new Dog("Buddy", 3);
        Dog yourDog = new Dog("Lucy", 1);

        System.out.println("My dog's name is " + myDog.getName() + ".");
        myDog.sit();
        myDog.rollOver();
        myDog.getOlder(2);

        System.out.println(myDog);
    }
}
```

Access Modifiers

Modifier	Access Level
`public`	Accessible from anywhere
`private`	Only within the same class
`protected`	Within the class and subclasses

Modifier	Access Level
(default)	Within the same package

Best practice: Make instance variables `private` and provide `public` getter/setter methods — this is **encapsulation**.

Skills Checklist

☐ I can define a Java class with fields, a constructor, and methods.
☐ I can create instances of a class using `new`.
☐ I understand the purpose of `this` in Java.
☐ I can write getter and setter methods.
☐ I understand access modifiers (`public`, `private`).

Lesson 7.3: Inheritance

```
public class Vehicle {
    protected String make;
    protected String model;
    protected int year;

    public Vehicle(String make, String model, int year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }

    public String getDescriptiveName() {
        return year + " " + make + " " + model;
    }

    public void readOdometer() {
        System.out.println("No odometer reading available.");
    }
}

public class Car extends Vehicle {
    private int odometer;

    public Car(String make, String model, int year) {
        super(make, model, year);   // Call parent constructor
        this.odometer = 0;
    }

    @Override
    public void readOdometer() {
        System.out.println("This car has " + odometer + " miles on it.");
    }
```

```
    public void updateOdometer(int miles) {
        if (miles >= odometer) {
            odometer = miles;
        } else {
            System.out.println("Error: Cannot roll back odometer.");
        }
    }
}

public class ElectricCar extends Car {
    private int batterySize;

    public ElectricCar(String make, String model, int year) {
        super(make, model, year);
        this.batterySize = 75;
    }

    public void describeBattery() {
        System.out.println("This car has a " + batterySize + "-kWh battery.");
    }
}
```

Abstract Classes and Interfaces

```
// Abstract class --- cannot be instantiated
public abstract class Shape {
    public abstract double area();      // Abstract method --- no body
    public abstract double perimeter(); // Must be implemented by subclasses

    public void printInfo() {           // Concrete method --- has body
        System.out.printf("Area: %.2f, Perimeter: %.2f%n", area(), perimeter());
    }
}

public class Circle extends Shape {
    private double radius;

    public Circle(double radius) { this.radius = radius; }

    @Override
    public double area() { return Math.PI * radius * radius; }

    @Override
    public double perimeter() { return 2 * Math.PI * radius; }
}

// Interface --- defines a contract
public interface Drawable {
    void draw();
    default void resize(double factor) {
        System.out.println("Resizing by factor " + factor);
    }
```

```
}
```

Skills Checklist

- ☐ I can create a subclass that extends a parent class.
- ☐ I can use `super()` to call the parent constructor.
- ☐ I can override methods using `@Override`.
- ☐ I understand abstract classes and interfaces.

Unit 8: Files and Exception Handling

Overview This unit covers reading and writing files in Java and handling exceptions.

Lesson 8.1: Reading from Files

Lesson 8.1: Reading from Files

Accessibility Tip: Always catch specific exception types: `catch (IOException e)` not bare `catch (Exception e)`. Print the message in every catch block: `System.out.println("Error: " + e.getMessage());`. A silent catch block hides bugs that are impossible to debug without visual scanning.

```
import java.io.*;
import java.util.*;

public class FileReaderDemo {
    public static void main(String[] args) {
        // Reading entire file using Scanner
        try (Scanner fileScanner = new Scanner(new File("my_notes.txt"))) {
            while (fileScanner.hasNextLine()) {
                String line = fileScanner.nextLine();
                System.out.println(line);
            }
        } catch (FileNotFoundException e) {
            System.out.println("Error: File not found - " + e.getMessage());
        }

        // Reading with BufferedReader (more efficient for large files)
        try (BufferedReader reader = new BufferedReader(new FileReader("my_notes.txt"))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        }

        // Reading all lines into a List
        try {
            List<String> lines = new java.nio.file.Files.readAllLines(
                java.nio.file.Paths.get("my_notes.txt")
            );
            for (String line : lines) {
```

```
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Simpler approach using `java.nio.file`

```
import java.nio.file.*;
import java.io.IOException;
import java.util.List;

public class ModernFileReader {
    public static void main(String[] args) {
        Path filePath = Paths.get("my_notes.txt");

        try {
            // Read all lines
            List<String> lines = Files.readAllLines(filePath);
            for (String line : lines) {
                System.out.println(line);
            }

            // Read entire file as one string
            String content = Files.readString(filePath);
            System.out.println(content);

        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        }
    }
}
```

Lesson 8.2: Writing to Files

```
import java.io.*;
import java.nio.file.*;

public class FileWriterDemo {
    public static void main(String[] args) {
        // Writing with PrintWriter (creates new or overwrites)
        try (PrintWriter writer = new PrintWriter("output.txt")) {
            writer.println("Hello, Java file I/O!");
            writer.println("Writing is easy.");
            writer.printf("Value: %d%n", 42);
        } catch (FileNotFoundException e) {
            System.out.println("Error: Could not create file.");
        }

        // Appending with FileWriter(true = append mode)
```

```
        try (PrintWriter appender = new PrintWriter(new FileWriter("output.txt", true))) {
            appender.println("This line is appended.");
        } catch (IOException e) {
            System.out.println("Error appending to file.");
        }

        // Modern NIO approach
        try {
            String content = "Hello from NIO!\nSecond line.";
            Files.writeString(Paths.get("modern_output.txt"), content);

            // Append
            Files.writeString(Paths.get("modern_output.txt"),
                "\nAppended line.",
                StandardOpenOption.APPEND);
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Lesson 8.3: Exception Handling

try-catch-finally

```
public class ExceptionHandling {
    public static void main(String[] args) {
        // Basic try-catch
        try {
            int result = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        }

        // Multiple catch blocks
        try {
            String input = "abc";
            int number = Integer.parseInt(input);
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Unexpected error: " + e.getMessage());
        }

        // try-catch-finally
        PrintWriter writer = null;
        try {
            writer = new PrintWriter("data.txt");
            writer.println("Some data");
        } catch (FileNotFoundException e) {
            System.out.println("Could not create file.");
        } finally {
```

```
            if (writer != null) {
                writer.close();
                System.out.println("File closed.");
            }
        }

        // try-with-resources (automatically closes resources)
        try (PrintWriter pw = new PrintWriter("data2.txt")) {
            pw.println("Better approach!");
        } catch (FileNotFoundException e) {
            System.out.println("Could not create file.");
        }
    }
}
```

Common Java Exceptions

Exception	When it occurs
`ArithmeticException`	Division by zero
`NumberFormatException`	Invalid number conversion (e.g., `Integer.parseInt("abc")`)
`ArrayIndexOutOfBoundsException`	Array index out of range
`NullPointerException`	Calling method on null reference
`FileNotFoundException`	File doesn't exist
`IOException`	General I/O error
`ClassCastException`	Invalid type cast

Custom Exceptions

```
public class InsufficientFundsException extends Exception {
    private double amount;

    public InsufficientFundsException(double amount) {
        super("Insufficient funds: need " + amount + " more.");
        this.amount = amount;
    }

    public double getAmount() { return amount; }
}

// Usage:
public void withdraw(double amount) throws InsufficientFundsException {
    if (amount > balance) {
        throw new InsufficientFundsException(amount - balance);
    }
    balance -= amount;
}
```

Skills Checklist

- ☐ I can read from a file using `Scanner` or `BufferedReader`.
- ☐ I can write to a file using `PrintWriter`.
- ☐ I can use `try-catch-finally` to handle exceptions.
- ☐ I can use try-with-resources for automatic resource cleanup.
- ☐ I know common Java exception types.

Unit 9: Collections, Data Analysis, and Testing

Overview This unit introduces Java's Collections Framework, basic data analysis, and unit testing with JUnit.

Lesson 9.1: Java Collections Framework

Lesson 9.1: Java Collections Framework

Accessibility Tip: Run JUnit tests from the terminal with `mvn test` or `./gradlew test`. Use the `--verbose` flag where available to get one result line per test. Each line names the test and its result—far more navigable than summary dots when using a screen reader.

Key Interfaces and Classes

```
Collection
|--- List
|    |--- ArrayList
|    +--- LinkedList
+--- Set
     |--- HashSet
     +--- TreeSet

Map
|--- HashMap
+--- TreeMap
```

`HashMap` — Key-Value Storage

```java
import java.util.HashMap;
import java.util.Map;

HashMap<String, Integer> inventory = new HashMap<>();

// Add entries
inventory.put("apples", 50);
inventory.put("bananas", 30);
inventory.put("oranges", 40);

// Access
System.out.println(inventory.get("apples"));           // 50
System.out.println(inventory.getOrDefault("grapes", 0)); // 0

// Check
System.out.println(inventory.containsKey("bananas"));   // true
System.out.println(inventory.containsValue(30));         // true

// Update
inventory.put("apples", 55);

// Remove
inventory.remove("bananas");
```

```
// Iterate
for (Map.Entry<String, Integer> entry : inventory.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}

// Keys only
for (String key : inventory.keySet()) {
    System.out.println(key);
}

// Values only
for (int value : inventory.values()) {
    System.out.println(value);
}

System.out.println("Size: " + inventory.size());
```

`HashSet` — Unique Elements

```
import java.util.HashSet;

HashSet<String> uniqueWords = new HashSet<>();
uniqueWords.add("apple");
uniqueWords.add("banana");
uniqueWords.add("apple");   // Duplicate --- ignored

System.out.println(uniqueWords.size());          // 2
System.out.println(uniqueWords.contains("banana")); // true
```

`TreeMap` — Sorted Keys

```
import java.util.TreeMap;

TreeMap<String, Integer> sorted = new TreeMap<>();
sorted.put("banana", 2);
sorted.put("apple", 1);
sorted.put("cherry", 3);

// Iterates in alphabetical key order
for (Map.Entry<String, Integer> e : sorted.entrySet()) {
    System.out.println(e.getKey() + ": " + e.getValue());
}
```

Sorting with Collections

```
import java.util.*;

List<Integer> numbers = new ArrayList<>(Arrays.asList(5, 2, 9, 1, 7));
Collections.sort(numbers);
System.out.println(numbers);   // [1, 2, 5, 7, 9]

Collections.sort(numbers, Collections.reverseOrder());
System.out.println(numbers);   // [9, 7, 5, 2, 1]
```

```
int max = Collections.max(numbers);
int min = Collections.min(numbers);
```

Lesson 9.2: Word Frequency Analysis

```
import java.util.*;

public class WordFrequency {
    public static Map<String, Integer> countWords(String text) {
        Map<String, Integer> frequency = new HashMap<>();
        String[] words = text.toLowerCase()
                             .replaceAll("[^a-z\\s]", "")
                             .trim()
                             .split("\\s+");

        for (String word : words) {
            frequency.put(word, frequency.getOrDefault(word, 0) + 1);
        }
        return frequency;
    }

    public static void main(String[] args) {
        String text = "the quick brown fox jumps over the lazy dog the quick brown fox";
        Map<String, Integer> freq = countWords(text);

        // Sort by frequency (most common first)
        List<Map.Entry<String, Integer>> entries = new ArrayList<>(freq.entrySet());
        entries.sort((a, b) -> b.getValue() - a.getValue());

        System.out.println("Word Frequencies:");
        for (Map.Entry<String, Integer> entry : entries) {
            System.out.printf("'%s': %d%n", entry.getKey(), entry.getValue());
        }
    }
}
```

Lesson 9.3: Working with CSV Files

```
import java.io.*;
import java.util.*;

public class CSVReader {
    public static void main(String[] args) {
        String filename = "students.csv";

        try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
            String headerLine = reader.readLine();
            String[] headers = headerLine.split(",");
            System.out.println("Columns: " + Arrays.toString(headers));
            System.out.println("---");
```

```
            String line;
            while ((line = reader.readLine()) != null) {
                String[] data = line.split(",");
                if (data.length == headers.length) {
                    String name = data[0].trim();
                    int age = Integer.parseInt(data[1].trim());
                    String grade = data[2].trim();
                    System.out.printf("Name: %-15s Age: %3d  Grade: %s%n",
                                      name, age, grade);
                }
            }
        } catch (FileNotFoundException e) {
            System.out.println("Error: File not found - " + filename);
        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Error: Invalid number format in CSV.");
        }
    }
}
```

Lesson 9.4: Unit Testing with JUnit

Adding JUnit to Your Project If using VS Code with Maven, add to `pom.xml`:

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
</dependency>
```

Writing Tests

```
// MyMath.java
public class MyMath {
    public static int add(int a, int b) { return a + b; }
    public static double circleArea(double radius) { return Math.PI * radius * radius; }
    public static String getGrade(int score) {
        if (score >= 90) return "A";
        if (score >= 80) return "B";
        if (score >= 70) return "C";
        return "F";
    }
}
```

```
// TestMyMath.java
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;
```

```java
public class TestMyMath {

    @Test
    public void testAddPositiveNumbers() {
        assertEquals(8, MyMath.add(5, 3));
    }

    @Test
    public void testAddNegativeNumbers() {
        assertEquals(-8, MyMath.add(-5, -3));
    }

    @Test
    public void testCircleArea() {
        double expected = Math.PI * 25;
        assertEquals(expected, MyMath.circleArea(5.0), 0.0001);
    }

    @Test
    public void testGradeA() {
        assertEquals("A", MyMath.getGrade(95));
    }

    @Test
    public void testGradeF() {
        assertEquals("F", MyMath.getGrade(55));
    }
}
```

Common JUnit Assertions

Method	Description
`assertEquals(expected, actual)`	Check equality
`assertTrue(condition)`	Check condition is true
`assertFalse(condition)`	Check condition is false
`assertNull(object)`	Check object is null
`assertNotNull(object)`	Check object is not null
`assertThrows(ExceptionClass, executable)`	Check exception is thrown

Skills Checklist

- ☐ I can use `HashMap` for key-value storage in Java.
- ☐ I can iterate through a map's entries, keys, and values.
- ☐ I can write unit tests using JUnit 5.
- ☐ I can use common assertion methods.

Unit 10: Project-Based Learning

Overview This unit guides students through building text-based Java applications.

Lesson 10.1: Designing a Text-Based Adventure Game

Room and Player Structure

```
import java.util.*;

public class Room {
    private String name;
    private String description;
    private Map<String, Room> exits;
    private List<String> items;

    public Room(String name, String description) {
        this.name = name;
        this.description = description;
        this.exits = new HashMap<>();
        this.items = new ArrayList<>();
    }

    public void addExit(String direction, Room room) {
        exits.put(direction.toLowerCase(), room);
    }

    public void addItem(String item) {
        items.add(item);
    }

    public Room getExit(String direction) {
        return exits.get(direction.toLowerCase());
    }

    public String getDescription() {
        StringBuilder sb = new StringBuilder();
        sb.append("--- ").append(name).append(" ---\n");
        sb.append(description).append("\n");
        if (!items.isEmpty()) {
            sb.append("You see: ").append(String.join(", ", items)).append("\n");
        }
        sb.append("Exits: ").append(String.join(", ", exits.keySet()));
        return sb.toString();
    }

    public String getName() { return name; }
    public List<String> getItems() { return items; }
}
```

```
import java.util.*;

public class Player {
    private String name;
    private Room currentRoom;
    private List<String> inventory;
    private int health;

    public Player(String name, Room startRoom) {
        this.name = name;
        this.currentRoom = startRoom;
        this.inventory = new ArrayList<>();
        this.health = 100;
    }

    public boolean move(String direction) {
        Room nextRoom = currentRoom.getExit(direction);
        if (nextRoom != null) {
            currentRoom = nextRoom;
            return true;
        }
        return false;
    }

    public boolean takeItem(String itemName) {
        List<String> roomItems = currentRoom.getItems();
        for (String item : roomItems) {
            if (item.equalsIgnoreCase(itemName)) {
                inventory.add(item);
                roomItems.remove(item);
                return true;
            }
        }
        return false;
    }

    public String getStatus() {
        return "Player: " + name + " | Health: " + health +
               " | Inventory: " + (inventory.isEmpty() ? "empty" : String.join(", ", inventory));
    }

    public Room getCurrentRoom() { return currentRoom; }
    public List<String> getInventory() { return inventory; }
}
```

```
import java.util.Scanner;

public class AdventureGame {
    public static void main(String[] args) {
        // Build the world
        Room cave = new Room("Dark Cave", "You are in a dimly lit cave.");
        Room tunnel = new Room("Stone Tunnel", "A narrow tunnel leads north and south.");
        Room clearing = new Room("Sunny Clearing", "A bright clearing with tall trees.");
```

```
        cave.addExit("north", tunnel);
        cave.addItem("torch");
        tunnel.addExit("south", cave);
        tunnel.addExit("north", clearing);
        clearing.addExit("south", tunnel);

        Player player = new Player("Hero", cave);
        Scanner scanner = new Scanner(System.in);

        System.out.println("=== Text Adventure ===");
        System.out.println("Commands: go [direction], take [item], inventory, look, quit");
        System.out.println();

        while (true) {
            System.out.println(player.getCurrentRoom().getDescription());
            System.out.println(player.getStatus());
            System.out.print("\n> ");

            String input = scanner.nextLine().trim().toLowerCase();
            String[] parts = input.split("\\s+", 2);
            String command = parts[0];
            String argument = parts.length > 1 ? parts[1] : "";

            switch (command) {
                case "go":
                    if (argument.isEmpty()) {
                        System.out.println("Go where?");
                    } else if (!player.move(argument)) {
                        System.out.println("You can't go that way.");
                    }
                    break;
                case "take":
                    if (player.takeItem(argument)) {
                        System.out.println("You picked up the " + argument + ".");
                    } else {
                        System.out.println("There's no " + argument + " here.");
                    }
                    break;
                case "inventory":
                    System.out.println("Inventory: " + player.getInventory());
                    break;
                case "look":
                    // Room description already shown at top of loop
                    break;
                case "quit":
                    System.out.println("Thanks for playing!");
                    scanner.close();
                    return;
                default:
                    System.out.println("Unknown command: " + command);
            }
            System.out.println();
```

```
        }
    }
}
```

Lesson 10.2: Command-Line To-Do Manager

```
import java.util.*;
import java.io.*;
import com.google.gson.*;   // Add Gson library for JSON support

public class TodoManager {
    private List<Map<String, Object>> todos = new ArrayList<>();
    private final String FILE_PATH = "todos.json";

    public void addTask(String description) {
        Map<String, Object> task = new HashMap<>();
        task.put("description", description);
        task.put("completed", false);
        todos.add(task);
        System.out.println("Task added: " + description);
    }

    public void viewTasks() {
        if (todos.isEmpty()) {
            System.out.println("No tasks in your list.");
            return;
        }
        System.out.println("--- Your To-Do List ---");
        for (int i = 0; i < todos.size(); i++) {
            Map<String, Object> task = todos.get(i);
            String status = (Boolean) task.get("completed") ? "[X]" : "[ ]";
            System.out.printf("%d. %s %s%n", i + 1, status, task.get("description"));
        }
        System.out.println("-----------------------");
    }

    public void markComplete(int taskNumber) {
        int index = taskNumber - 1;
        if (index >= 0 && index < todos.size()) {
            todos.get(index).put("completed", true);
            System.out.println("Task marked complete: " + todos.get(index).get("description"));
        } else {
            System.out.println("Invalid task number.");
        }
    }

    public static void main(String[] args) {
        TodoManager manager = new TodoManager();
        Scanner scanner = new Scanner(System.in);

        System.out.println("=== To-Do Manager ===");

        while (true) {
```

```
            System.out.println("\n1. Add Task  2. View Tasks  3. Mark Complete  4. Quit");
            System.out.print("Choice: ");
            String choice = scanner.nextLine().trim();

            switch (choice) {
                case "1":
                    System.out.print("Task description: ");
                    manager.addTask(scanner.nextLine());
                    break;
                case "2":
                    manager.viewTasks();
                    break;
                case "3":
                    manager.viewTasks();
                    System.out.print("Task number: ");
                    try {
                        manager.markComplete(Integer.parseInt(scanner.nextLine()));
                    } catch (NumberFormatException e) {
                        System.out.println("Invalid input.");
                    }
                    break;
                case "4":
                    System.out.println("Goodbye!");
                    scanner.close();
                    return;
                default:
                    System.out.println("Invalid choice.");
            }
        }
    }
}
```

Unit 11: Capstone Project — Personal Data Manager in Java

Overview Students build a fully functional command-line personal data manager in Java.

Project Options

- **Contact Manager:** name, phone, email, notes
- **Book Tracker:** title, author, genre, read status
- **Inventory System:** item name, quantity, location, notes
- **To-Do App:** description, due date, priority, status

Required Features

1. **Add item** — prompt for all attributes and save to list.
2. **View all items** — display all stored items clearly.
3. **Edit item** — select by number/name and modify.
4. **Delete item** — remove selected item.

5. **Search/filter** — find items by keyword.
6. **Persistence** — save/load to a JSON or CSV file.

Architecture Recommendations

```
src/
|--- model/
|   +--- Contact.java        (data model class)
|--- storage/
|   +--- DataManager.java    (file I/O)
|--- ui/
|   +--- ConsoleUI.java      (user interface)
+--- Main.java               (entry point)
```

Assessment Criteria

Functionality (60%)

☐ Add, view, edit, and delete items work correctly.
☐ Data persists between program runs.
☐ Program handles invalid input without crashing.

Code Quality (20%)

☐ Methods are small and have a single responsibility.
☐ Classes follow OOP principles (encapsulation).
☐ Meaningful names for all variables, methods, and classes.
☐ Appropriate comments and Javadoc.

Accessibility & UX (15%)

☐ Clear, descriptive prompts for all input.
☐ Explicit confirmation messages after every action.
☐ Consistent output format.
☐ Informative error messages.

Documentation (5%)

☐ `README.md` with setup and usage instructions.
☐ Reflection on challenges and solutions.